POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Curriculum Reinforcement Learning through Teleportation: The Teleport MDP

Tesi di Laurea Magistrale in
Computer Science and Engineering -
Ingegneria Informatica

Author: **Cristian Carmine Spagnuolo**

Student ID: 951757
Advisor: Prof. Marcello Restelli
Co-advisors: Dott. Alberto Maria Metelli, Dott. Luca Sabbioni
Academic Year: 2023-2024

"To achieve goals you have never achieved,
you need to do things you have never done."
Stephen R. Covey

Abstract

Deep Reinforcement Learning (DRL) has emerged as a powerful approach for tackling complex decision-making tasks, leading to significant advancements in robotics, game-playing, and autonomous systems. Despite these successes, DRL agents still struggle to match or surpass human performance in complex domains, i.e. navigating environments with sparse rewards, high-dimensional state-action spaces, and long-term credit assignment problems remains challenging. Curriculum Learning (CL) presents a promising technique to overcome these obstacles. Inspired by the human brain's learning process, CL proposes structuring training data to gradually increase task complexity, mirroring how humans acquire knowledge by mastering simpler concepts before tackling more difficult ones. Despite empirical evidence supporting CL's effectiveness, its wider adoption has been hindered by the complexities of curriculum design and a lack of theoretical understanding. This manuscript introduces a novel framework, the Teleport Markov Decision Process (TMDP), which extends the standard MDP model by including a teleportation probability and a teleport probability distribution for sampling the next state when a teleport occurs. The teleportation probability introduces a natural ordering of task complexity: higher values correspond to simpler tasks and vice versa, converging to the original problem when the probability is zero. Therefore, modifying the teleportation probability during learning generates an implicit curriculum, effectively combining the task sequencing and generation phases. We develop theoretical foundations by adapting the Safe Policy Iteration framework to TMDPs, establishing performance guarantees and bounds on state visit distribution shifts. Building on these results, we introduce the Teleport Model Policy Iteration (TMPI) algorithm, relevant from a theoretical perspective but with limited practical applicability. We then develop two practically relevant curriculum algorithms: Static Teleport (S-T) and Dynamic Teleport (D-T). Finally, we evaluate our Teleport-based curricula in two benchmark environments, providing empirical evidence of the effectiveness of the TMDP framework.

Keywords: Reinforcement Learning, Curriculum Learning, Dynamic Curriculum, Teleport MDP, Non-Stationary.

Abstract in lingua italiana

L'Apprendimento per Rinforzo Profondo (DRL) si è affermato come un potente approccio per la risoluzione di compiti decisionali complessi, portando a notevoli progressi in campi come la robotica, i sistemi autonomi e la robotica. Nonostante la loro popolarità, gli agenti DRL riscontrano ancora difficoltà nel superare le prestazioni umane in domini complessi, ad esempio la navigazione in ambienti con elevato numero di stati e azioni, ricompense sparse e modalità particolari di attribuzione del credito a lungo termine rimane una sfida.

L'Apprendimento Curricolare (CL) si presenta come un approccio promettente per affrontare queste problematiche. Ispirato al processo di apprendimento del cervello umano, il CL propone di ordinare, con un determinato criterio, i dati da sottoporre all'agente seguendo l'idea di aumentare gradualmente la complessità del compito. Ciò rispecchia il modo in cui gli esseri umani acquisiscono conoscenze, padroneggiando prima nozioni basilari per poi approcciare concetti più difficili. Nonostante le evidenze empiriche a sostegno dell'efficacia del CL, la sua applicazione è stata limitata dalle complessità di progettazione dei curricula e dalla mancanza di risultati teorici.

Questo manoscritto introduce un nuovo framework, denominato *Teleport Markov Decision Process* (TMDP), estendendo il modello standard degli MDP con l'aggiunta di una probabilità di teletrasporto e una distribuzione di probabilità del teletrasporto, dal quale campionare lo stato successivo in caso di teletrasporto. La probabilità di teletrasporto introduce un ordinamento naturale sulla complessità dei task: valori alti corrispondono a task più semplici e viceversa, convergendo al problema originale per valori nulli. Pertanto, modificare la probabilità di teletrasporto nel corso dell'apprendimento genera un curriculum implicito, combinando di fatto le fasi di sequenziamento e generazione dei compiti.

Adattando il framework definito come *Safe Policy Iteration* al contesto dei TMDPs, presentiamo delle basi teoriche con garanzie di miglioramento in termini di prestazioni, limitando le variazioni della distribuzione di visita degli stati.

I risultati raggiunti ci consentono di introdurre l'algoritmo *Teleport Model Policy Iteration* (TMPI), rilevante dal punto di vista teorico, meno a livello pratico. In seguito, presen-

tiamo due algoritmi curricolari con effettivo riscontro pratico: *Static Teleport* (S-T) e *Dynamic Teleport* (D-T). Questi ultimi possono essere utilizzati per arricchire alogoritmi di diversa natura, in ambito del RL, adattandoli ai curricula basati sul teletrasporto.

Infine, valutiamo i nostri algoritmi curriculari basati sul Teletrasporto in due ambienti di riferimento, fornendo evidenze empiriche dell'efficacia del framework dei TMDPs.

Ci teniamo a sottolineare che questa Tesi non si pone in alcun modo l'obiettivo di fornire una soluzione definitiva per il design di curricula efficaci ed efficienti nell'ambito del Reinforcement Learning. Al contrario, il nostro obiettivo è quello di ispirare lavori futuri a indagare più a fondo il framework proposto in modo da estenderne le proprietà e presentare delle soluzioni sempre più all'avanguardia.

Parole chiave: Apprendimento per Rinforzo, Apprendimento Curriculare, Curriculum Dinamico, MDP con Teletrasporto, Non Stazionarietà.

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

1 Introduction **1**
1.1 Thesis Outline 3

2 Machine Learning and Sequential Decision Making **5**
2.1 Sequential Decision Making 7
2.2 Markov Decision Process 9
2.2.1 Problem Definition 9
2.2.2 Trajectories and Return 11
2.2.3 Policies 11
Transition Kernels 12
2.2.4 Expected Return and Value Function 14
Bellman Equations 15
Optimality Criteria 17
2.2.5 Non-stationary MDPs 19
2.3 Dynamic Programming 20
2.3.1 Policy Evaluation 20
2.3.2 Policy Iteration 21
2.3.3 Value Iteration 22

3 Reinforcement Learning **23**
3.1 Taxonomy of Reinforcement Learning 23
3.2 Policy Evaluation for Reinforcement Learning 26
3.2.1 Monte-Carlo Evaluation 27
3.2.2 Temporal Difference Evaluation 29

3.2.3 Monte-Carlo and TD trade-off . . . 30
3.3 Value-Based Control . . . 31
3.3.1 SARSA . . . 31
Policy Evaluation . . . 31
Policy Improvement . . . 32
3.3.2 Q-learning . . . 33
3.4 Policy Search . . . 34
3.4.1 Policy Optimization and Policy Gradient Theorem . . . 35
3.4.2 Parametric Policies . . . 38
Direct Parameterization . . . 39
Softmax Policies . . . 39
3.4.3 Actor-Only Policy Gradient . . . 40
Monte Carlo Policy Gradient Theorem . . . 41
3.4.4 Actor-Critic Policy Gradient . . . 41
Generalized Advantage Estimation . . . 42
3.4.5 Safe Learning Methods . . . 43
Trust Region Methods . . . 44
Trust Region Policy Optimization . . . 44
Proximal Policy Optimization . . . 47

4 Curriculum Learning 49
4.1 Curricula in Reinforcement Learning . . . 50
4.2 Curriculum Design . . . 52
4.3 Curriculum Evaluation . . . 53
4.4 State of the Art . . . 54
4.4.1 Task Generation . . . 55
4.4.2 Sequencing . . . 56
Sample Sequencing . . . 56
Co-Learning . . . 57
State visit distribution changes . . . 58
No Restrictions . . . 58
4.4.3 Knowledge Transfer . . . 59
4.5 Open Questions . . . 60

5 Teleport MDP 61
5.1 Introduction and motivations . . . 61
5.2 Framework definition . . . 63
5.2.1 Performance Metric . . . 69

5.2.2 Advantage Functions 70

6 A Curriculum Approach for TMDPs **73**
6.1 Theoretical foundations 73
6.1.1 Bounds on state visit distribution 73
Teleport Bias 74
6.1.2 Bound on performance improvement 75
6.2 Teleport Model Policy Iteration 79
6.2.1 The algorithm 79
6.3 Static and Dynamic Teleport Curriculum 82
6.3.1 Implementation Details 86

7 Experimental Results **87**
7.1 Frozen Lake domain 87
7.1.1 Environment description 88
7.1.2 Reward Shape 88
7.1.3 Experiments 90
7.2 River Swim domain 94
7.2.1 Environment description 94
7.2.2 Experiments 95

8 Conclusions **99**
8.1 Limitation and Future Works 100

Bibliography **101**

A Proofs and Derivations **109**

B Additional Experiments **119**

List of Figures **125**

List of Tables **127**

List of Algorithms **129**

List of Symbols **131**

Acknowledgements **133**

1 | Introduction

In recent years, artificial intelligence (AI) has seen remarkable progress, largely driven by the power of Deep Neural Networks (DNNs). These models have revolutionized machine learning, excelling in tasks like computer vision, natural language processing, and even mastering complex games through Deep Reinforcement Learning (DRL). The versatility and power of DNNs have enabled machines to achieve human-level performance—and in some cases, surpass it—across a wide array of applications. Notwithstanding these impressive strides, we are still far from creating truly intelligent systems. As we venture into increasingly complex domains, we find that even our most advanced DNNs fall short of human cognition. In Reinforcement Learning (RL), sparse rewards and high-dimensional state-action spaces environments remain challenging.

Considering that neural networks draw inspiration from the human brain, researchers have sought to align their training processes with human learning methods, giving rise to Curriculum Learning (CL, [74]). Just as humans learn to crawl before walking and to add before multiplying, CL proposes structuring the training process for artificial agents in a similar, incremental manner. The core principle is simple yet impactful: by gradually increasing task complexity, we can foster more efficient and effective learning. This mirrors the way humans acquire knowledge and skills, mastering fundamental concepts before tackling more advanced ones. CL has been shown to enhance learning across various domains, offering a promising avenue to address the limitations currently faced by DNNs in complex RL tasks. Despite its intuitive appeal and empirical success, two key challenges have hindered the widespread adoption of CL. First, the design of effective curricula is a complex task, often requiring domain knowledge and significant trial and error. Second, the theoretical foundations of CL remain underdeveloped, limiting our understanding of why and how curricula work, and how to design them optimally.

At this juncture our research seeks to make its contribution, introducing the novel framework of Teleport Markov Decision Processes (TMDPs). TMDPs extend the standard Markov Decision Process (MDP) formulation by incorporating a *teleportation* mechanism that allows the relocation of the agent into any state in the middle of the episode, ef-

fectively enhancing the exploration of the state space. The teleport rate τ serves as a controller for task difficulty while the following states are drawn from the teleport probability distribution ξ. Starting with high teleport rates, the agent is frequently reset to different states. This approach simplifies the task by enhancing environmental exploration while limiting long-term foresight reducing the effective discount factor. As the agent learns and improves, the teleport probability is gradually reduced, increasing the task's complexity and converging toward the original problem formulation. In this way, the agent can exploit previously acquired knowledge from simpler tasks to boost learning progress on subsequent, more complex tasks.

Our research makes both theoretical and practical contributions. It provides a formal mathematical framework for CL, allowing for rigorous analysis and optimization. It also offers a more flexible approach to curriculum design, reducing the need for domain-specific expertise. Building on this foundation, we derive practical algorithms for real-world RL scenarios. We introduce the *Teleport Model Policy Iteration* (TMPI) algorithm, which, while primarily of theoretical interest, offers valuable insights into the properties of TMDPs. TMPI optimizes a lower bound on performance improvement by dynamically evaluating the advantages of updating the policy and/or the model and computing the step size for these updates. A significant limitation of TMPI lies in the accurate evaluation of the advantage terms. While [30] offers a potential solution for estimating the policy advantage, the model advantage estimation poses a real challenge, as it requires stepping into the field of *model-based* techniques, which is beyond the scope of this manuscript.

To overcome these constraints, we introduce two practical algorithms designed to limit state visit distribution shifts during updates: *Static Teleport* (S-T) and *Dynamic Teleport* (D-T). As the names suggest, the key distinction between these algorithms lies in the teleport rate schedule. The S-T algorithm employs a static scheduling mechanism, making it particularly suitable for off-policy learning as it does not consider the effects of policy updates. Conversely, the D-T algorithm monitors the impact of both policy and model updates, adjusting the teleport rate based on policy variations. When the policy undergoes significant changes, indicating that the agent is acquiring new knowledge, the teleport rate update is reduced or potentially skipped to maintain stability. Conversely, when the policy stabilizes, suggesting that the agent can handle increased task complexity, a reduction in the teleport rate is warranted.

We want to emphasize we do not aim to provide a definitive solution for designing curricula in the field of Reinforcement Learning. On the contrary, our goal is to inspire future work to investigate further the proposed framework, extending its properties and presenting increasingly cutting-edge solutions.

1.1. Thesis Outline

The content of this manuscript is structured as follows. Chapter 2 provides a comprehensive overview of machine learning and sequential decision-making, allowing a better understanding of the theoretical foundations of our work. We present the definition of Markov Decision Processes (MDPs), a mathematical framework for modelling decision-making problems in either deterministic or stochastic environments. We also discuss various solution methods for MDPs, including dynamic programming and reinforcement learning. Chapter 3 delves deeper into reinforcement learning, exploring different approaches to policy evaluation and control. We discuss value-based methods such as SARSA and Q-learning, as well as policy search techniques like Policy Gradient algorithms. We also examine safe learning and trust region methods, which aim to ensure that the agent's learning process is stable and robust. Chapter 4 introduces the concept of curriculum learning (CL) and discusses its application in reinforcement learning. We present the key features for designing a curriculum and some commonly used performance metrics. We conclude the analysis with an overview of the main works in the field, highlighting the challenges and open questions. Chapter 5 presents the Teleport Markov Decision Process (TMDP) framework, a novel extension of the standard MDP formulation that incorporates a teleportation mechanism. We discuss the theoretical properties of TMDPs, including performance metrics and advantage functions. Chapter 6 expands the Safe Policy Iteration framework to the context of TMDPs, providing theoretical performance guarantees under specific assumptions. We introduce a theoretical algorithm *Teleport Model Policy Iteration* (TMPI), even though it may not be directly applicable to real-world scenarios. Then, we develop two practical and adaptable algorithms named *Teleport Curriculum* (T-C) and *Dynamic Teleport* (D-T) that allow us to augment standard RL algorithms to handle a teleport-based curriculum. Chapter 7 presents the experimental results of our research along with the description of the environments. Appendix A reports the proofs and derivations omitted in the main text, while Appendix B shows all learning curves from our experiments.

In summary, this thesis presents a comprehensive study of curriculum learning in the context of reinforcement learning. By introducing the TMDP framework and associated algorithms, we provide a theoretically grounded and practically efficient approach to automate curriculum design, addressing key challenges in the field and paving the way for more adaptive and efficient learning systems.

2 | Machine Learning and Sequential Decision Making

Machine Learning (ML), a cornerstone sub-discipline of *Artificial Intelligence (AI)*, focuses on the development of algorithms that can learn from data without explicit programming. These algorithms can operate independently or require human oversight, aiming to improve efficiency and effectiveness across a broad spectrum of applications. A widely adopted definition is the one provided by [35]:

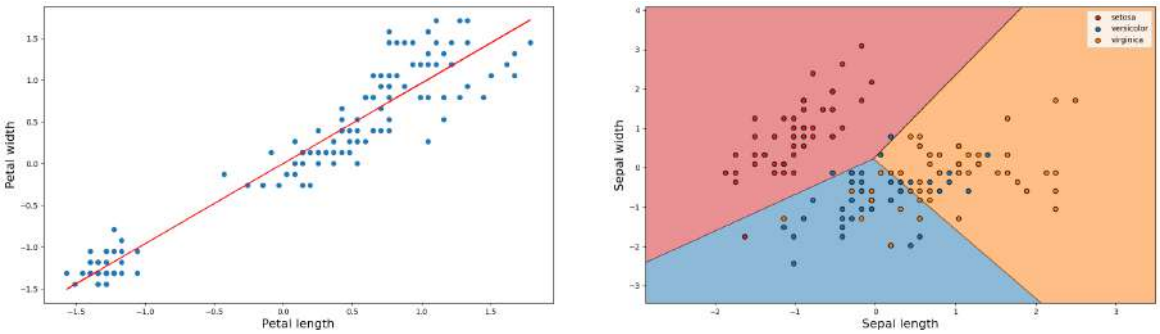
Definition 2.1. *A computer program is said to learn from experience* E *for some class of task* T *and a performance measure* P *if its performance at tasks in* T*, measured by* P*, improves with experience* E*.*

This definition introduces three key concepts:

- **Task (T)**: specifies the problem domain or set of problems the program aims to solve. This could range from image recognition to natural language processing.
- **Experience (E)**: refers to the data or interactions the program has been exposed to. This could be historical data, ongoing interactions, or any form of input the program processes and learns from.
- **Performance Measure (P)**: the metric used to assess how effectively the program performs the given tasks, setting up the basis for optimization.

The field of machine learning encompasses three primary domains:

- **Supervised Learning** involves learning from a dataset $D = \{(x_i, y_i) \mid i = 1, 2, \ldots, n\}$, where each tuple (x_i, y_i) consists of an input feature vector x_i and a corresponding target output y_i. The objective is to discover a function of arbitrary complexity that can accurately predict the target y_i based on the input features x_i.



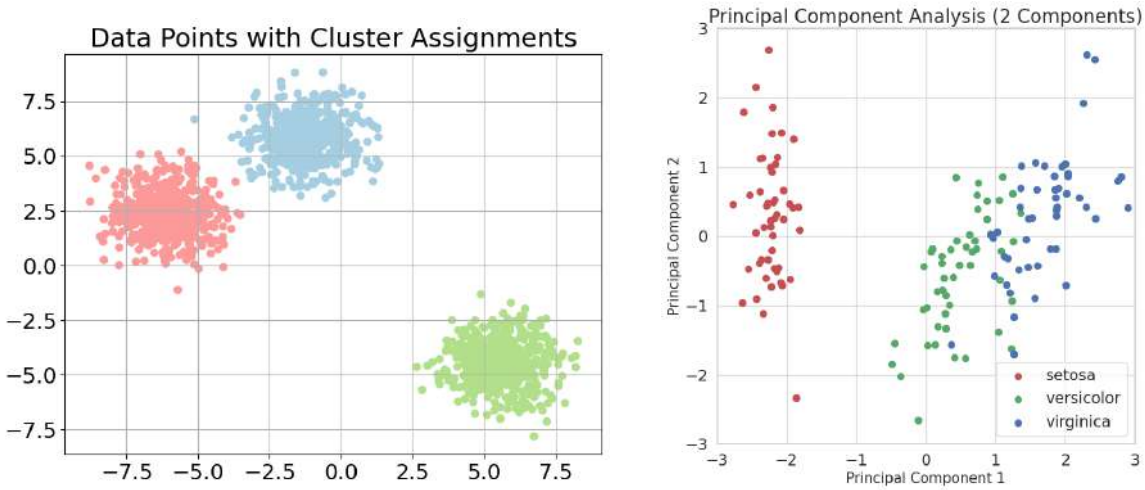
(a) An example of Regression: taking the length of a petal, the model predicts its width.

(b) An example of Classification: given the length and the width of the sepal, each flower is classified in one of the three different species

Figure 2.1: Supervised Learning mainly addresses the challenges of Regression (a), and Classification (b). Specifically, regression aims to predict a continuous numerical value as the output, while classification focuses on determining the category to which a given sample belongs.

- **Unsupervised Learning** tackles scenarios where the data is unlabeled, meaning it consists of a collection of samples $D = \{x_i \mid i = 1, 2, \ldots, n\}$. The objective is to uncover hidden patterns or structures within the data. This can be achieved through various approaches, such as dimensionality reduction or data clustering, aiming to provide a more meaningful representation of the data. The key distinction from supervised learning lies in the absence of a predefined target output y_i, from here the name *unsupervised*.

- **Reinforcement Learning (RL)**: operates on a dataset $D = \{(s_i, a_i, s_i', r_i) \mid i = 1, 2, \ldots, n\}$ consisting of 4-tuples (s_i, a_i, s_i', r_i), where s_i denotes the current state, a_i the action taken, s_i' the resultant next state and r_i the immediate reward received. The objective is to discover an optimal policy, $\pi^\star$, that dictates the best action to select in each state to maximize the cumulative reward obtained through the interaction between an agent and a dynamic environment. This setup is typically framed as a *Markov Decision Process* (MDP), extensively discussed in Section 2.2. Reinforcement Learning is the central focus of this manuscript. Drawing inspiration from [68], this and the following chapters will provide the background knowledge for a comprehensive understanding of this document.



(a) An example of Clustering: data points are grouped into distinct categories based on a chosen similarity measure. This allows us to identify underlying patterns or structures within the data.

(b) An example of Dimensionality Reduction: considering a dataset with four features, by applying PCA, we can project them onto a lower-dimensional space, still capturing a high percentage of the original variance in the data, 96% in this case.

Figure 2.2: Unsupervised Learning tackles the problems of Clustering (a), grouping data into similar classes and Dimensionality Reduction (b), mapping them from a higher dimensional input space to a lower dimensional one.

2.1. Sequential Decision Making

Reinforcement Learning seeks to identify the optimal sequence of actions that will maximize cumulative numerical rewards within a given environment. The learning entity, or agent, discerns these choices through environmental interaction, following a trial-and-error approach. The challenge intensifies because each action influences immediate outcomes and carries lasting consequences for future observations, introducing the complexity of delayed reward assessment. Consequently, to optimize total rewards, the agent may need to prioritize actions that yield modest benefits in the near term but pave the way for more substantial rewards later on.

A pivotal challenge in RL is the *exploration* vs. *exploitation* dilemma. Unlike *supervised learning*, where the aim is to generalize from a training set to perform accurately in new scenarios, RL involves learning by directly interacting with the environment and observing the outcomes of actions through collected rewards. This necessitates a careful balance between exploring various state-action pairs to understand the environment and exploiting the acquired knowledge to maximize rewards. A common strategy employed by several RL algorithms is to start emphasizing exploration, to assess the impact of each action.

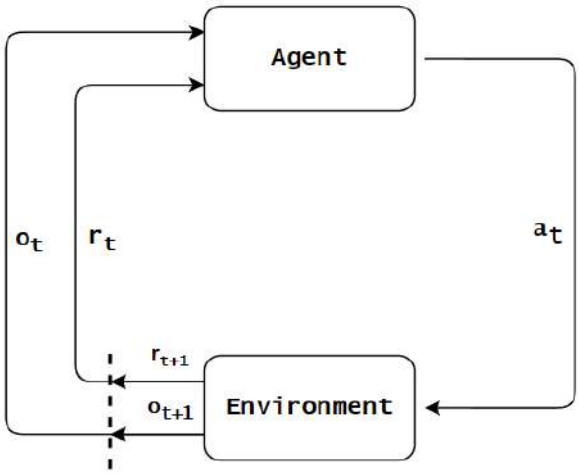


Figure 2.3: Interaction between an agent and the environment, at time t.

As learning progresses, the focus gradually shifts towards exploiting actions that appear to provide higher cumulative rewards.

The whole process can be modelled as a *Sequential Decision Making* problem, whose structure is shown in Figure 2.3. At each time step t, the learning *agent* receives a representation of the *environment's* state, o_t, and a reward signal r_t. On that basis, it decides the action to execute a_t, which in turn generates a change in the environment, o_{t+1}, with the associated reward r_{t+1}. The interaction goes on until a *terminal* state is reached.

This chain of interactions generates a *history*, h_t, defined as a sequence of tuples, action a_t, observation o_t, reward r_t, for each time step t:

$$h_t = \{(a_1, o_1, r_1), (a_2, o_2, r_2), \ldots, (a_t, o_t, r_t)\}. \tag{2.1}$$

Based on the history, h_t, the agent will decide the next action and consequently, the environment will evolve, generating the following observation and reward. Typically, the agent extrapolates from the history the current *state*, formally:

$$s_t = f(h_t). \tag{2.2}$$

To evolve, the environment follows its internal state representation, s_t^e, generating the observation-reward pair. In contrast, the agent constructs its representation, s_t^a, processing the feedback of the environment. In general, the two representations do not align perfectly. This mismatch gives rise to *Partially Observable* problems, where the agent

is incapable of producing an exact replication of the environmental state. Such scenarios necessitate a probabilistic approach, considering distributions over all feasible states. However, in many RL scenarios, a critical yet restrictive assumption is made: the environment is treated as *Fully Observable*, formally:

$$o_t = s_t^e = s_t^a.$$

From now on, we will operate under the assumption of *full observability*. Beyond this, RL environments can be further categorized along several dimensions, summarized as follows:

- **Stationary vs. Non-stationary**: in a stationary environment, any change is directly determined by actions taken by the agent. Conversely, a non-stationary environment may evolve independently of the agent's behaviour, possibly due to external factors or inherent dynamics unrelated to the agent's interactions.
- **Deterministic vs. Stochastic**: A deterministic environment ensures that any action results in a predictable outcome. In contrast, outcomes in a stochastic environment are subject to variability and can be best described using probability distributions.

The power of RL algorithms lies in adaptability, enabling them to address these problems without needing explicit definitions of environmental dynamics. Their goal is to learn through exploration, devising strategies that optimize cumulative rewards based on experiential learning.

2.2. Markov Decision Process

This section introduces a fundamental mathematical framework, known as *Markov Decision Process* (MDP, [50]), used to model a wide range of reinforcement learning problems.

2.2.1. Problem Definition

Definition 2.2 (Markov Property). *A stochastic process X_t, is called Markovian if:*

$$\mathbb{P}(X_{t+1} = j \mid X_t = i, X_{t-1} = k_{t-1}, \ldots, X_1 = k_1, X_0 = k_0) = \mathbb{P}(X_{t+1} = j \mid X_t = i). \quad (2.3)$$

Focusing on Sequential Decision Making (SDM) problems, the state signal is considered a stochastic process S_t, with a certain probability distribution defined over all states given

the history:

$$\mathbb{P}(S_{t+1} = s' \mid A_t, S_t, A_{t-1}, S_{t-1}, \ldots, A_1, S_1, A_0, S_0) = \mathbb{P}(S_{t+1} = s' \mid A_t, S_t). \tag{2.4}$$

The probabilities $\mathbb{P}(S_{t+1} = s' \mid A_t, S_t)$, denoted as *transition probabilities*, typically follow the *stationary* assumption.

Definition 2.3 (Stationary Transition Probabilities)**.** *Transition probabilities in a Markovian stochastic process X_t are said to be stationary if they do not change in time, formally:*

$$\mathbb{P}(X_{t+1} = j \mid X_t = i) = \mathbb{P}(X_1 = j \mid X_1 = i) = p_{i,j}. \tag{2.5}$$

Adopting stationary transition probabilities simplifies the problem's space and time complexities. Moreover, assuming that the current state fully represents the history, allows deriving useful theoretical insights and results.

A fully observable RL problem, following the Markov property is called a *Markov Decision Process (MDP)* and can be formally described as follows:

Definition 2.4 (Markov Decision Process)**.** *A Markov Decision Process (MDP) is a Markovian stochastic process defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma, \mu \rangle$ where:*

- *$\mathcal{S}$ is referred to as state space, representing the finite set of all states;*
- *$\mathcal{A}$ is denoted as action space, identifying the finite set of all actions;*
- *P is the stationary transition probability matrix. Each element, $P(s'|s,a)$, expresses the probability of moving from state s, to state s' when action a is selected;*
- *R is known as the reward function $R(s,a) = \mathbb{E}[r|s,a]$. Given a state-action pair (s,a), it returns the expected value of the stochastic reward r[1];*
- *$\gamma \in [0,1]$ is the discount factor. It is a property of the problem allowing to weights future;*
- *μ describes the initial state distribution. Formally $\mu_s = P(s_0 = s)$, $\forall s \in \mathcal{S}$.*

[1] In the general case, the reward is considered to be stochastic, meaning that choosing the same action in the same state does not necessarily lead to the same reward.

2.2.2. Trajectories and Return

The objective in reinforcement learning is to maximize the *return*, composed of two key components: the *time horizon*[2], denoted by T, stating the number of time steps until an episode ends, and the *cumulative reward*, defined as the sums of immediate rewards received during each timestep interaction.
Outlining a *trajectory*, $\nu = h_T$, as the complete history of an agent's interaction with the environment up to a specific time horizon T and denoting the trajectory's length by $T(\nu)$, we can introduce the *return function* $G_\gamma(\nu)$ as the discounted cumulative sum of rewards across a trajectory ν:

$$G_\gamma(\nu) := \sum_{t=0}^{T(\nu)} \gamma^t r_t, \tag{2.6}$$

where γ is the discount factor. Digressing briefly, we can explore the discount factor through various lenses. Economically, it models the preference for immediate rewards over future gains. Lower values of γ accentuate immediate outcomes, denoting a short-term perspective, whereas higher values endorse a long-term approach. From a mathematical standpoint, γ also introduces convergence properties for trajectories of infinite length. Assuming the existence of minimum and maximum immediate rewards, $R_{\min}$ and $R_{\max}$ respectively, the return function can be bounded as follows:

$$\frac{R_{\min}}{1-\gamma} \leq G_\gamma(\nu) \leq \frac{R_{\max}}{1-\gamma}.$$

2.2.3. Policies

With the problem framework and objectives established, we now turn to formalizing the agent's decision-making mechanism through the concept of *policies*.

Definition 2.5 (Policy). *A policy, $\pi \in \Pi$, is a mapping $\pi : \mathcal{S} \times \mathcal{A} \to [0,1]$, assigning to each state $s \in \mathcal{S}$ a probability distribution over the set of all possible actions $\mathcal{A}$, such that:*

$$\pi(a|s) = \mathbb{P}(a|s), \quad \forall a \in \mathcal{A}, \forall s \in \mathcal{S}. \tag{2.7}$$

Remark 2.1. *For continuous action spaces $\mathcal{A}$, the policy $\pi(s)$ is generalized to a probability density function over actions within $\mathcal{A}$.*

Within the scope of MDPs, the analysis can be streamlined to focus on policies that

[2]The time horizon can either be finite ($T < \infty$) or infinite ($T = \infty$).

are both *stationary* and *Markovian*. It has been proved that for any given MDP, an optimal policy exists that is simultaneously *stationary*, *Markovian*, and *deterministic*. Nonetheless, *stochasticity* is essential for fostering exploration, which is critical during the learning phase and would be prevented by a purely deterministic approach.

Moreover, it is possible to quantify the distribution of the trajectories, $\rho^\pi(\nu)$, resulting from the agent's interactions with the environment under a stationary and Markovian policy π, as:

$$\rho^\pi(\nu) \coloneqq \mu(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) P(s_{t+1}|s_t, a_t). \tag{2.8}$$

Transition Kernels

The combination of an MDP $\mathcal{M}$ with a policy π gives birth to a Markov Chain (MC, [42]) over the state space $\mathcal{S}$, evolving following the transition kernel P^π:

$$P^\pi(\cdot|s) = \sum_{a \in \mathcal{A}} \pi(a|s) P(\cdot|s, a). \tag{2.9}$$

Considering t consequent steps, starting from s and following the policy π, the transition kernel representing the probability distribution on the state space $\mathcal{S}$, for the MC can be recursively defined as follows:

$$\begin{aligned} P^{\pi^1}(\cdot|s) &\coloneqq P^\pi(\cdot|s); \\ P^{\pi^{t+1}}(\cdot|s) &\coloneqq \sum_{s' \in \mathcal{S}} P^{\pi^t}(s'|s) P^\pi(\cdot|s'). \end{aligned} \tag{2.10}$$

Informally, $P^{\pi^t}(s'|s)$, represents the probability of reaching s', starting from s, in t-steps, following π. Building upon this logic, it is possible to define the γ-discounted *state-occupancy* measure $d^\pi_{s_0}$ as:

$$d^\pi_{s_0}(\cdot) \coloneqq \frac{1-\gamma}{\gamma} \sum_{t=1}^{\infty} \gamma^t P^{\pi^t}(\cdot|s_0). \tag{2.11}$$

Or, leveraging its recursive structure:

$$d^\pi_{s_0}(\cdot) = (1-\gamma) P^\pi(\cdot|s_0) + \gamma \sum_{s \in S} d^\pi_{s_0}(s) P^\pi(\cdot|s) \tag{2.12}$$

Proof.

$$\begin{aligned}
\gamma \sum_{s \in S} d^{\pi}_{s_0}(s) P^{\pi}(\cdot|s) &= (1-\gamma) \sum_{s \in S} \sum_{t=1}^{\infty} \gamma^t P^{\pi^t}(s|s_0) P^{\pi}(\cdot|s) \\
&= (1-\gamma) \sum_{t=1}^{\infty} \gamma^t \sum_{s \in S} P^{\pi^t}(s|s_0) P^{\pi}(\cdot|s) \\
&= \frac{(1-\gamma)}{\gamma} \sum_{t=1}^{\infty} \gamma^{t+1} P^{\pi^{t+1}}(\cdot|s_0) \\
&= \frac{(1-\gamma)}{\gamma} \sum_{t=2}^{\infty} \gamma^t P^{\pi^t}(\cdot|s_0) \\
&= \frac{(1-\gamma)}{\gamma} \sum_{t=1}^{\infty} \gamma^t P^{\pi^t}(\cdot|s_0) - (1-\gamma) P^{\pi}(\cdot|s_0) \\
&= d^{\pi}_{s_0}(\cdot) - (1-\gamma) P^{\pi}(\cdot|s_0).
\end{aligned}$$

□

Intuitively, $d^{\pi}_{s_0}(s')$ quantifies the probability of reaching a given state s', when starting from state s_0 and following policy π over an unspecified number of steps. To consider the overall distribution over the state space during the interaction, we must also consider the initial state distribution μ:

$$\begin{aligned}
d^{\pi}_{\mu}(s) &\coloneqq (1-\gamma)\mu(s) + \gamma \sum_{s_0 \in \mathcal{S}} \mu(s_0) d^{\pi}_{s_0}(s) && (2.13) \\
&= (1-\gamma)\mu(s) + \gamma \sum_{s_0 \in \mathcal{S}} \mu(s_0) \Big((1-\gamma) P^{\pi}(s|s_0) + \gamma \sum_{s' \in S} d^{\pi}_{s_0}(s') P^{\pi}(s|s') \Big) \\
&= (1-\gamma)\mu(s) + \gamma \sum_{s' \in \mathcal{S}} \Big((1-\gamma)\mu(s') + \gamma \sum_{s_0 \in S} \mu(s_0) d^{\pi}_{s_0}(s') \Big) P^{\pi}(s|s') \\
&= (1-\gamma)\mu(s) + \gamma \sum_{s' \in \mathcal{S}} d^{\pi}_{\mu}(s') P^{\pi}(s|s') && (2.14)
\end{aligned}$$

The probability of reaching a certain state at some point during an interaction can be intuitively broken down into two terms. The former, considers the likelihood of starting from that state, while the latter evaluates the probability of reaching that state from one of the initial ones. Based on this, the distribution over the state-action pairs can be evaluated as follows:

$$\delta^{\pi}_{\mu}(s,a) \coloneqq \pi(a|s)\, d^{\pi}_{\mu}(s), \quad \forall (s,a) \in \mathcal{S} \times \mathcal{A}. \tag{2.15}$$

2.2.4. Expected Return and Value Function

In most Reinforcement Learning scenarios, the objective is to maximize the *expected* return, accounting for the stochasticity of either the environment, the policy, or both. Therefore, defined a policy-dependent performance index, $J(\pi)$, the *objective function* can be encoded as:

$$\max_{\pi \in \Pi} J(\pi) \coloneqq \mathop{\mathbb{E}}_{\nu \sim \rho^\pi} \left[G_\gamma(\nu) \right] = \mathop{\mathbb{E}}_{\substack{s_0 \sim \mu \\ a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim P(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \tag{2.16}$$

Several algorithms consider the *value function*, which can be seen as an estimation of how good is to be in a state s, given a policy π.

Definition 2.6 (Value Function)**.** *The value function $V^\pi : \mathcal{S} \to \mathbb{R}$ is defined as the expected return starting from state s, and following policy π:*

$$V^\pi(s) \coloneqq \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim P(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \middle| s_0 = s \right] = \mathop{\mathbb{E}}_{\nu \sim \rho^\pi} \left[G_\gamma(\nu) | s_0 = s \right]. \tag{2.17}$$

Measuring the quality of each action in each state can be more useful. Based on this rationale, it is possible to define the action-value function or Q-function.

Definition 2.7 (Action-Value Function)**.** *The action-value function $Q^\pi : \mathcal{S} \to \mathbb{R}$ is the expected return starting from state s, choosing action a and then following policy π.*

$$Q^\pi(s, a) \coloneqq \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim P(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \middle| s_0 = s, a_0 = a \right]. \tag{2.18}$$

There is a core distinction between these two measures. The value function, $V(s)$, is instrumental for *assessing* the overall effectiveness of a policy π, providing an integrated view of the expected returns from a given state under π. Conversely, the action-value function, $Q^\pi(s, a)$, is pivotal for *policy improvement*. It enables the comparison of the expected returns associated with different actions for each state, thus allowing the identification of promising actions in terms of potential reward.

Consequently, the introduction of the *advantage function*, denoted as $A^\pi(s, a)$, assumes relevance. This function quantifies the expected increase in return when deviating from

the policy's prescribed action in state s, choosing instead action a. Formally:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \tag{2.19}$$

To summarize, the expected performance of a policy, represented by $J(\pi)$ or J^π, quantifies the score of a policy π considering all potential starting states. This metric can be precisely defined in terms of the state-value function $V^\pi(s)$ by marginalizing over the initial state distribution $\mu(s)$, as follows:

$$J(\pi) = \mathop{\mathbb{E}}_{s\sim\mu}\left[V^\pi(s)\right] = \sum_{s\in\mathcal{S}} \mu(s)V^\pi(s) \tag{2.20}$$

Bellman Equations

The value function definitions are formulated through the expected returns from the trajectories generated by interactions between the agent and the environment. The Bellman Equations [7], offer a recursive decomposition of the decision-making process from the Markov property of the transition dynamics.

Theorem 2.1 (Bellman Expectation Equation for Value Function)**.** *The value function can be decomposed into immediate reward plus discounted value of the next state:*

$$\begin{aligned} V^\pi(s) &= \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s'\in\mathcal{S}} P(s'|s,a)\big[R(s,a,s') + \gamma V^\pi(s')\big] = \\ &= \sum_{a\in\mathcal{A}} \pi(a|s) \underbrace{\Big[R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a)V^\pi(s') \Big]}_{Q^\pi(s,a)} \end{aligned} \tag{2.21}$$

This equation incorporates the probabilities of taking certain actions, transitioning between states, and accumulating rewards over time. In essence, this recursive formulation comprises two distinct parts. The first, is the expected immediate reward for taking action a in state s, weighted by the probability of having that state-action pair under policy π. The second represents the discounted long-term value of being in state s. It is calculated by considering all possible subsequent states that could be reached from s after taking action a and discounting by γ. The same principle can be applied as well to the action-value function.

Theorem 2.2 (Bellman Expectation Equation for Action-Value Function)**.** *The action-value function can be decomposed as the immediate reward plus the discounted value of*

the next state-action pair:

$$
\begin{aligned}
Q^\pi(s,a) &= \sum_{s'\in\mathcal{S}} P(s'|s,a)\Big[R(s,a,s') + \gamma \sum_{a'\in\mathcal{A}} \pi(a'|s')Q^\pi(s',a')\Big] \\
&= R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a) \underbrace{\sum_{a'\in\mathcal{A}} \pi(a'|s')Q^\pi(s',a')}_{V^\pi(s')} && (2.22)\\
&= R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a)V^\pi(s') && (2.23)
\end{aligned}
$$

The Q-function extends the value function by considering the immediate reward from the first action, followed by future rewards as the policy dictates.

Bellman equations provide a mathematical framework for evaluating and optimizing policies, thus enabling the systematic approach to solve decision-making problems under uncertainty. By introducing two operators, we can derive valuable properties.

Definition 2.8 (Bellman Expectation Operator for Value function)**.** *The Bellman expectation operator for state value functions is the linear operator $T^\pi : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$, such that:*

$$(T^\pi V^\pi)(s) = \sum_{a\in\mathcal{A}} \pi(a|s)\Big[R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a)V^\pi(s')\Big] \tag{2.24}$$

Definition 2.9 (Bellman Expectation Operator for Action-Value function)**.** *The Bellman expectation operator for state-action value functions is the linear operator $T^\pi : \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|} \to \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$, such that:*

$$(T^\pi Q^\pi)(s,a) = R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a) \sum_{a'\in\mathcal{A}} \pi(a'|s')Q^\pi(s',a') \tag{2.25}$$

Bellman expectation operators transform value functions into subsequent value functions through the recursive principles defined by the Bellman equation. This allows to express the Bellman equations in vector form as follows:

$$T^\pi V^\pi = V^\pi; \tag{2.26}$$

$$T^\pi Q^\pi = Q^\pi. \tag{2.27}$$

Optimality Criteria

As highlighted in Section 2.2.4, our primary objective is to maximize the expected return. In practice, however, it is often more convenient to establish optimality criteria in terms of value functions.

Definition 2.10 (Optimal Value Function). *The optimal state-value-function $V^\star(s)$ is the maximum value function over all policies:*

$$V^\star(s) = \max_{\pi \in \Pi} V^\pi(s). \tag{2.28}$$

Definition 2.11 (Optimal Action-Value Function). *The optimal action-value-function $Q^\star(s,a)$ is the maximum action-value function over all policies:*

$$Q^\star(s,a) = \max_{\pi \in \Pi} Q^\pi(s,a). \tag{2.29}$$

The relevance of these two value functions arises from their alignment with the optimal policy, assessing the highest achievable value for each state within the MDP. Knowledge of the optimal value function and its associated policy means solving the MDP. Furthermore, value functions introduce a hierarchical structure over policies based on their performance, whereby:

$$V^\pi(s) \geq V^{\pi'}(s) \quad \forall s \in \mathcal{S} \implies \pi \geq \pi'.$$

Given this premise, we can characterize the *optimal policy* $\pi^\star$, as one that consistently outperforms or matches every other policy in terms of outcomes. While an MDP may have multiple optimal policies, they all converge on identical optimal value functions, $V^\star(s)$ and $Q^\star(s,a)$. The following theorem underlines the properties of optimal policies:

Theorem 2.3 (MDP Optimality). *For any Markov Decision Process:*

- *There always exists an optimal policy $\pi^\star$, that is better or equal than any other policy: $\pi^\star \geq \pi$, $\forall \pi \in \Pi$.*
- *All optimal policies share the optimal state-value function: $V^{\pi^\star}(s) = V^\star(s)$, $\forall s \in \mathcal{S}$.*
- *All optimal policies share the optimal action-value function: $Q^{\pi^\star}(s,a) = Q^\star(s,a)$, $\forall s \in \mathcal{S}, \forall a \in \mathcal{A}$.*
- *There always exists a* deterministic *optimal policy.*

By maximizing the expected future reward, $Q^\star(s,a)$, for all states s and actions a, a

deterministic optimal policy can be identified:

$$\pi^\star(a|s) = \begin{cases} 1, & if\ a \in \arg\max_{a' \in A} Q^\star(s, a'); \\ 0, & otherwise. \end{cases} \tag{2.30}$$

This outcome is particularly relevant as it confirms the existence of a deterministic optimal policy which is both simple to implement and easy to derive from the optimal action-value function by choosing the highest valued action. As shown in Theorems 2.1 and 2.2, optimal value functions can be described through distinct Bellman equations.

Definition 2.12 (Bellman Optimality Equation for Value Function)**.** *The Bellman optimality equation for the optimal state-value function is:*

$$\begin{aligned} V^\star(s) &= \max_{a \in \mathcal{A}} Q^*(s, a) \\ &= \max_{a \in \mathcal{A}} \left\{ R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\star(s') \right\}. \end{aligned} \tag{2.31}$$

Definition 2.13 (Bellman Optimality Equation for Action-Value Function)**.** *The Bellman optimality equation for the optimal action-value function is:*

$$\begin{aligned} Q^\star(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^*(s') \\ &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_{a' \in \mathcal{A}} Q^\star(s, a') \end{aligned} \tag{2.32}$$

In line with the approach used for the Bellman expectation equations, the optimality equations can be similarly expressed in operator form.

Definition 2.14 (Bellman Optimality Operator for Value Function)**.** *The Bellman optimality operator for the optimal state-value function is the non-linear operator, defined as* $T^\star : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$*, such that:*

$$(T^* V^*)(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\star(s') \right] \tag{2.33}$$

Definition 2.15 (Bellman Optimality Operator for Action-Value Function)**.** *The Bellman optimality operator for the optimal action-value function is the non-linear operator, defined as* $T^\star : \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|} \to \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$*, such that:*

$$(T^\star Q^\star)(s,a) = R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a) \max_{a'\in\mathcal{A}} Q^\star(s',a') \tag{2.34}$$

A key distinction lies between Bellman expectation and optimality operators. Expectation operators are inherently linear, allowing for the derivation of closed-form solutions. In contrast, optimality operators are *nonlinear*, meaning they lack closed-form solutions and entail alternative approaches, i.e. numerical methods, for their resolution.
Notably, all operators exhibit contractive behaviour, where their fixed point represents a value function. This property enables the application of the Banach-Caccioppoli fixed-point theorem [4], allowing to reach the fixed point from any initial vector $f \in \mathbb{R}^{|\mathcal{S}|}$:

$$\lim_{k\to\infty} (T^\pi)^k f = V^\pi \tag{2.35}$$

$$\lim_{k\to\infty} (T^\star)^k f = V^\star \tag{2.36}$$

Given the iterative nature of this approach, determining when a sufficiently accurate approximation has been reached is crucial. To this end, we can quantify the deviation between successive approximations at iteration k, denoted as $(T^\star)^k f$, and its predecessor, using the metric $\epsilon = \|(T^\star)^k f - (T^\star)^{k-1} f\|_\infty$. Then, by monitoring the value of ϵ across iterations, we can identify when the change falls below a predefined threshold, indicating convergence to a good approximation of the real value function. The distance between the optimal value function $V^\star$ and f can be bounded as:

$$\|V^\star - f\|_\infty = \frac{2\gamma\epsilon}{1-\gamma}$$

This principle applies also to the action value function.

2.2.5. Non-stationary MDPs

To suit our goals a brief exploration of *Non-Stationary* MDP can be meaningful. Unlike standard MDPs, which assume fixed transition probabilities and reward functions, Non-Stationary MDPs allow these components to change as the agent interacts with the environment [12]. The probability transition along with the reward functions can be

reformulated in a time-dependent fashion:

$$P(s'|s,a) = P^t(s_{t+1} = s'|s_t = s, a_t = a),$$
$$R(s,a,s') = R^t(s,a) = \mathbb{E}[r_t|s_t = s, a_t = a].$$

The main challenge lies in adapting to these changes. Agents must balance recent experiences with historical data. As we will see in the following chapters, the TMDP framework falls into this category.

2.3. Dynamic Programming

In a finite MDP, assuming complete knowledge of both the transition kernel P and the reward model R, we can apply *dynamic programming* (DP), to identify an optimal policy, as outlined by [9]. DP algorithms transform the Bellman Equation into a set of iterative update rules, leveraging its contraction properties to progress steadily towards an optimal solution.

2.3.1. Policy Evaluation

Given a policy π, *policy evaluation* is a *prediction task* aiming to compute the value function V^π. To this end, the Bellman Expectation Operator, (Definition 2.8) is iteratively applied: at each iteration $k = 1, 2, \ldots$ the following value function is computed as:

$$V_{k+1} = T^\pi V_k. \tag{2.37}$$

The algorithm goes on until $\|V_k - V_{k-1}\| \leq \epsilon(1-\gamma)$, in other words, an ϵ-approximation of the unique fixed point of Bellman Equation, nominally V^π is found:

$$\|V_k - V_{k-1}\|_\infty \leq \epsilon(1-\gamma) \Longrightarrow \|V_k - V^\pi\|_\infty \leq \epsilon$$

Additionally, the linear nature of expectation equations allows also the derivation of a closed-form solution.

$$\begin{aligned} V^\pi &= R^\pi + \gamma P^\pi V^\pi \\ &= (I - \gamma P^\pi)^{-1} R^\pi, \end{aligned} \tag{2.38}$$

Note. *Leveraging the characteristics of stochastic matrices, the inversion is always feasible for $\gamma \in (0,1)$. The stochastic nature of the matrix P^π, guarantees its eigenvalues,*

$\lambda_i^{P^\pi}$, fall within the interval $[-1, 1]$. Consequently, defining $A = I - \gamma P^\pi$, the eigenvalues of A, are expressed as $\lambda_i^A = 1 - \gamma \lambda_i^{P^\pi}$. Given $\gamma < 1$, it follows that $\lambda_i^A > 0, \forall i$, proving that matrix A is positive-definite and thus invertible. Following a similar reasoning, we can also compute Q^π.

2.3.2. Policy Iteration

Moving the focus from prediction to *control*, dynamic programming allows us to compute the optimal policy. Given any initial policy π_0, *policy iteration* (PI), iterates over the two following steps:

1. Policy evaluation: prediction task aiming to compute Q^{π_k}, from π_k.
2. Policy improvement: control task extracting $\pi_{k+1}(s) \leftarrow \arg\max_a Q^{\pi_k}(sa)$.

Section 2.3.1 describes the policy evaluation step. Instead, the *policy improvement* step returns a greedy policy w.r.t. Q^{π_k}. This process results in a non-decreasing performance improvement for each complete iteration of PI. Specifically, the following outcome is described in [68]:

Theorem 2.4 (Policy Improvement Theorem). *Let $\mathcal{M}$ be an MDP. Let $\pi, \bar{\pi} \in \Pi$ be two stationary, Markovian policies. If holds that*

$$\sum_{a \in \mathcal{A}} \bar{\pi}(a|s) Q^\pi(s, a) \geq V^\pi(s), \quad \forall s \in \mathcal{S};$$

then:

$$V^{\bar{\pi}}(s) \geq V^\pi(s), \quad \forall s \in \mathcal{S}.$$

The condition is implicitly satisfied by construction for any greedy policy, as shown by [50]. Furthermore, if the difference between the value function of the greedy policy, $V^{\bar{\pi}}$ and that of its predecessor, V^π, does not exceed a specified threshold, i.e., $\|V^{\bar{\pi}} - V^\pi\|_\infty \leq \epsilon$, then $V^{\bar{\pi}}$ qualifies as an ϵ-approximation for the fixed point $V^\star$ of the Bellman Optimal Equation.

Concerning the convergence characteristics, it is noteworthy that the Policy Iteration method exhibits a quadratic convergence rate [33]. Additionally, given the finite nature of the MDP and that the total number of deterministic policies is upper bounded by $|S|^{|A|}$, it can be demonstrated that PI is guaranteed to converge to the optimal policy after a finite sequence of iterations[57].

2.3.3. Value Iteration

Policy Iteration is computationally expensive. At each iteration, the presence of a policy evaluation step slows down the process. To overcome this limitation, an alternative approach has been introduced. *Value Iteration* (VI), implements an iterative application of Bellman Optimality Operator $T^\star$ (Definition 2.14). Starting from an arbitrary value function V_0, the following one is computed as follows:

$$V_{k+1} = T^\star V_k. \tag{2.39}$$

Assuming we want a maximum deviation between the optimal value function $V^\star$ and our approximation V_k, we can define a threshold $\epsilon = \|V^\star - V_{k+1}\|_\infty$ and stop the algorithm when:

$$\|V_{k+1} - V_k\|_\infty = \frac{\epsilon(1-\gamma)}{2\gamma}. \tag{2.40}$$

Then, considering T as the total number of iterations, the greedy policy $\bar{\pi}$, associated with the last value function, V_T is retrieved as:

$$\bar{\pi}(s) \in \arg\max_{a \in \mathcal{A}} \left(R(s,a) + \gamma \mathop{\mathbb{E}}_{s' \sim P(\cdot|s,a)} [V_T(s')] \right) \tag{2.41}$$

See [50], for complete proof of convergence.

In conclusion, this chapter has provided an overview of the components of Reinforcement Learning problems. We have introduced the Markov Decision Process (MDP) framework, clarified the objective of the agent as maximizing the (discounted) cumulative reward, and defined the desired outcome of an algorithm in terms of optimal policy. Additionally, we have extensively discussed value functions, as they form the foundation for many RL algorithms. With these fundamental concepts established, we will now proceed to explore the algorithms designed to solve real-world RL problems.

3 | Reinforcement Learning

While Dynamic Programming, introduced in Section 2.3, is a powerful mathematical technique for solving control problems by breaking them down into simpler sub-problems, its real-world applicability is limited. Assuming a complete understanding of the MDP is ambitious, especially in real-world contexts where the dynamics of the process might be unknown, contiguous or too huge for memory storage. Reinforcement learning algorithms, widely inspired by DP approaches, provide a solution without requiring a pre-defined model. An RL agent actively interacts with the environment, learning from received rewards to refine its decision-making policy. This trial-and-error approach allows RL to adaptively discover optimal behaviour in complex, stochastic, or partially known environments. This key advantage makes RL well-suited for domains like autonomous systems, robotics, and game playing, where sampling from the environment is essential to build statistical models for prediction and control purposes.

In environments characterized by uncertainty and stochasticity, reinforcement learning agents must face the exploration-exploitation dilemma. This trade-off involves extensively exploring the states and action spaces to gather sufficient knowledge about the environment's dynamics and rewards. Effectively managing this balance is crucial for optimal performance.

3.1. Taxonomy of Reinforcement Learning

This section presents a taxonomy of existing Reinforcement Learning algorithms. We categorize them based on several criteria. These categories are independent and do not claim to be exhaustive. We acknowledge that not all RL algorithms can be neatly classified along proposed dimensions.

- **Objective**. The distinction already mentioned in the context of Dynamic Programming appears again. There are mainly two types of tasks:

  - *Prediction.* The focus here is on assessing the effectiveness of a given policy. This involves estimating the value functions corresponding to that policy, essentially performing the Policy Evaluation process detailed in section 2.3.1.
  - *Control.* The control objective seeks to optimize the decision-making process within the MDP framework by identifying the optimal policy. This can be achieved following policy iteration or value iteration, as baseline.

  Despite evaluating the value function is a pivotal concept in RL, the main focus of this manuscript is on optimization tasks.

- **Model Estimation**. Without complete knowledge of the Markov Decision Process, sampling becomes essential to gather information from the environment. Subsequently, this knowledge can be used to pursue a variety of objectives.
  - *Model-based.* These algorithms use collected samples to build an approximation of the MDP's dynamics. This enables the direct application of Dynamic Programming techniques to solve the control problem.
  - *Model-free.* Contrary to building an explicit model, the majority of RL algorithms focus directly on optimizing the value function or policy without attempting to model the environment's reward dynamics or transition probabilities.
- **Learned Estimator**. Focusing on model-free approaches, it is possible to optimize different objectives.
  - *Value-based.* These approaches focus on learning the value function, from which the greedy optimal policy is derived.
  - *Policy-based.* Here, the algorithms directly optimize the policy function, represented by a parameter vector, θ, that gets updated through gradient ascent to favour actions that lead to higher rewards.
  - *Actor-Critic.* Merging the strengths of both value-based and policy-based frameworks, actor-critic methods simultaneously optimize the policy (the actor) and evaluate the value function (the critic).
- **Interaction**. As discussed earlier, reinforcement learning agents interact with the environment to gather information. This interaction can be categorized based on how data collection and learning are managed:

  - *Offline RL.* In this approach, the agent collects data from the environment beforehand (through simulation or past experiences). Samples are then used to train the policy or the value function in a separate phase. Offline RL is typically used when directly interacting with the environment is impractical or expensive.
  - *Online RL.* Here, the processes of data collection and learning are interleaved. The agent engages with its environment executing actions and getting feedback. This data is then used to refine the policy or value function. Online RL is well-suited for real-time scenarios where the environment can be dynamically changed.

- **Sampling policies**. The policy responsible for dictating the agent's actions during the sampling phase, termed *behavioural*policy, can be distinct from the *target* policy, which is the focus of optimization. This differentiation gives rise to:
  - *On-policy RL.* The agent uses the same policy for both data collection and learning. It essentially learns by evaluating and improving its own actions.
  - *Off-policy RL.* Conversely, in this approach, the agent collects data using a behavioural policy that differs from the one it is trying to learn. This allows for potentially richer experiences but requires techniques to handle the mismatch.

- **Representation**. Finally, we can focus on how an agent represents its knowledge about the environment. Here are the two main approaches:
  - *Tabular RL.* Information, like policies and value functions, is stored in tables. These tables can be finite vectors, matrices, or tensors. However, this method is only feasible for finite MDPs. To address continuous environments, discretization techniques can be applied beforehand to create a manageable representation.
  - *Function approximation.*These approaches use parametric functions to represent the policy or value function. The core idea is to learn the parameters rather than storing every possible value in a table. This allows function approximation to handle continuous MDPs directly and scale to larger environments. Additionally, non-parametric approaches are also possible.

3.2. Policy Evaluation for Reinforcement Learning

This section provides an introduction to model-free policy evaluation algorithms. Our goal is not to provide an exhaustive treatment, refer to [68] for a deeper dive, but to introduce a fundamental concept that underpins policy optimization algorithms discussed later.

Here, we focus on approximating the value function V^π of a given policy π within an unknown, continuous-spaces MDP. It is important to distinguish this from the simpler task of value estimation, where we only need to estimate the value of a specific state $V^\pi(s)$. Policy evaluation aims to reconstruct the entire value function, providing an approximation for every state in the environment. Let $\hat{V}$ be an approximation of V^π we can formulate the policy evaluation problem as minimizing the *Root Mean Squared Error* (RMSE) with respect to the state-space occupancy measure d^π_μ.

$$L(\hat{V}) \coloneqq \left(\mathop{\mathbb{E}}_{s\sim d^\pi_\mu} \left[\left(V^\pi(s) - \hat{V}(s) \right) \right] \right)^{1/2} \tag{3.1}$$

Broadening our scope, we can consider the L_p norm under an arbitrary probability measure ρ, with $p \in [1, \infty)$:

$$L_{p,\rho}(\hat{V}) \coloneqq \|V^\pi(s) - \hat{V}(s)\|_{p,\rho} = \left(\sum_{s\in\mathcal{S}} \rho(s) \left(V^\pi(s) - \hat{V}(s) \right)^p \right)^{1/p}$$

The state space structure dictates the method used to minimize the value function error, Equation (3.1). For a discrete state space, a tabular representation of the value function V^π is employed. In this setting, assuming the induced Markov chain with transition p^π is irreducible[1], minimizing Equation(3.1) transforms into the minimization of $\|V^\pi - \hat{V}\|_1$.

On the other hand, continuous state spaces are typically managed with function approximation. Let $V^\omega : \mathcal{S} \to \mathbb{R}$ be an approximation of V^π, parameterized by a vector $\omega \in \Omega \subseteq \mathbb{R}^d$ of dimension d. We assume V^ω is differentiable w.r.t. ω. Here, the objective in Equation (3.1) is expressed as:

$$\min_{\omega\in\Omega} L(V^\omega) \tag{3.2}$$

While function approximation allows for efficient representation, it can introduce an irreducible bias, reflecting the distance between the true value function V^π and the

[1] A Markov chain is irreducible if any state can be reached from any other state with a positive transition probability.

space of representable value functions defined by the parameter space Ω, denoted as $\mathcal{V}_\Omega := \{V^\omega : \omega \in \Omega\}$. We can formalize the bias as:

$$Bias(\Omega) := \min_{\omega \in \Omega} L(V^\omega) \tag{3.3}$$

If $V^\pi \in V_\Omega$, the bias becomes zero and the approximation is said to be *realizable*.

Policy Evaluation can be achieved through two primary methods: Monte Carlo approximations, which rely on sampling episode outcomes, and Temporal-Difference learning, which leverages a learning process to estimate the value function.

3.2.1. Monte-Carlo Evaluation

In statistics, the term Monte Carlo [52] refers to a class of estimation techniques that derive estimators by averaging repeated random samples. In reinforcement learning, we often need to estimate the value function, which represents the expected future reward from a given state $s \in \mathcal{S}$ under a specific policy π. Consequently, when dealing with *episodic*[2] environments we can exploit MC methods. Gathering a set of N independent trajectories $\{\nu^{(n)}\}_{n=1}^N$, where the initial state of each trajectory s_0^n coincides with the goal state s, we can consider the following estimator:

$$\hat{V}_N^\pi(s) := \frac{1}{N} \sum_{n=1}^{N} G(\nu^{(n)}) = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=0}^{T} \gamma^t r_t \tag{3.4}$$

Each return $G(\nu^{(n)})$ starting from state s is an unbiased estimator of the value function $V^\pi(s)$. Consequently, the average of returns over i.i.d. trajectories is also an unbiased estimator:

$$\mathbb{E}[\hat{V}_N^\pi(s)] = V^\pi(s).$$

Furthermore, according to the central limit theorem, $\hat{V}_N^\pi(s)$ converges almost sure to $V^\pi(s)$ when $N \to \infty$. The variance of this estimate also decreases proportionally to $1/N$.

However, generating distinct batches of trajectories for each state s proves to be highly inefficient in terms of sampling complexity. A more efficient strategy entails analyzing sub-trajectories that pass through the state s, regardless of their initial states. Defying

[2] An environment is said to be episodic when episodes have a guaranteed finite length, in other words, the agent is guaranteed to reach an absorbing state in finite time.

the *first passage time* t_s for a trajectory ν as:

$$t_s \coloneqq \inf\{t \in T(\nu) \colon s_t = s\} \tag{3.5}$$

Averaging over the partial returns, $G(\nu_{t_s}^{(n)})$, associated with the sub-trajectories starting from $\nu_{t_s}^{(n)}$ for each n, still provides an unbiased and consistent estimation for $V^\pi(s)$. This approach is referred to as *first-visit* Monte Carlo estimator. While this method is unbiased, it suffers from high variance, leading to slow convergence, as shown in [31]. An alternative is the *every-visit* Monte Carlo estimator, accounting for partial returns upon every visitation of the target state. Although this method is biased (due to potentially counting rewards multiple times), it is consistent (converges to the true value with increasing N) and has higher sample efficiency.

The every-visit utilizes the same state multiple times within a single trajectory. This introduces correlations between samples, violating the i.i.d. property that the first-visit method enjoys. This method is useful when the available sample size is limited. In contrast, the first-visit approach is generally preferred when a large number of samples are available. While both methods exhibit decreasing variance and bias with increasing samples, the variance reduction in the first-visit method is asymptotically faster. Therefore, when dealing with big datasets, an unbiased estimator, like first-visit, tends to be more reliable.

To improve computational efficiency, we can leverage incremental updates to estimate the value function. This approach avoids storing the entire set or partial returns for each trajectory. Here's the update rule for incremental updates:

$$\begin{aligned}\hat{V}_N^\pi(s) &= \hat{V}_{N-1}^\pi(s) + \alpha_N\Big(G(\nu^{(n)}) - \hat{V}_{N-1}^\pi(s)\Big) \\ &= (1-\alpha_N)\hat{V}_{N-1}^\pi(s) + \alpha_N G(\nu^{(n)})\end{aligned} \tag{3.6}$$

where α_N denotes the learning rate or step size, and $G(\nu^{(n)})$ is the most recent partial return observed in the N-th trajectory starting from state s. While setting $\alpha_N = 1/N$ recovers the empirical mean Equation (3.4), we can employ different learning rate schedules. The estimator remains consistent as long as the step sizes satisfy the *Robbinson-Monro* conditions [51]:

$$\sum_N \alpha_N = \infty, \qquad \sum_N \alpha_N < \infty \tag{3.7}$$

Monte Carlo methods face two key challenges. Monte Carlo methods present two main problems. The first issue is their dependency on an episodic environment, needing the

end of the whole episode before any updates can be made. This limitation prevents their application in scenarios without a finite endpoint. Secondly, the variance of Monte Carlo estimates is directly proportional to the episode length. This is because the return, G incorporates a long sum of random variables. The longer the episode, the higher the variance, potentially leading to slow learning and noisy estimates.

3.2.2. Temporal Difference Evaluation

Temporal Difference (TD) methods [68], offer a viable alternative for non-episodic interaction. TD tackles the issue of high variance in Monte Carlo estimates by truncating the sum of future rewards considered. TD can be seen as an approximation of the iterative policy evaluation algorithm, described in Section 2.3.1. At their essence, TD methods aim to minimize the divergence, known as the Bellman Error (BE), between the current estimate of a state's value and the expected value following the application of the Bellman Operator. The BE, defined for an estimator $\hat{V}$, in the state s can be formally expressed as:

$$BE(s) = \mathop{\mathbb{E}}_{a\sim\pi(\cdot|s)}\left[R(s,a) + \gamma \mathop{\mathbb{E}}_{s'\sim P(\cdot|s,a)}\left[\hat{V}(s')\right]\right] - \hat{V}(s). \tag{3.8}$$

In case $\hat{V} = V^\pi$ the error is uniformly zero in the whole state space, therefore the goal of the evaluation problem can be defined as the minimization of the *BE*. To this end, TD focuses on the (1-step) *sub-trajectories* in the form (s_t, a_t, r_t, s_{t+1}) to compute the Temporal Difference Error (TDE), δ_t as follows:

$$\delta_t := r_t + \gamma\hat{V}(s_{t+1}) - \hat{V}(s_t). \tag{3.9}$$

Then, the TDE represents a single realization to estimate the BE as:

$$BE(s_t) = \mathop{\mathbb{E}}_{\substack{a_t\sim\pi(\cdot|s_t),\\ s_{t+1}\sim P(\cdot|s_t,a_t)}}\left[\delta_t\right].$$

Then the $TD(0)$ method employs an iterative correction following the update rule:

$$\hat{V}^{(t+1)}(s_t) = (1-\alpha)\hat{V}^{(t)}(s_t) + \alpha_t \underbrace{\left(r_t + \gamma\hat{V}^{(t)}(s_{t+1})\right)}_{\text{TD target}},$$

where $\alpha_t > 0$ is the *learning rate*. This process, known as *bootstrapping*, employs previous estimations to perform updates. To sum up, we can identify the following key differences between the two methods:

- **Environment**. Unlike Monte Carlo approaches, TD methods can be applied in environments with infinite horizons.
- **Bias-Variance trade-off**. Although TD methods introduce some bias by truncating the sum of future rewards, they significantly reduce the variance compared to Monte Carlo methods. In practice, this often leads to faster convergence.
- **Online updates**. TD updates can be computed after each single interaction with the environment, making them online methods. On the other hand, MC are deep first strategies, an update can be computed only after reaching a terminal state.

3.2.3. Monte-Carlo and TD trade-off

Both methods aim to estimate a value function, however, they inherently introduce estimation errors due to three main sources: bias, variance, and the irreducible error introduced by the nature of the process. To manage these errors, a careful bias-variance trade-off is essential. This is where the $TD(n)$ methods play a crucial role by bridging the gap between MC and TD approaches. The basic idea is to compute the TD target is over subsets of length n of the trajectory, $(s_t, a_t, r_t, s_{t+1}, \ldots, r_{t+n-1}, s_{t+n})$, as follows:

$$G_n^{(t)} := \sum_{i=0}^{n-1} \gamma^i r_{t+1} + \gamma^n \hat{V}^{(t)}(s_{t+n})$$

Furthermore, $TD(\lambda)$ methods strike a balance using exponential averages of TD targets across various horizons, controlled by the parameter $\gamma \in [0, 1]$. The method evolves the estimator into:

$$G_\lambda^{(t)} := (1 - \lambda) \sum_{i=0}^{\infty} G_n^{(t)}$$

The standard $TD(\lambda)$ approach, also known as the *forward* view, sacrifices real-time processing capabilities. This limitation is overcome by adopting the *backward* view, in combination with *eligibility traces*, [68]. This technique allows solving the credit assignment problem considering both the frequency and recency of state visits as follows:

$$e_t(s) = \begin{cases} \gamma\lambda e_{t-1}(s), & if s \neq s_t, \\ 1, & s = s_t. \end{cases}$$

Consequently, the update rule becomes:

$$\hat{V}^{(t+1)}(s_t) = \hat{V}^{(t)}(s_t) + \alpha_t e_t(s_t)\big(r_t + \gamma \hat{V}^{(t)}(s_{t+1}) - \hat{V}^{(t)}(s_t)\big),$$

In summary, these predictive algorithms are building blocks for effective control strategies, allowing the evaluation of policies. $TD(\lambda)$ offers a comprehensive framework that spans from TD updates ($\lambda = 0$) to MC-based approach ($\lambda = 1$). Although our discussion focuses on estimating state-value functions, these methods are equally applicable for the state-action value functions where the TD error can be computed as follows:

$$\delta_t = r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t). \tag{3.10}$$

3.3. Value-Based Control

This section focuses on value-based control methods, frequently compared with policy-based optimization algorithms (section 3.4). We provide a high-level overview of the core algorithmic concepts. For more detailed explanations of value-based prediction methods refer to [68].

Notation. *To simplify notation, in the following sections, we switch from using $\hat{Q}$ to just Q. This assumes the context makes it clear we are referring to the estimated value.*

3.3.1. SARSA

State Action Reward State Action (SARSA, [53]) is an *on-policy*[3] method that can be seen as the RL version of Policy Iteration. It involves mainly two steps: Policy Evaluation and Policy Improvement.

Policy Evaluation

TD learning is used to evaluate the policy since there is no knowledge of the environment's dynamics. Formally, at each time instance t the environment is sampled to get (s_t, a_t, r_t, s_{t+1}) and the Q function is updated as:

$$Q^{(t+1)}(s_t, a_t) = Q^{(t)}(s_t, a_t) + \alpha \delta_t,$$

[3]TD updates are performed using the experience gathered by following the same policy being learned.

where δ_t is the TD error defined in equation 3.10. Again, all variants mentioned in 3.2.3 are still applicable.

Policy Improvement

Once an estimation of the value function of the current policy π_t is obtained, the next step, described in section 2.3, tries to improve the policy. This traditionally involves extracting the greedy-policy w.r.t. $Q^{(t)}$ to retrieve a higher-scoring policy π_{t+1}. However, a crucial challenge arises here: the exploration-exploitation dilemma. While greedy policies are attractive for maximizing immediate rewards based on acquired knowledge, they can be detrimental in the long run. When solely relying on trial-and-error interaction with the environment, greedy policies may miss out on exploring new possibilities that could lead to even greater rewards in the future.

To address this exploration-exploitation trade-off, various techniques have been developed. In the case of finite action spaces, a popular approach is the ϵ-greedy policy.

Definition 3.1 ($\epsilon - GreedyPolicy$). *Let $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ be a bounded, measurable function, and let $\epsilon \in [0, 1]$. A policy $\pi \in \Pi$ is denoted as $\epsilon - greedy$ with respect to f if, $\forall (s, a) \in \mathcal{S} \times \mathbf{A}$:*

$$\pi(a|s) = \begin{cases} \frac{\epsilon}{|\mathcal{A}|} + 1 - \epsilon, & if\ a = arg\max_{a \in \mathcal{A}} f(s, a), \\ \frac{\epsilon}{|\mathcal{A}|}, & otherwise. \end{cases} \tag{3.11}$$

This method introduces a degree of randomness into the decision-making process. With probability ϵ, the agent explores by selecting a random action, while with probability $(1 - \epsilon)$, it acts greedily. If $\epsilon = 0$, it reduces to a greedy policy.

Besides their simplicity, ϵ-greedy policies have a major drawback: they do not rely on the agent's knowledge about the quality of each action. In contrast, *Boltzmann* (or *Softmax*) policies incorporate these value function estimates, leading to a more informed distribution of action selection.

Definition 3.2 (Boltzmann Policy). *Let $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ be a bounded, measurable function, and let $\tau > 0$. A policy $\pi \in \Pi$ is denoted as Boltzmann with respect to f if, $\forall (s, a) \in \mathcal{S} \times \mathbf{A}$:*

$$\pi(a|s) = \frac{e^{\frac{Q(s,a)}{\tau}}}{\sum_{a' \in \mathcal{A}} e^{\frac{Q(s,a)}{\tau}}}, \tag{3.12}$$

where the parameter τ, called temperature controls the exploration rate. The policy

becomes greedy when $\tau \to 0$. Additionally, this technique can be extended to continuous action spaces as well. SARSA is guaranteed to converge toward the optimal policy if the *GLIE* (Greedy in the Limit with Infinite Exploration) assumption is satisfied. In other words, if all state-action pairs are explored an infinite amount of times and, with $t \to \infty$ the policy converges to a greedy one.

Theorem 3.1 (Convergence of SARSA, Theorem 1 [64]). *Let $\mathcal{M}$ be a discrete MDP, with the Q-values stored in a lookup table. If:*

1. *each action is executed infinitely often in every state, which is visited infinitely often;*
2. *in the limit for $t \to \infty$, the leaning policy is a.s. greedy with respect to the Q-value function $Q^{(t)}$;*
3. *the step sizes α_t satisfies the Robbins-Monro conditions;*
4. $\mathbb{V}ar[R(s,a)] < \infty, \quad \forall (s,a) \in \mathcal{S} \times \mathcal{A},$

then $Q^{(t)}$ converges to $Q^\star$, and the leaning policy π_t converges to $\pi^\star$.

3.3.2. Q-learning

The RL counterpart of Value Iteration is called *Q-learning* [72]. Unlike SARSA, Q-learning operates as an *off-policy* method, allowing the separation between the *behavioural* policy that controls the exploration and the *target* policy for which the value function is optimized. The optimal quality function $Q^\star$ is then computed by the iterative application of the empirical version of the Bellman Optimality Operator $\hat{T}^\star$:

Definition 3.3 (Empirical Bellman Optimality Operators). *Let $\mathcal{M}$ be an MDP. Consider a bounded, measurable function $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and a transition (s, a, r, s'). The Empirical Bellman Optimality Operator $\hat{T}^\star$ is defined as:*

$$(\hat{T}^\star f)(s,a) := r + \gamma \max_{a' \in \mathcal{A}} f(s', a'). \tag{3.13}$$

Q-learning offers a key advantage over SARSA in terms of convergence. Q-learning does not require the GLIE property to be satisfied thanks to its off-policy nature. The pseudocode of Q-learning is reported in Algorithm 3.1.

Algorithm 3.1 Q-learning

Require: Learning rate sequence $\{\alpha_t\}_t$, exploration coefficient ϵ, number of episodes N.
Ensure: Q-function estimation.

```
Initialize Q^(0) arbitrarily
for episode = 1, ..., N do
    t ← 0
    while s_t is not terminal do
        a_t ~ π_{Q^(t)}(s_t)
        take action a_t, observe s_{t+1}, r_t
        δ_t = r_t + γ max_{a∈A}{Q^(t)(s_{t+1}, a)} − Q^(t)(s_t, a_t)
        Q^(t+1)(s_t, a_t) = Q^(t)(s_t, a_t) + α_t δ_t
        t ← t + 1
    end while
end for
```

3.4. Policy Search

In contrast to the value-based methods discussed in the previous section, which learn the optimal policy indirectly through an approximation of the value function, *Policy Optimization* (PO) or *Policy Search* methods follow a different approach [14]. These algorithms directly explore the policy space to find the optimal one. Many policy search algorithms fall under the umbrella of *Policy Gradient* (PG) methods [73], [5], which adopt *parametric policies* (Section 3.4.2).

PO offers several methodological benefits over value-based approaches.

- **Integration of supervised learning techniques**. The policy parameterization allows exploring the solution within the space Π_Θ, thus enabling the application of gradient-based optimization techniques, commonly used in supervised learning.
- **Convergence guarantees**. Unlike value-based methods, which typically require restricting the value function approximation to linear forms to guarantee convergence, several policy optimization algorithms offer convergence guarantees even with non-linear policy representations. This allows using more complex, task-specific policies or general-purpose deep neural networks reducing the risk of divergent behaviours.
- **Handling continuous action spaces**. As discussed in section 3.3, value-based methods rely on action maximization over learned value functions to derive greedy policies. This process becomes impossible or computationally intensive for continuous action spaces. Policy optimization algorithms avoid this problem by directly defining a policy function, allowing to sample actions efficiently.

- **Robustness to noise**. Value-based methods rely on greedy (or ϵ-greedy) policies that are highly susceptible to unexpected disturbances. A small variation of the measured state may result in a different action selection. In PO, policies can be explicitly designed to be less sensitive to noisy or biased states.
- **Partial observability**.While this manuscript focuses on fully observable environments, it is worth noting that policy search algorithms are well-suited for partial observable problems. Their strength arises from two key aspects. First, PO algorithms rely less on Bellman's equations than value-based methods. Second, PO enables learning stochastic policies, which can outperform deterministic ones in partially observable MDPs, as shown by [65].
- **Prior knowledge injection**. Policies can be explicitly designed to incorporate domain knowledge. This can be crucial in safety-critical applications. Unlike indirect policy learning methods, which may face challenges in consistently enforcing such constraints, PO simplifies the integration of these requirements.

Despite these strengths, there are also some limitations. Notably, it generally does not produce a value function, which can be useful for predictions and knowledge transfer across tasks. Policy design can also restrict the exploration of potentially superior solutions, and convergence is typically guaranteed only to local optima. This makes PO more suited for fine-tuning existing controllers rather than learning entirely from scratch.

Notation. *In contexts where parametric policies are employed, we often simplify the notation by representing the dependency on π_θ as θ in function arguments, as well as in superscripts and subscripts. For example, $J(\theta)$ represents $J(\pi_\theta)$ or Q_θ is the short version of Q_{π_θ}.*

3.4.1. Policy Optimization and Policy Gradient Theorem

The goal of policy optimization is searching for the optimal policy within the policy space. As seen in section 2.2.4, optimization relies on a *total order* relation over policies, provided by the scalar measure $J(\pi)$ (equation 2.20). From this follows that a policy π' performs better than π if $J(\pi') \geq J(\pi)$. Under the assumption that policies are differentiable w.r.t. parameters θ, we can conclude the same for $J(\theta)$. Exploiting the gradient of the expected return, it is possible to optimize the policy with *Stochastic Gradient Ascend* (SGA) approaches, applying the general update rule:

$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_\theta J(\theta_t), \tag{3.14}$$

where α is the learning rate and $\nabla_\theta J(\theta_t)$ is the gradient, w.r.t. θ of the expected return at time step t, $J(\theta_t)$

The *Policy Gradient Theorem* (PGT) comes into play providing a valuable approach to compute $\nabla_\theta J(\theta)$. To better understand the PGT, we first provide a useful lemma:

Lemma 3.1 (Lemma 20 from [13]). *Let $\pi : \mathcal{S} \to \Delta\mathcal{A}$, and f and g be integrable functions on $\mathcal{S}$, such that, for all $s \in \mathcal{S}$:*

$$f(s) = g(s) + \gamma \sum_{s' \in \mathcal{S}} P^\pi(s'|s) f(s').$$

Then:

$$f(s) = g(s) + \frac{\gamma}{1-\gamma} \sum_{s \in \mathcal{S}} d_s^\pi(s') g(s').$$

This lemma serves two purposes. First, it lays the groundwork for proving the Policy Gradient Theorem 3.2. Second, it offers a fresh perspective on the expected return. It allows us to express $J(\pi)$ solely in terms of the reward function and the probability of visiting each state-action pair, under the state-occupancy measure.

Corollary 3.1. *Let $\mathcal{M}$ an MDP, and let $\pi : \mathcal{S} \to \Delta_\mathcal{A}$. Then:*

$$J(\pi) = \frac{1}{1-\gamma} \mathop{\mathbb{E}}_{(s,a)\sim\delta_\mu^\pi} \big[R(s,a)\big]. \tag{3.15}$$

Proof. Let us start considering the state value function $V^\pi(s)$:

$$\begin{aligned} V^\pi(s) &= \sum_{a \in \mathcal{S}} \pi(a|s) Q^\pi(s,a) = \\ &= \sum_{a \in \mathcal{S}} \pi(a|s) R(s,a) + \gamma \sum_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \overbrace{\pi(a|s) P(s'|s,a)}^{P^\pi(s'|s)} V^\pi(s') && (3.16) \\ &= \sum_{a \in \mathcal{S}} \pi(a|s) R(s,a) + \frac{\gamma}{1-\gamma} \sum_{s' \in \mathcal{S}} d_s^\pi(s') \sum_{a' \in \mathcal{A}} \pi(a'|s') R(s',a'), && (3.17) \end{aligned}$$

where 3.16 is obtained by applying the Bellman Equation 2.23 and 3.17 derives from

lemma 3.1. Then from the definition of the expected return 2.20:

$$
\begin{aligned}
J^{\pi}(s) &= \sum_{s \in \mathcal{S}} \mu(s) V^{\pi}(s) \\
&= \sum_{s \in \mathcal{S}} \Big[\mu(s) + \frac{\gamma}{1-\gamma} \sum_{s_0 \in \mathcal{S}} d^{\pi}_{s_0}(s) \mu(s_0) \Big] \sum_{a \in \mathcal{A}} \pi(a|s) R(s,a) \\
&= \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi}_{\mu}(s) \sum_{a \in \mathcal{A}} \pi(a|s) R(s,a), \qquad (3.18)
\end{aligned}
$$

where 3.18 derives from 2.13. □

We are now ready to state and prove the main theorem.

Theorem 3.2 (Policy Gradient Theorem, from [69]). *Let $\mathcal{M}$ be an MDP, and let $\pi : \mathcal{S} \to \Delta_{\mathcal{A}}$ be differentiable w.r.t. θ. Then:*

$$
\nabla_{\theta} J(\theta) = \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi_\theta}_{\mu}(s) \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_\theta}(s,a) \qquad (3.19)
$$

Proof. Starting again from the Bellman equation 2.23:

$$
\begin{aligned}
\nabla_{\theta} Q^{\pi_\theta}(s,a) &= \nabla_{\theta} \Big[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a,) V^{\pi}(s') \Big] \\
&= \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) \nabla_{\theta} V^{\pi_\theta}(s')
\end{aligned}
$$

Then, we can express ∇V^{π_θ} as:

$$
\begin{aligned}
\nabla_{\theta} V^{\pi_\theta}(s) &= \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) Q^{\pi_\theta}(s,a) \\
&= \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_\theta}(s,a) + \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \nabla_{\theta} Q^{\pi_\theta}(s,a) \\
&= \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_\theta}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) P(s'|s,a) \nabla_{\theta} V^{\pi_\theta}(s') \\
&= \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_\theta}(s,a) + \frac{\gamma}{1-\gamma} \sum_{s' \in \mathcal{S}} d^{\pi_\theta}_{s}(s') \sum_{a' \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a'|s') Q^{\pi_\theta}(s',a'), \qquad (3.20)
\end{aligned}
$$

where 3.20 derives from lemma 3.1. Finally, following the same reasoning of Corollary 3.1

we get:

$$\begin{aligned}
\nabla_\theta J_\mu(\theta) &= \sum_{s\in\mathcal{S}} \mu(s)\nabla_\theta V^{\pi_\theta}(s) \\
&= \sum_{s\in\mathcal{S}} \Big[\mu(s) + \frac{\gamma}{1-\gamma}\sum_{s_0\in\mathcal{S}} d_{s_0}^{\pi_\theta}(s)\mu(s_0)\Big] \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\sum_{s\in\mathcal{S}} d_\mu^{\pi_\theta}(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\sum_{s\in\mathcal{S}} d_\mu^{\pi_\theta}(s) \sum_{a\in\mathcal{A}} \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s,a), \qquad (3.21)
\end{aligned}$$

where 3.21 is obtained by applying the *log-trick* $\nabla f = f \nabla \log f$. □

The theorem introduces a new concept: the *score function* of a policy, denoted by $\nabla_\theta \log \pi_\theta$ This quantity suggests the direction, in the parameter space Θ, that maximizes the likelihood of choosing a specific action in a given state. Then, the *policy gradient* can be seen as the expected value, under the state-occupancy measure, of the Q function times the policy score. One of the most meaningful extensions of the PGT involves the inclusion of a *baseline* into the gradient term. This baseline, which can be any action-independent integrable function $b : \mathcal{S} \to \mathbb{R}$, helps reduce the variance of the resulting estimators. Indeed, noting:

$$\sum_{s\in\mathcal{S}} d_\mu^{\pi_\theta}(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) b(s) = \sum_{s\in\mathcal{S}} d_\mu^{\pi_\theta}(s) b(s) \nabla_\theta \underbrace{\sum_{a\in\mathcal{A}} \pi_\theta(a|s)}_{1} = 0$$

From this follows that the Policy Gradient Theorem can be also expressed as:

$$J(\theta) = \frac{1}{1-\gamma} \mathop{\mathbb{E}}_{(s,a)\sim\delta_\mu^{\pi_\theta}} \big[Q^{\pi_\theta}(s,a) - b(s)\big]. \qquad (3.22)$$

The value function V^{π_θ} is a common choice for the baseline. In such a case, the gradient is obtained from the advantage function A^{π_θ}.

3.4.2. Parametric Policies

This manuscript mainly focuses on *parametric policies*, since their pivotal role in policy optimization. In this category, a policy, denoted as $\pi_\theta \colon \mathcal{S} \to \Delta_\mathcal{A}$, is defined by a parameters vector $\theta \in \Theta \subseteq \mathbb{R}^m$. The solution is then searched within the space $\Pi_\Theta = \{\pi_\theta \in \Delta_\mathcal{A}^\mathcal{S} \mid \theta \in \Theta\}$. A critical assumption in this framework is that π_θ is stochastic

and differentiable w.r.t. θ for all $\theta \in \Theta$. In other words, each policy, π_θ, within the family, Π_Θ, is uniquely identified by a set of parameters, θ.

Direct Parameterization

A naive approach to parameterize a policy, in finite MDPs is *direct parameterization*. A scalar parameter is explicitly assigned to each action probability, such that:

$$\pi_\theta(a_i|s_j) = \theta_{ij}, \tag{3.23}$$

and $\theta \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$ is subject to the *simplex constraints*:

$$\theta_{ij} \geq 0 \qquad \textit{and} \qquad \sum_{i=1}^{|\mathcal{A}|} \theta_{ij} = 1, \quad \forall i \in [1, |\mathcal{S}|]. \tag{3.24}$$

These two constraints are required to ensure the policy is well-defined and must be explicitly enforced by the algorithm. All stochastic and deterministic policies can be represented this way. A key challenge with direct parametrization is that even a small update of policy parameters $\pi_{\theta+\delta}$ can violate the simplex constraints. In the following section, we will present a class of parameter policies that implicitly handles these constraints.

Softmax Policies

Softmax policies, also known as *Gibbs* or *Boltzmann* policies, are popular parametric policy classes when dealing with finite action spaces. Let $h : \Theta \times \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ be an energy function and τ a positive scalar called *temperature*. A softmax policy $\pi_theta \in \Pi_\Theta$ can be formally defined as:

$$\pi_\theta(a|s) = \frac{e^{h_\theta(s,a)/\tau}}{\sum_{a' \in \mathcal{A}} e^{h_\theta(s,a')/\tau}}. \tag{3.25}$$

Essentially, the probability of selecting a specific action is determined by its associated energy, or *preference*, denoted by $h_\theta(\cdot|s)$. This preference function can be any function parameterized by θ, such as a neural network. The *temperature* parameter acts as an entropy regularizer. Higher temperatures lead to more even probabilities (higher entropy) resulting in more explorative policies. Conversely, Conversely, lower temperatures prioritize actions with higher preference scores (lower entropy), making the policy more deterministic. Finally, the denominator in Equation 3.25 serves as a normalization factor, ensuring that $\pi_\theta(\cdot|s)$ remains a probability distribution. The score of a Softmax policy is:

$$\nabla_\theta \log \pi_\theta(a|s) = \frac{1}{\tau}\left(\nabla_\theta h_\theta(s,a) - \mathop{\mathbb{E}}_{a' \in \pi_\theta(\cdot|s)}\Big[\nabla_\theta h_\theta(s,a')\Big]\right). \tag{3.26}$$

Linear Softmax. The linear approach offers a simple but effective way to parameterize a policy:

$$h_\theta(s,a) = \theta^T \phi(s,a), \tag{3.27}$$

where $\phi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^m$ is a *feature map*. The score becomes:

$$\nabla_\theta \log \pi_\theta(a|s) = \frac{1}{\tau}\Big(\phi(s,a) - \mathop{\mathbb{E}}_{a' \in \pi_\theta(\cdot|s)}\Big[\phi(s,a')\Big]\Big). \tag{3.28}$$

Tabular Softmax. When dealing with discrete action and state spaces, it is possible to parametrize the Softmax policy to cover all possible full-support stochastic policies. However, ensuring this parameterization is non-redundant requires additional considerations. Let $|\mathcal{S}| = n$ and $d = |\mathcal{A}|$. The finite state space assumption, allows us to consider each state $s_j \in \mathcal{S}$ separately, with the parameter vector $\theta_i \in \mathbb{R}^{d-1}$, for $i = 1, ..., n$. Then θ_{ij} denotes the $j-$th element of θ_i and $\theta = [\theta_1^T, ..., \theta_n^T]^T$ is the concatenation of all θ_i. One approach to ensure a non-redundant parameterization is the following:

$$\pi_\theta(a_j|s_i) = \begin{cases} \dfrac{e^{\theta_{ij}/\tau}}{1 + \sum_{k=1}^{d-1} e^{\theta_{ik}/\tau}}, & \text{if } j \in [1,\, d-1], \\ \dfrac{1}{1 + \sum_{k=1}^{d-1} e^{\theta_{ik}/\tau}}, & \text{if } j = d. \end{cases} \tag{3.29}$$

Computing the gradient w.r.t. θ we get:

$$\nabla_\theta \log \pi_\theta(a|s) = \frac{1}{\tau}\Big(e_j - \pi\theta(\cdot|s_i)\Big), \tag{3.30}$$

where e_j, with $j \in [1,\, d-1]$ denotes the j-th base vector of $\mathbb{R}^{d-1}$ and $e_d = 0$.

3.4.3. Actor-Only Policy Gradient

The term *Policy Gradient* (PG) wraps all algorithms that optimize parametric policies relying on first-order information. The general framework of an actor-only PG algorithm is outlined in Algorithm 3.2. Policy updates occur once a batch of $N \in \mathbb{N}$ complete trajectories is collected, where N is called *batch size*.
The dataset $D_k = \{\nu_1, \ldots, \nu_N | \nu_i \sim p_{\theta_k}\}$ collects these trajectories. Given that $\hat{\nabla}_\theta J$ is an unbiased estimator of the policy gradient, i.e., $\mathbb{E}[\hat{\nabla}_\theta J(\theta; \mathcal{D})] = \nabla J(\theta)$, SGD theory ensures convergence to a local optimum, under specific conditions on the step-size sequence and the regularity of $J(\theta)$, ash shown by [11].

Algorithm 3.2 Actor-Only Policy Gradient

Require: Initial policy parameter θ_0, learning rate sequence $\{\alpha_t\}_t$,
batch size N.
for $k = 0, 1, \ldots$ **do**
  Collect N trajectories with θ_k to generate $\mathcal{D}_k$
  Compute policy gradient estimate $\hat{\nabla}_\theta J(\theta_k; \mathcal{D}_k)$
  Update policy parameters $\theta_{k+1} \leftarrow \theta_k + \alpha_k \hat{\nabla}_\theta J(\theta_k; \mathcal{D}_k)$
end for

Monte Carlo Policy Gradient Theorem

The PGT finds its first application in an algorithm proposed by [69], which will be referred to as *Monte Carlo PGT* to avoid confusion. This algorithm (Algorithm 3.3) assumes an episodic interaction with the environment to produce trajectories of indefinite length where rewards are undiscounted and the continuation probability is $\gamma < 1$.
As detailed in Section 3.2.1, the return $G(\nu^{(n)})$, observed under the policy π_θ, provides an unbiased estimate of $Q^{\pi_\theta}(s_t, a_t)$. Thus, we can build the following gradient estimator:

$$\hat{\nabla}_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{|\nu|-1} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) G(\nu_i), \tag{3.31}$$

where $G_(\nu_i)$ is the return of trajectory ν_i.

Algorithm 3.3 Monte Carlo PGT

Require: Initial policy parameter θ_0, learning rate sequence $\{\alpha_t\}_t$.
for $k = 0, 1, \ldots$ **do**
  Generate trajectory $\nu \sim p_{\theta,\gamma}$
  Let H $= |\nu|$
  for $t = 0, \ldots, H-1$ **do**
    $G_t(\nu) = \sum_{i=t}^{H-1} \gamma^{i-t} r_{i+1}$
  end for
  $\hat{\nabla}_\theta J(\theta_k) = \sum_{t=0}^{H-1} \nabla_\theta \log \pi_{\theta_k}(a_t|s_t) G_t(\nu)$
  $\theta_{k+1} \leftarrow \theta_k + \alpha_k \nabla_\theta J(\theta_k)$
end for

3.4.4. Actor-Critic Policy Gradient

Policy Gradient estimators suffer from high variance, leading to slow convergence during learning. To mitigate this issue, a possible solution, as anticipated by equation 3.22, is introducing an action-independent baseline function. Algorithms adopting an estimation of the value function V^{π_θ} are defined *Actor-Critic* methods, [32], a general implementation

is reported in Algorithm 3.4. The policy to be optimized is the *actor*, whose actions are evaluated by the *critic*, that is the value function estimation V^ω learnable following any of the methods presented in Section 3.2. Furthermore, it is possible to replace the baseline $b(s)$ with the critic $V^\omega(s)$, without introducing any bias, and get the following gradient:

$$\hat{\nabla}_\theta J(\theta) = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{|\nu|-1} \nabla_\theta \log \pi_\theta(a_t^i|s_t^i)\big(r_t + \gamma V^\omega(s_{t+1}) - V^\omega(s_t)\big).$$

This approach leverages the temporal difference target to estimate the advantage function and is thus called *Advantage Actor-Critic* (A2C), [69]. In contrast, an alternative approach involves incorporating a parameterized estimator of the Q-value, Q_ω. However, this often introduces a small bias in policy updates. To mitigate this bias, a sufficient condition is satisfying the *compatible critic hypothesis* i.e.

$$\nabla_\omega Q^\omega(s,a) = \nabla_\theta \log \pi_\theta(a|s), \quad \forall (s,a) \in \mathcal{S} \times \mathcal{A}.$$

Algorithm 3.4 Actor-Critic Policy Gradient

Require: Initial policy parameter θ_0, initial critic parameters ω_0,
learning rate sequence $\{\alpha_t\}_t$.

1: Observe initial state s_0
2: **for** $t = 0, 1, \dots$ **do**
3: Perform action $a_t \sim \pi_{\theta_t}(\cdot|s_t)$
4: Observe reward r_{t+1} and next state s_{t+1}
5: $\hat{\nabla}_\theta J(\theta_t) = \nabla_\theta \log \pi_{\theta_t}\pi(a_t|s_t)\Big(Q^{\omega_t}(s_t,a_t) - b_t\Big)$
6: $\theta_{t+1} \leftarrow \theta_t + \alpha_t \hat{\nabla}_\theta J(\theta_t)$
7: Update critic parameters ω_t into ω_{t+1}
8: **end for**

Generalized Advantage Estimation

The update procedure for the critic's parameters (Algorithm 3.4, Line 7) is intentionally left abstract allowing the use of various policy evaluation methods. Here, we present a technique, introduced by [59], known as *Generalized Advantage Estimation* (GAE), which draws inspiration from eligibility traces, reported in section 3.2.3. GAE enables the building of an advantage function estimate from a value-function critic V_ω, which can be derived using MC or TD evaluation. The GAE estimator is expressed as a discounted sum of TD errors:

$$A_\lambda^\omega(s_t,a_t) = \sum_{h=0}^{\infty}(\gamma\lambda)^h \delta_{t+h}^\omega, \tag{3.32}$$

where δ_t is the temporal difference error (Equation 3.9) and $\lambda \in [0, 1]$ controls the bias-variance trade-off. The two extreme cases are $\lambda = 1$, corresponding to the MC evaluation:

$$A_1^\omega(s_t, a_t) = \sum_{h=0}^{\infty} G_t - v^\omega(s_t), \tag{3.33}$$

which is unbiased but has higher variance and $\lambda = 1$, leading to TD(0):

$$A_0^\omega(s_t, a_t) = \delta_t^\omega, \tag{3.34}$$

that in general introduces a small bias[4], but has less variance.

Remark 3.1. *The computation of this quantity involves the knowledge of the value function of future states, as well as future rewards. To this end, it is typically computed backward, once a trajectory is completed. Another approach relies on using eligibility traces, mentioned in Section 3.2.3.*

3.4.5. Safe Learning Methods

Policy gradient methods determine update directions for policy parameters by relying on first-order derivatives. However, this gradient information is inherently local, providing a limited view of how the policy might change with updates. As a result, even apparently small parameter adjustments of the form $\theta' = \theta + \alpha \nabla_\theta J(\theta)$ can lead to a significant change in the distribution of state-action pairs visited by the agent, $\delta_\mu^{\theta'}$. This phenomenon, known as *distributional shift*, can cause performance fluctuation and even trigger unexpected and potentially detrimental behaviour. An important result is provided by [30]:

Theorem 3.3 (Performance Difference, Lemma 6.1 [30])**.** *Let $\mathcal{M}$ be and MDP, and $\pi, \pi' \in \Pi$ be two policies. Then:*

$$J(\pi') - J(\pi) = \sum_{s \in \mathcal{S}} d_\mu^{\pi'}(s) \sum_{a \in \mathcal{A}} \pi'(a|s) A^\pi(s, a). \tag{3.35}$$

Building upon these findings, we can establish lower bounds on the performance of subsequent policies by utilizing the advantage and return of the current policy. This approach yields performance improvement guarantees within the framework of *Approximate Policy Iteration* (API). Notably, [47], [48], and [44] expanded this work in the realm of policy search. These studies establish a robust theoretical framework and provide guidelines

[4]This quantity is unbiased when the V-function estimate is unbiased.

for selecting safe step sizes to avoid performance degradation. However, in complex environments, the learning rates for safety guarantees are typically very low, leading to slow training progress. As a result, these methods are quite inefficient and rarely used in practical applications.

Trust Region Methods

Drawing inspiration from Convex Optimization, an alternative paradigm addresses this issue. Unlike *Line Search* (LS) methods, which sequentially compute update direction and step size, *Trust Region* (TR) approaches focus on finding the optimal solution within a predefined region of the parameter space. Specifically, they define a trust region, which is a subspace of solutions within a certain distance from the current policy, and the optimization problem is solved under this constraint.

In the following paragraphs, we will explore two well-known policy search algorithms: *Trust Region Policy Optimization* (TRPO, [58]) and *Proximal Policy Optimization* (PPO, [60]) .

Trust Region Policy Optimization In the realm of RL, *Trust-Region* methods are designed to constrain policy updates within a predefined neighbourhood of the previous parameters. This constraint is enforced using an appropriate distance metric, ensuring smoother policy transitions. To quantify the notion of *distance* in the policy space, we use the *Kullback-Leibler* (KL) divergence:

$$D_{\mathcal{KL}}(\pi(\cdot|s)||\pi'(\cdot|s))) := \int_{\mathcal{A}} \pi(a|s) \log \frac{\pi(a|s)}{\pi'(a|s)} da. \tag{3.36}$$

The direct application of Theorem 3.3 necessitates knowledge of the occupancy measure $d_\mu^{\pi'}$ under the new policy π', which can significantly slow down the optimization process. However, by leveraging the computation of the policy gradient, we gain some insights on the distribution under the current policy d_μ^π, leading to the definition of the following surrogate objective:

$$L_\pi(\pi') := J(\pi) + \int_{\mathcal{S}} d_\mu^\pi(s) \int_{\mathcal{A}} \pi'(a|s) A^\pi(s,a)\, da\, ds. \tag{3.37}$$

This objective provides a first-order local approximation of $J(\pi')$, offering valuable information about the neighbourhood of π. The main result provided in [58] allows to lower bound the expected improvement as follows.

Theorem 3.4 (Theorem 1 [58]). *Let α be the maximum $\mathcal{KL}$ divergence, defined as:*

$$\alpha = D_{\mathcal{KL}}^{max}(\pi||\pi') := \max_{s\in\mathcal{S}} D_{\mathcal{KL}}\Big(\pi(\cdot|s)||\pi'(\cdot|s)\Big),$$

then:

$$J(\pi') \geq L_\pi(\pi') - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha,$$

where $\epsilon := \max_{(s,a)\in\mathcal{S}\times\mathcal{A}}|A^\pi(s,a)|$

From this equation, we can trivially conclude that, even if there are no performance improvement guarantees, the loss is directly limited by the policy dissimilarity. The central idea of TRPO (Algorithm 3.5) is the following: at each iteration, a trust region is established around the current policy π_{θ_t}, and the algorithm seeks to solve the following constrained optimization problem

$$\begin{aligned} \arg\max_{\theta\in\Theta} \quad & L_{\theta_t}(\theta) \\ s.t. \quad & D_{\mathcal{KL}}^{\max}(\pi_{\theta_t}||\pi_{\theta_{t+1}}) \leq \epsilon, \end{aligned} \tag{3.38}$$

where θ denotes the current policy parameters and $\epsilon > 0$ is a small threshold defining the trust region. Finally, it can be useful to reformulate the constrained optimization problem in terms of the expected values, obtaining:

$$\begin{aligned} \arg\max_{\theta\in\Theta} \quad & \mathop{\mathbb{E}}_{(s,a)\sim\delta_\mu^{\pi_{\theta_t}}}\left[\frac{\pi_{\theta_{t+1}}(a|s)}{\pi_{\theta_t}(a|s)}A^{\theta_t}(s,a)\right] \\ s.t. \quad & D_{\mathcal{KL}}^{\max}\Big(\pi_{\theta_t}(\cdot|s)||\pi_{\theta_{t+1}}(\cdot|s)\Big) \leq \epsilon, \end{aligned} \tag{3.39}$$

TRPO has demonstrated exceptional performance in simulated robot locomotion with deep policies, establishing itself as a state-of-the-art deep reinforcement learning algorithm for continuous control. Despite its empirical success, the theoretical foundations of TRPO have been debated. Initially, TRPO was presented as a *safe* method due to its claimed guarantees of monotonic performance improvement. However, the multitude of approximations required for the practical implementation raises questions about the true guarantee properties of the algorithm.

Algorithm 3.5 Trust Region Policy Optimization (TRPO)

Require: Initialize policy parameter θ_0, initial value function parameter ϕ_0, learning rate α, batch size N, maximum KL-divergence δ.

1: **for** $t = 0, 1, 2, \ldots$ **do**
2: Collect a batch of N trajectories using the current policy π_{θ_t}
3: Compute advantages $\hat{A}_t$ and target values $\hat{V}_t$ for each trajectory
4: Estimate the policy gradient:

$$\hat{g} = \mathbb{E}\left[\nabla_\theta \log \pi_{\theta_t}(a_t|s_t)\hat{A}_t\right]$$

5: Estimate the Fisher Information Matrix:

$$\hat{F} = \mathbb{E}\left[\nabla_\theta \log \pi_{\theta_t}(a_t|s_t)\nabla_\theta \log \pi_{\theta_t}(a_t|s_t)^T\right]$$

6: Solve for the step direction d using conjugate gradient method: $\hat{F}d = \hat{g}$
7: Compute the step size: $\lambda = \sqrt{\frac{2\delta}{d^T\hat{F}d}}$
8: Update policy parameters: $\theta_{t+1} \leftarrow \theta_t + \lambda d$
9: Compute value function loss:

$$L^{VF}(\phi) = \frac{1}{2}\mathbb{E}\left[(V_\phi(s_t) - \hat{V}_t)^2\right]$$

10: Update value function parameters: $\phi \leftarrow \phi - \alpha\nabla_\phi L^{VF}(\phi)$
11: Update $\theta_t \leftarrow \theta_{t+1}$ and $\phi_t \leftarrow \phi$
12: **end for**

Subsequent research works such as [40, 62] reinterpret TRPO as an approximate mirror descent algorithm [6], considering the KL divergence functions as the *Bregman* distance associated with negative entropy. While this perspective discards the original monotonic improvement guarantees, it offers a more intuitive explanation for the empirical success of TRPO.

In summary, TRPO is a policy search reinforcement learning algorithm that directly updates the policy. It maximizes an approximation of the expected return through a constrained optimization problem, ensuring that consecutive policies remain sufficiently similar. The strength of this algorithm compared to Policy Gradient methods lies in the fact that the constraint in Equation 3.38 operates directly in the policy space, rather than in the parameter space of the policy as seen with Policy Gradient updates Equation 3.14. This distinction not only renders the parameter more interpretable and easier to tune but also enables direct control over the policy update.

Proximal Policy Optimization PPO (Algorithm 3.6) offers an alternative policy search technique, simplifying the computational challenges of the nonlinear optimization of TRPO while preserving the core principle of proximity between consecutive policy updates. Despite the TRPO equation (3.38) could be transformed into a Policy Gradient problem using Lagrangian multipliers, this approach was avoided due to the difficulty of choosing appropriate multipliers for different updates during the learning process.

Algorithm 3.6 Proximal Policy Optimization (PPO)

Require: Initialize policy parameter θ_0, initial value function parameter ϕ_0,
learning rate α, clipping parameter ϵ, batch size N, number of epochs K.

1: **for** $t = 0, 1, 2, \ldots$ **do**
2: Collect a batch of N trajectories using the current policy π_{θ_t}
3: Compute advantages $\hat{A}_t$ and target values $\hat{V}_t$ for each trajectory
4: **for** *epoch* $= 1, 2, \ldots, K$ **do**
5: Shuffle the batch and divide it into mini-batches
6: **for** each mini-batch **do**
7: Compute the ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_t}(a_t|s_t)}$
8: Compute surrogate loss $L^{CLIP}(\theta)$:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

9: Update policy parameters: $\theta \leftarrow \theta + \alpha \nabla_\theta L^{CLIP}(\theta)$
10: Compute value function loss:

$$L^{VF}(\phi) = \frac{1}{2}\mathbb{E}\left[(V_\phi(s_t) - \hat{V}_t)^2\right]$$

11: Update value function:$\phi \leftarrow \phi - \alpha \nabla_\phi L^{VF}(\phi)$
12: **end for**
13: **end for**
14: Update $\theta_t \leftarrow \theta$ and $\phi_t \leftarrow \phi$
15: **end for**
16: return optimal policy $\pi_{\theta^\star}$ and value function $V_{\phi^\star}$

Instead, PPO begins by maximizing the same approximated performance measure as TRPO:

$$\mathop{\mathbb{E}}_{(s,a)\sim\delta_\mu^{\pi_{\theta_t}}}\left[\frac{\pi_{\theta_{t+1}}(a|s)}{\pi_{\theta_t}(a|s)} A^{\theta_t}(s,a)\right], \tag{3.40}$$

without considering the constraint. Optimizing this objective alone could result in excessively large policy updates. To address this issue, the approximated performance measure

is modified as follows:

$$L^{PPO}(\pi_\theta) = \mathbb{E}_{(s,a)\sim\delta_\mu^{\theta_t}}\left[\min\left(\underbrace{r_t(\pi_{\theta_{t+1}})A^{\pi_{\theta_t}}(s,a)}_{L^{TRPO}},\ \underbrace{clip\big(r_t(\pi_{\theta_{t+1}}), 1-\epsilon, 1+\epsilon\big)A^{\pi_{\theta_t}}(s,a)}_{L^{CLIP}}\right)\right],$$

where the hyperparameter ϵ is usually set to 0.2 and $r_t(\pi_{\theta_{t+1}}) = \frac{\pi_{\theta_{t+1}}(a|s)}{\pi_{\theta_t}(a|s)}$.

The surrogate objective contains two terms: the same TRPO term and a clipped version that restricts the probability ratio between the two policies within the range $[1-\epsilon, 1+\epsilon]$ limiting the policy divergence. By taking the minimum, PPO adopts a conservative approach: it allows for larger policy updates when their advantage is negative while preventing overly aggressive updates when their performance is better. In essence, this final performance measure serves as a lower bound for L^{TRPO}, incorporating the constraint principle that was explicitly enforced in TRPO.

In conclusion, PPO is a Policy Search method similar to TRPO, addressing the need for proximity between a policy and its update following Policy Gradient method principles. Unlike TRPO, PPO does not rely on explicit constraints. Instead, it directly optimizes the performance measure L^{PPO}, which is designed to guide policy parameter updates towards the direction of the greatest improvement in the estimated return, while simultaneously penalizing excessively optimistic updates. This approach allows PPO to achieve the same level of stability as TRPO but with a simpler implementation and comparable overall performance.

4 | Curriculum Learning

Deep neural networks have emerged as the state-of-the-art method to address several tasks in different domains. However, a notable drawback of current neural models is their reliance on a shuffled presentation of training samples. The standard approach of random initialization and gradient-based optimization often struggles to find good solutions and can get trapped in suboptimal local minima. This contrasts with human learning, where simpler concepts are typically approached before more complex ones. Given that neural networks draw inspiration from the human brain, it is logical to consider aligning their training process with human learning methods [66].

Guided by this idea, **Curriculum Learning** (CL) emerges as a strategic approach in machine learning. At its core, CL is built on the principle that carefully structuring the learning process by gradually increasing the complexity of the tasks can significantly enhance learning efficiency and effectiveness.

Curriculum Learning is not merely an ad hoc machine learning strategy, but rather a concept deeply analyzed in cognitive science and educational psychology. Early research by [17] on neural network language acquisition laid the groundwork for structured learning approaches, setting the stage for CL. Building upon this, [74] highlighted the advantages of CL, suggesting it can both avoid local minima in challenging environments and speed up the convergence process of learning algorithms.

While CL has found significant success in RL, its applications extend to other machine learning paradigms. Presenting simpler patterns before more advanced ones can enhance model performance and learning efficiency in Supervised Learning. Similar principles can guide the exploration of data structures in Unsupervised Learning, facilitating improved feature discovery and representation learning. Key benefits of CL include:

- **Enhancing learning efficiency**. Structuring the learning process can lead to faster convergence and reduced training times.
- **Improving model generalization**. By grounding learning in simpler but foundational tasks, models may develop more robust and generalizable representations.

- **Increasing learning stability**. Gradually increasing task complexity can help mitigate learning instability and convergence issues in challenging domains.

In addition to its practical contributions, Curriculum Learning poses interesting questions regarding the nature of intelligence and the mechanisms underlying effective learning. By emulating the gradual complexity increase, characteristic of human educational curricula, CL offers a valuable framework for investigating the nuanced interplay between structured training, task difficulty, and learning efficiency in machine learning.

4.1. Curricula in Reinforcement Learning

In Reinforcement Learning, a curriculum can be visualized as an ordered sequence of experience samples. This structure does not just organize past experiences, it also guides the agent towards acquiring new and more complex ones. When a task is modelled as an MDP, a curriculum can be formally defined as:

Definition 4.1 (Curriculum, [39]). *Let $\mathcal{T}$ be a set of tasks, where $m_i = (S_i, A_i, p_i, r_i)$ is a task in $\mathcal{T}$. Let $\mathcal{D}^{\mathcal{T}}$ be the set of all possible transition samples from tasks in $\mathcal{T}$: $\mathcal{D}^{\mathcal{T}} = \{(s, a, r, s') \mid \exists m_i \in \mathcal{T}$ s.t. $s \in \mathcal{S}_i, a \in \mathcal{A}_i, s' \sim p_i(\cdot|s, a), r \leftarrow r_i(s, a, s')\}$. A curriculum $\mathcal{C} = (\mathcal{V}, \mathcal{E}, g, \mathcal{T})$ is a direct acyclic graph, where $\mathcal{V}$ is the set of vertices, $\mathcal{E} \subseteq \{(x, y)|(x, y) \in \mathcal{V} \times \mathcal{V} \wedge x \neq y\}$ is the set of directed edges, and $g : \mathcal{V} \to \mathcal{P}(\mathcal{D}^{\mathcal{T}})$ is a function that associates vertices to subsets of samples in $\mathcal{D}^{\mathcal{T}}$, where $\mathcal{P}(\mathcal{D}^{\mathcal{T}})$ is the power set of $\mathcal{D}^{\mathcal{T}}$. A directed edge $\langle v_j, v_k \rangle$ in $\mathcal{C}$ indicates that samples associated with $v_j \in \mathcal{V}$ should be trained before samples associated with $v_k \in \mathcal{V}$. All paths terminate on a single sink node $v_t \in \mathcal{V}$.*

This approach leverages a curriculum function that partitions the training data based on a vertex-to-sample mapping. This mapping dictates a specific learning order: the agent must be trained on samples associated with vertex $v_j \in \mathcal{V}$ before progressing to those linked to vertex $v_k \in \mathcal{V}$, if an edge (v_j, v_k) exists. Importantly, the mapping is not bijective, allowing for repeated training on previously encountered samples.

The huge number of tasks in $\mathcal{D}^{\mathcal{T}}$ makes unfeasible the definition of a curriculum graph at the sample level. To overcome the problem, we resort to more abstract representations, leveraging various task-related dimensions.

Definition 4.2 (Single-task Curriculum, [39]). *A single-task curriculum is defined as a curriculum $\mathcal{C}$ where the cardinality of the set of tasks considered for extracting samples $|\mathcal{T}| = 1$, and consists of the target task only, m_t.*

In other words, the source task and the target one coincide, therefore there is no need for transfer learning. The aim is to identify the optimal sequence of samples for training. Another relevant approach involves structuring the curriculum at the task level, where each vertex of the curriculum's graph wraps samples derived from a single task. At this granularity, we articulate a curriculum as follows:

Definition 4.3 (Task-level Curriculum, [39]). *For each task $m_i \in \mathcal{T}$, let $\mathcal{D}_i^{\mathcal{T}}$ be the set of all samples associated with task m_i: $\mathcal{D}_i^{\mathcal{T}} = \{(s, a, r, s') \mid s \in \mathcal{S}_i, a \in \mathcal{A}_i, s' \sim p_i(\cdot \mid s, a), r \leftarrow r_i(s, a, s')\}$. A task-level curriculum, denoted as $\mathcal{C} = (\mathcal{V}, \mathcal{E}, g, \mathcal{T})$, assigns each vertex to samples exclusively from a single task in $\mathcal{T}$. Thus, the mapping function g is defined by $g : \mathcal{V} \to \{\mathcal{D}_i^{\mathcal{T}} \mid m_i \in \mathcal{T}\}$.*

In contrast to supervised learning, where samples for each task are predetermined, reinforcement learning involves the dynamic generation of experience, shaped by the agent's evolving behavioural policy and the interactions with previous tasks. Consequently, crafting a task-level curriculum involves sequencing tasks to ensure that the learning policy encourages the acquisition of beneficial samples in subsequent tasks. Again, the same task can be mapped to multiple vertexes, supporting an iterative learning process, and enabling the agent to fine-tune the policy, through repeated exposure to available tasks as investigated in [70].

An additional, more streamlined form of curriculum simplification is the linear sequence, characterized by:

Definition 4.4 (Sequence Curriculum, [39]). *A sequence curriculum is a curriculum $\mathcal{C}$ where the indegree and outdegree of each vertex v is at most 1, and there is exactly one source node and one sink node.*

Despite its simplicity, a sequence curriculum is the most commonly employed structure in training models. Nevertheless, these abstractions can be combined in various ways. For example, a task-level sequence curriculum can be represented as an ordered list of tasks, $[m_1, m_2, \ldots, m_n]$. The framework presented in this thesis will adopt this approach. As detailed in the following chapters, the agent will be trained on samples from task m_i and will progress to task m_j, with $i < j$, after acquiring sufficient knowledge from the preceding task obtaining what is called a *task-level sequence curriculum*.

4.2. Curriculum Design

As already anticipated, the rationale behind CL lies in the observation that defining a meaningful order over training samples can significantly improve performance. This often translates into achieving better asymptotic rewards or faster convergence compared to training with shuffled data. Intuitively, CL allows the agent to tackle a simplified version of the problem before gradually increasing its complexity. This structured approach enables the agent to navigate the solution space more efficiently, leveraging prior knowledge to handle progressively difficult tasks. Designing an effective curriculum necessitates a thorough understanding of the relevant aspects. In particular, grasping the following concepts is essential:

- **Task Generation**: The definition of intermediate tasks is pivotal in bridging the gap between the learner's initial capabilities and the requirements of the target task. The quality and structure of these tasks significantly influence learning trajectories, which can involve either a single target task or multiple intermediate steps. Recent advancements have trended towards automated task generation, however, human oversight remains necessary for aspects such as hyper-parameter tuning and assessing task solvability.

- **Sequencing**: At the heart of curriculum design lies the challenge of sequencing. This crucial operation involves introducing tasks in a logical order to shape an optimal learning pathway. Determining the ideal sequence and timing for task presentation is arguably one of the most complex aspects of curriculum learning. This process can be approached either *manually*, relying on expert knowledge, or through *automated* algorithms. The development of a fully automated sequencing method represents a compelling yet highly challenging research direction, and it forms one of the primary focuses of this manuscript.

- **Transfer Learning**. It should now be clear that source tasks can differ from the target task, in terms of state and action spaces, reward functions, and transition dynamics. This variation introduces the problem of effectively *transferring knowledge* from simpler tasks to more complex ones. Transfer can involve *low-level* knowledge, such as complete policies or value functions, which can directly initialize the learner for subsequent tasks, or *high-level* knowledge, providing strategic guidance for future learning. Unfortunately, to transfer low-level knowledge an important assumption must hold: the state and action spaces do not change over tasks. When this condition is not met, high-level transfer becomes the most viable option and can be implemented through techniques such as *reward shaping* [41] or *partial policies*.

Another distinction to be mentioned is the one between *static* and *dynamic* curricula. A *static* curriculum is established before the beginning of the learning process, with a fixed sequence of tasks that does not change in response to the learner's performance. In contrast, a *dynamic* curriculum adjusts to the learner's progress in real-time, adapting the complexity and sequencing of tasks to the agent's current capabilities.

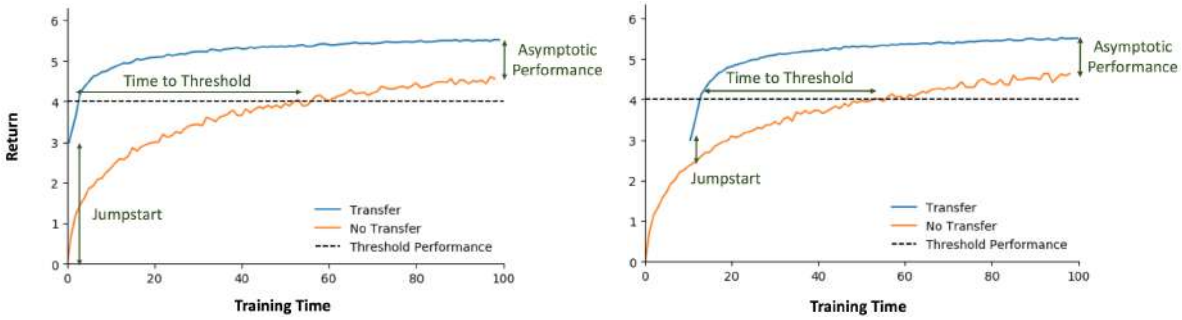
While dynamic curricula offer significant advantages, they introduce new complexities. In Chapter 6 we provide a technique to develop both static and dynamic curricula.

4.3. Curriculum Evaluation

Evaluating the effectiveness of a curriculum requires specific metrics to quantify the benefits of transitioning from source tasks to the target task. The assessment involves the comparison of the learning curves on the target task both with and without the application of transfer learning. Several key metrics can be employed:

- **Time to Threshold**: assesses the computational effort required to achieve a specified performance threshold, $G_0 \geq \delta$, with and without transfer learning.
- **Asymptotic Performance**: quantifies the difference in convergence performance on the target task achieved by a curriculum-based method versus a non-curriculum one.
- **Jump Start** evaluates the initial performance boost on the target task attributable to the knowledge transferred, offering insights into the early benefits of transfer learning.
- **Total Reward Ratio**: contrasts the cumulative rewards obtained by the agent during training up to a fixed point, comparing scenarios with transfer learning to those without.

A key consideration is deciding whether or not to keep into account the learning time of source tasks. Ignoring these costs results in what is termed *weak transfer*, Figure 4.1a, while their inclusion leads to *strong transfer* settings, Figure 4.1a. Additionally, the investment required for curriculum development must be accounted for. Given the prevalent practice of manually creating curricula, quantifying the overall effort in terms of time, resources and prior knowledge remains unfeasible.



(a) Performance curves in weak transfer setting. (b) Performance curves in strong transfer setting.

Figure 4.1: Performance metrics for transfer learning using weak transfer (a), and strong transfer (b), with offset curves (origin: [39]).

Although automated methods for curriculum generation exist, they still necessitate some prior knowledge. Consequently, the design phase's expenses are often regarded as sunk costs. The choice of evaluation metrics should align with the specific objectives and constraints of the application. Considerations might include:

- **Data Acquisition Costs**. Assessing the costs for gathering data for final tasks compared to those for intermediate tasks. Situations with less expensive intermediate tasks might lean towards a *weak transfer* scenario.
- **Performance Improvement Priorities**. Determining whether the emphasis is on early performance gains, enhancing end-stage performance, or optimizing the path to reach a specific performance milestone.
- **Evaluation Objectives**. Deciding whether the evaluation aims to compare the effectiveness of different curricula or to measure the impact of a specific curriculum in contrast to traditional training methodologies.

These guidelines emphasize the importance of adapting curriculum evaluation to address the specific challenges and objectives of each application. We now provide an overview of some curriculum learning methods employed within the field of reinforcement learning.

4.4. State of the Art

In this section, we delve into the existing literature on curriculum learning within the context of RL. Building on the previously discussed foundational concepts and evaluation metrics, we will present a survey on related works, inspired by the classification framework proposed by [39]. The literature will be categorized according to the three main elements of curriculum learning: *task generation*, *task sequencing*, and *transfer learning*. By exam-

ining these components, we aim to provide a better understanding of the state-of-the-art approaches and methodologies that have shaped the field, highlighting the progress made and the challenges that remain. This exploration will set the stage for the introduction of our novel MDP framework, the *Teleport MDP* (Chapter 5), which seeks to integrate curriculum learning into traditional reinforcement learning paradigms.

4.4.1. Task Generation

Task generation involves creating a set of intermediate tasks to be included in the curriculum, ensuring that the knowledge acquired from these tasks is relevant for the final task while avoiding *negative transfer*[1]. Although limited formal studies have addressed this problem in the context of Reinforcement Learning, existing works generally assume that the domain can be parameterized, with different parameter instantiations leading to different tasks.

For instance, [37] presents several methods for intermediate task creation based on an MDP representation of the domain. Each task is associated with a *task descriptor*, a vector of parameters specifying the degrees of freedom. By varying these degrees of freedom, it is possible to define intermediate tasks. The methods include:

- **Task simplification**: reduces the degrees of freedom.
- **Promising initialization**: edits the set of initial states by adding states closer to the goal or, more generally, closer to higher rewards.
- **Action simplification**: prunes the action set to remove actions with a high mistake rate.
- **Mistake learning**: allows rolling back to a previous state when a mistake is detected, resuming learning from there. In principle, a mistake is any action or sequence of actions that deviates from the optimal policy. However, since the optimal policy is unknown, several alternatives can be considered, such as any action causing an unsuccessful episode termination, any action with no state change, or any action leading to a large negative reward.

An alternative framework, based on *Object Oriented MDPs* (OO-MDPs, [63]), assumes each task is associated with a class environment, parameterized by specific attributes. A domain-specific function generates intermediate tasks by assigning values to these attributes, simplifying the problem. The varying types and numbers of objects within the

[1]Negative transfer occurs when transferring knowledge from a source task negatively affects performance.

environment create a range of tasks with differing levels of complexity. However, the random generation of these tasks necessitates careful validation to ensure their solvability.

4.4.2. Sequencing

We now turn our attention to task sequencing. As previously established, sequencing aims to order a set of tasks to facilitate learning. Task sequencing methods differ significantly based on the relationship between source and target tasks. Some techniques reorder experiences within a single task, while others require identifying criteria to rank and order distinct tasks to optimize performance on the final goal. This section explores the different approaches, starting from a simple sample re-order, moving to multi-agent strategies and concluding the analysis with techniques that rely on different intermediate tasks.

Sample Sequencing

When sample sequencing involves experiencing directly from the target task, the problem aligns with applying CL strategies in a supervised learning framework. Therefore, training samples should be presented to the learner in a specified order. For instance, the *Deep Q Network* (DQN, [36]), uses a buffer to store tuples of past experience. At each training step, the buffer is sampled to train the model in mini-batches. While the original formulation picks uniformly at random, a sequencing algorithm can introduce an order within the buffer. *Prioritized Experience Replay* (PER, [56]), prioritizes important transitions, such as those with high expected learning progress in terms of TD error. The idea is that replaying samples with larger TD errors enables stronger updates. As transitions are learned, the distribution of important transitions shifts, implicitly defining a curriculum.

Acting from a different perspective, it is possible to re-struct the sampling process to implicitly dictate an order. In general, early visited states can be considered easier to reach compared to those approached later on. However, in settings with a sparse reward, earlier states may not provide a reward signal, introducing the need for some reward-shaping techniques. An implementation of this idea is provided by *Hindsight Experience Replay* (HER, [2]). HER enables learning not only from desired outcomes but also from undesired ones. The problem is framed as learning a *Universal Value Function Approximator* (UVFA, [55]). A UVFA is a value function $V^{\pi}(s, g)$ defined over both a state s and a goal g. The agent is given an initial state s_1 and a goal g. While executing its policy π, the agent may fail to reach g, ending up in a different state s_T. Although this trajectory does not help reach g, it is useful for s_T. By replacing the goal state with the terminal

state, $g \leftarrow s_T$, the trajectory can be used by any off-policy algorithm to learn how to reach s_T. This approach generates an implicit curriculum since easier-to-reach states are approached earlier in the training process.

The main limitation of this solution is the uniform replacement of all goals in previous trajectories, regardless of their relevance to achieving the desired performance. An enhanced version, *Curriculum-guided HER* (CHER, [18]), introduces the concepts of *curiosity* and *proximity*, based on distance and similarity metrics, respectively. Curiosity encourages the exploration of different goals, while proximity promotes goals closer to the target. At each optimization step, the objective is to select a subset of goals that maximize the weighted sum of diversity and proximity scores. The curriculum initially emphasizes diverse goals, gradually shifting towards proximity-based goals over time. Informally, this reflects the exploration-exploitation challenge, from a curriculum design lens.

Co-Learning

Continuing our exploration, it is now time for multi-agent systems. *Co-Learning* is a multi-agent framework in which the curriculum arises from the interaction of multiple agents with the environment. These agents can either cooperate or act adversarially to improve over time.*Asymmetric Self-Play*, [67], is based on a teacher-student framework. The teacher submits a task, while the student attempts to execute it. The two agents learn policies that optimize different objectives. The student tries to minimize the number of steps required to complete the task, while the teacher aims to maximize the gap between the student's solution time and the effort required for the task setup. Essentially, the teacher aims to find an activity that is easy to set up but challenging for the student to solve. This iterative process implicitly defines a curriculum as both agents learn and improve their performance.

Robust Adversarial RL (RARL, [45]), presents another idea. In this framework, a protagonist agent learns to achieve its goals while contending against disturbances applied by an adversary. The adversary, in turn, learns to progressively escalate the difficulty of the protagonist's tasks, initially presenting easier challenges and gradually increasing the level of difficulty. In general, adversarial settings have proven particularly effective for curriculum design, as the convergence of an agent to an optimal strategy drives the adversary to develop an optimal counter-strategy, naturally defining a cat-and-mouse game that creates a dynamic and escalating learning environment.

State visit distribution changes

We now move beyond the constraints of the *single-task* assumption, exploring curriculum approaches that embrace the complexities of *multi-task* learning. We start considering methods that do not change the state-action space of the problem, focusing instead on the state visit distribution. The framework presented in this manuscript, Chapter 5, falls into this category.

One of the simplest approaches is *Learning from Easy Missions*(LEM, [3]). The idea is to modify the initial state distribution to start closer to the goal state and gradually move further as the learning process goes on. The distance is measured according to a domain-specific heuristic. A more general framework, to perform this backward expansion is presented by *Reverse Curriculum Generation* (RCG, [19]). RGC takes a goal state as a seed and then generates nearby starting states through a random walk, considering some noise perturbation over actions. Both approaches are inspired by Dynamic Programming, where the solution for the final problem is built from the solutions of sub-problems. Once the agent learns how to behave in the proximity of the goal it can be moved a bit further. Leveraging prior knowledge implicitly shapes rewards, offering an alternative to explicitly designing complex reward signals.

No Restrictions

Finally, we will discuss some curriculum applications where sources and target tasks present a significant mismatch.

Formalizing the sequencing as an MDP, an interesting approach is denoted as *Curriculum MDP* (CurrMDP, [38]). This framework involves two interacting MDPs: the first one models the *learning agent*, i.e. the student, while the second one can be seen as a high-level *meta MDP*, modelling the *curriculum agent*, i.e. the teacher. In the curriculum agent, the state space $\mathcal{S}^{\mathcal{C}}$ is the set of parameterized policies the learning agent can learn, whereas the actions space $\mathcal{A}^{\mathcal{C}}$ is the set of tasks that the learning agent can train on. The process follows this logic: the student is initialized with a random policy, representing an initial state $s_0 \in \mathcal{S}^{\mathcal{C}}$, the teacher picks an action a, resulting in selecting a task for the learning agent. When the task is learned, the student updates its policy, causing a transition, following $P^{\mathcal{C}}$, in the meta-MDP. The reward can be measured as the number of time steps to learn the task. Under this scheme, the goal of the curriculum agent is to minimize the time needed by the learning agent to reach a policy achieving a desired performance threshold. Then, the curriculum is defined by the policy of the teacher $\pi^{\mathcal{C}}$, which specifies the sequence of tasks the learning agent will train on.

The sequencing phase can also be thought as a combinatorial optimization problem. Given a fixed set of tasks, the goal is to find the permutation that leads to the best curriculum, under some performance metrics. A popular class of methods relies on *meta-heuristics* algorithms[2]. In [20], the task sequencing problem is performed adapting four representative meta-heuristic algorithms: *beam search*[43], *tabu search*[23], *genetic algorithms*[24] and *colony optimization*[16]. The first two are *trajectory-based*. Starting from a guess of the solution, they explore its neighbourhood to perform an improvement. On the contrary, the last two are *population-based*. An initial set of candidate solutions is progressively improved according to a fitness score of the population.

4.4.3. Knowledge Transfer

We conclude this section by highlighting techniques aiming to successfully apply knowledge transfer to bias the exploration toward higher-valued states and speed up the learning process. A common challenge to cope with is known as *catastrophic forgetting*. Previously acquired knowledge can be lost while learning new tasks. To better fit new problems, the weight of parameter vector may be updated, eventually affecting performance on previous tasks. *Progressive Neural Networks* [54] propose a solution. A new output layer is trained for each task, relaying on interconnection to previously learned networks to share knowledge. o avoid catastrophic forgetting, some parameters are frozen, but issues remain. First, they do not scale with many source tasks, as the number of parameters grows accordingly. Second, having multiple output heads a task label at inference time to specify which layer should produce the output.

Transferring high-level knowledge solves the challenge. Focusing on partial policies, which provide action probabilities only for a subset of state-action pairs, we find [75]. The key concept is to progressively increase the state and action spaces while the agent is learning. The agent will learn a full policy on the source task, which in turn becomes partial when transferred to the following one. If tasks can be divided into subtasks, it is possible to share *options* [22]. An option is the policy learned on a sub-task, combining options of source tasks gives birth to the policy for the original problem.

As an emerging field, however, its full capabilities remain undiscovered. A thorough examination of available literature has revealed several open questions that we will highlight in the following section.

[2]Domain agnostic approaches that are not bounded to a specific problem, allowing a black-box usage.

4.5. Open Questions

1. **Theoretical results**.While empirical evidence supports the benefits of curricula, there is a lack of theoretical understanding, especially in RL contexts.

2. **Fully automated task creation**. The Effectiveness of the curriculum is determined by the quality of tasks that can be accessed. Existing works either assume that tasks are manually defined or present a set of rules for a semi-automatic task generation procedure, which however still require a manual interaction for hyper-parameter tuning and task feasibility checks.

3. **Combining task generation and sequencing**. Curriculum design consists of three parts: task generation, sequencing and knowledge transfer. Current works have tackled each of these steps independently. The ability to generate *online* the next task may be the key to devise an end-to-end curriculum solution, limiting human intervention.

4. **Reusing curricula**. One of the main limitations of CL is the effort required to design a good curriculum, however, there may be some application scenarios in which the same curriculum may be shared to train multiple agents, to amortize the overall costs.

5. **Transferring different types of knowledge**. In current works the type of knowledge transferred between tasks was fixed and defined apriori, however, this opens the question of whether transferring different types of knowledge among tasks may be beneficial. So, besides deciding just which task to transfer from, it may be an idea to evaluate what can be transferred.

6. **Understanding regular principles for curriculum design**. Determining an order criterion over samples and ensuring that what is provided to the agent is at the appropriate complexity level is a major challenge. Finding some general principles that non-expert humans follow while designing a curriculum could be helpful to guide an automatic task-generation procedure.

5 | Teleport MDP

In this chapter, we will introduce a novel framework that enables an implicit curriculum definition combining the task sequencing and the task generation phases, described in Section 4.2. First, we will outline the motivations that shaped the direction of this research. Then, in Section 5.2, we will define the *Teleport Markov Decision Processes* (TMDP), extending the MDP formalization to facilitate curriculum learning.

5.1. Introduction and motivations

Policy gradient algorithms struggle when the reward signal is sparse. Under this setting, gradient estimations for trajectories that fail to reach a rewarding state become uninformative. The absence of directional information makes it hard to determine which actions should be reinforced, hindering policy improvement and potentially converging to sub-optimal solutions. In Section 3.3.1, we discussed the necessity of stochastic policies for exploration, but they are not sufficient on their own. Additional techniques, like *Reward Shaping* [41], can significantly improve the learning process by driving the agent toward the goal. However, the main challenge lies in designing rewards that do not alter the optimal policy. If not carefully structured, reward shaping can lead to *reward hacking*, where the agent exploits the shaped rewards for high returns while deviating from optimal behaviour. *Entropy Regularization* [61], offers a way to modify the reward function by explicitly rewarding stochastic behaviour as follows:

$$\widetilde{r}_t = (1-\beta)r_t + \beta\mathbb{H}(\pi_\theta(\cdot|s_t)), \tag{5.1}$$

where $\mathbb{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a\sim s_t}[-log(\pi(\cdot|s_t)]$ is the *entropy* of the policy.

Encouraging the agent to take unexplored actions increases the likelihood of discovering new states. However, this indirect exploration may be inadequate in highly stochastic environments. The Chain MDP is a classic example of this challenge (Figure 5.1). Despite its straightforward structure, the environment presents a significant exploration barrier due to its sparse rewards and the complexity of reaching paying states. In such a case,

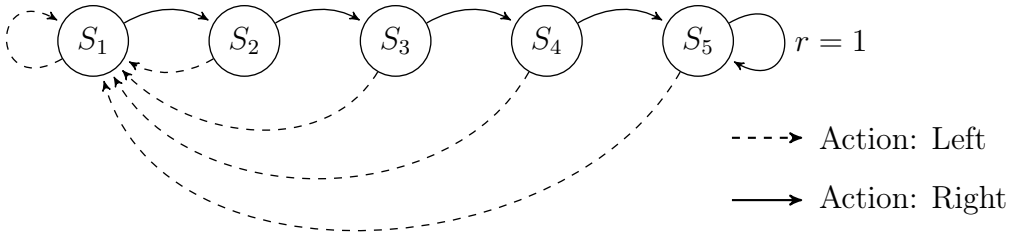


Figure 5.1: Chain MDP with 5 states. Actions are deterministic and omitted rewards are assumed to be 0.

a more direct approach is beneficial. Assuming having access to a *restart distribution*, the agent can gain information about states that may never reach otherwise. The idea is to draw the initial state from this distribution μ, rather than considering the initial state distribution μ_0 of the original formulation. A common choice for μ is a uniform probability distribution. However, a more effective approach is to introduce a curriculum. In Section 4.4.2 we mentioned (LEM, [3]) or (RCG, [19]). These methods initially restart near the goal and gradually shift further away as the agent learns. While effective, they have limitations. LEM requires manual configuration of the entire learning process, from task creation to sequencing. RCG automates these steps but, like LEM, relies on domain-specific heuristics to measure the distance from the goal, limiting their applicability and potentially biasing toward suboptimal behaviours.

In light of these challenges, our study aims to contribute to the theoretical foundations of CL while providing an integrated and scalable approach to curriculum design. The Teleport MDP framework addresses this by implicitly handling task generation and sequencing, minimizing manual intervention and making CL more accessible. The general idea is to extend the standard MDP formulation to promote curriculum learning. A *teleport agent* agent can efficiently explore the state space through teleportation, which occurs with rate $\tau \in [0, 1)$ and follows a teleport probability distribution, denoted as ξ. The fundamental assumption that must always hold is:

1. **Teleport Assumption**: We can arbitrarily reset the agent to any state $s \in \mathcal{S}$ (or a subset thereof) during each trajectory.

While this assumption is stronger than having access to restart distributions only at the beginning of a trajectory it is weaker than full knowledge of the environment's transition model. In practice can be reasonable, especially in simulated environments. A straightforward implementation involves early episode termination with probability τ drawing the subsequent starting state from the teleport distribution ξ, rather than the restart

distribution μ or the initial state distribution μ_0.

There is an implicit order to the values of τ: the higher the teleport rate, the easier the problem, as will be show in the following section. Thus, to implement a curriculum we must address two key questions:

1. **When** should we update τ?
2. **How** can we determine the new value of τ?

Consequently, generating a static curriculum becomes straightforward. The task generation and sequencing phases are simplified to selecting a series of values for the teleport rate τ with the ultimate goal to converge to $\tau = 0$, i.e. the original problem without teleportation. However, devising a dynamic curriculum that adapts to the agent's performance is the real challenge and will be the core focus of this manuscript. In the following section, we will formalize the *Teleport Markov Decision Process* (TMDP), enhancing the standard formulation of an MDP (Section 2.2) to support CL.

5.2. Framework definition

The Teleport MDP can be seen as a specific instance of the broader *Configurable MDP* (CMDP, [34]) framework. In CMDPs, agents modify environmental aspects, primarily the transition model, to optimize performance. Before delving into the similarities and key distinctions between the two approaches, let us formalize a Teleport MDP.

Definition 5.1 (Teleport MDP). *A Teleport MDP (TMDP) is an extension of a Markov Decision Process defined by the tuple $\mathcal{T} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma, \mu, \tau, \xi \rangle$ where:*

- *$\mathcal{S}$ is referred to as state space, representing the finite set of all states;*
- *$\mathcal{A}$ is denoted as action space, identifying the finite set of all actions;*
- *P is the transition probability matrix. Each element, $P(s'|s,a)$, expresses the probability of moving from state s, to state s' when action a is selected;*
- *R is known as the reward function $R(s,a) = \mathbb{E}[r|s,a]$. Given a state-action pair (s,a), it returns the expected value of the stochastic reward $r(s,a)$;*
- *$\gamma \in [0,1]$ is the discount factor. It is a property of the problem allowing to weight future rewards;*
- *μ describes the initial state distribution. Formally $\mu_s = P(s_0 = s)$, $\forall s \in \mathcal{S}$.*
- *$\tau \in [0,1)$ is the teleport rate. It specifies the frequency at which the teleport occurs.*

- *ξ is the state teleport probability distribution. It specifies a new probability distribution from which sampling states when a teleport occurs.*

Essentially, a TMDP introduces a competition between two probability distributions to govern transition dynamics, resulting in a new transition model function, P_τ.

$$P_\tau(s'|s,a) = (1-\tau)P(s'|s,a) + \tau\xi(s'). \tag{5.2}$$

Notation. *From now on, we will refer to this new probability transition function P_τ as the teleport transition matrix, the teleport dynamics or teleport transition model. It is important not to confuse this with the teleport probability distribution ξ.*

Informally, the agent can reach s' either following the regular transition probability function or through teleportation.

At first glance, given a value of $\tau = \overline{\tau}$, a TMDP could be thought as a CMDP where $\mathcal{P} = \{P_{\overline{\tau}}\}$ is the *model space*. This relationship holds even with multiple values of τ. Let us consider an **ordered list** of values of τ, $T = \{\tau_1, \tau_2, \ldots, \tau_n | \tau_i < \tau_{i+1}\}$ with $\tau_i \in [0, 1)$. The model space becomes $\mathcal{P} = \{\bigcup_i P_{\tau_i}\}$. However, the natural ordering of elements, associated with τ, is lost despite covering the same probability transition space. This relation is crucial for automating the sequencing phase, making TMDP suited for curriculum learning. To better grasp this concept, let us examine how the teleport mechanism works. There are two main effects:

(i) Teleportation enables *direct exploration* of the state space, not solely relying on the randomness of actions. This mechanism resembles restart distributions but offers finer-grained control over the agent's movement. While ξ can act as a restart distribution, TMDPs provide the additional advantage of controlling the teleport rate τ. This allows for agent relocation at any point during an episode, not just at the end, at the cost of introducing a bias.

(ii) When a teleport occurs, the current trajectory is *truncated*, and the agent restarts from a state s_{t+1} drawn from ξ. This suggests an interesting relationship with the discount factor, γ. From a probabilistic perspective, γ reflects the likelihood of continuing for another time step, guiding the agent's decision to prioritize immediate rewards or invest in future gains. The presence of an external factor (teleportation)

capable of ending an episode, can be seen as changing the effective discount factor:

$$\gamma_{\text{eff}} = \gamma(1 - \tau). \tag{5.3}$$

Several studies have demonstrated the effectiveness of varying the discount factor. In the planning setting ([10, 28]) have shown that the optimal solution may be found within the set of policies having a planning horizon γ smaller than the one specified by the problem definition. Other studies, such as [49] and [1], analyze the effect of γ as a regularizer. Focusing on Deep RL, [21] empirically showed that gradually increasing γ can effectively stabilize the learning process and improve convergence rates.

Controlling τ allows us to design a curriculum that starts with higher values, corresponding to a lower γ_{eff}, encouraging exploration and short-term goals. As learning progresses, τ is gradually reduced (thus moving γ_{eff} toward γ), shifting the focus to long-term planning.

This approach is particularly beneficial in sparse-reward environments. The agent is frequently teleported in the early stages, allowing it to explore the state space faster, failing upon rewarding states. These accidental discoveries provide valuable information that guides subsequent stages, where the agent can leverage this knowledge to reach states closer to the goal. The resulting curriculum, according to the categories introduced in Chapter 4, can be classified as *task-level sequence* curriculum.

A slight reformulation of some basic concepts in the MDP theory is necessary to adapt them to TMDP contexts, where modifying the teleport rate τ plays an active role in the learning process, alongside the policy itself. Building on this, it is interesting to evaluate the difference that two teleport dynamic functions, P_τ and $P_{\tau'}$, assigns to a given transition.

Lemma 5.1. *Given a TMDP $\mathcal{T}$, the difference between the probabilities that two teleport transition matrices $P_{\tau'}$ and P_τ assign to each transition is given by:*

$$P_{\tau'}(\cdot|s,a) - P_\tau(\cdot|s,a) = (\tau - \tau')\big(P(\cdot|s,a) - \xi(\cdot)\big). \tag{5.4}$$

Proof. Starting from the simple expansion of the teleport transition matrices, according

to (Equation 5.2) we obtain:

$$\begin{aligned} P_{\tau'}(\cdot|s,a) - P_{\tau}(\cdot|s,a) &= (1-\tau')P(\cdot|s,a) + \tau'\xi(\cdot) - (1-\tau)P(\cdot|s,a) - \tau\xi(\cdot) \\ &= (\tau-\tau')P(\cdot|s,a) - (\tau-\tau')\xi(\cdot) \\ &= (\tau-\tau')\big(P(\cdot|s,a) - \xi(s)\big). \end{aligned}$$

□

Introducing a fixed policy π, the Markov chain generated by the combination of a TMDP $\mathcal{T}$ with π, evolves over the state space $\mathcal{S}$ following the *teleport kernel* P_τ^π defined as:

$$\begin{aligned} P_\tau^\pi(\cdot|s) &= \sum_{a\in\mathcal{A}} \pi(a|s) P_\tau(\cdot|s,a) \\ &= (1-\tau)P^\pi(\cdot|s) + \tau\,\xi(\cdot). \end{aligned} \tag{5.5}$$

Proof. Combining equations 2.9 and 5.2 we obtain:

$$\begin{aligned} P_\tau^\pi(\cdot|s) &= \sum_{a\in\mathcal{A}} \pi(a|s) P_\tau(\cdot|s,a) \\ &= \sum_{a\in\mathcal{A}} \pi(a|s)\Big((1-\tau)P(\cdot|s,a) + \tau\,\xi(\cdot)\Big) \\ &= (1-\tau)\sum_{a\in\mathcal{A}} \pi(a|s)P(\cdot|s,a) + \tau \sum_{a\in\mathcal{A}} \pi(a|s)\xi(\cdot) \\ &= (1-\tau)P^\pi(\cdot|s) + \tau\,\xi(\cdot). \end{aligned} \tag{5.6}$$

□

Or, in vector notation:

$$P_\tau^\pi := (1-\tau)P^\pi + \tau\,\Xi, \tag{5.7}$$

where ξ is the teleport probability distribution and Ξ is defined as follows:

$$\Xi = \begin{bmatrix} \xi \\ \xi \\ \vdots \\ \xi \end{bmatrix}, \tag{5.8}$$

To define a dynamic curriculum that adapts to the agent's performance, it is useful to quantify the difference between two teleport transition kernels $P_{\tau'}^\pi$ and P_τ^π when the policy π is fixed.

Corollary 5.1. *Given a TMDP $\mathcal{T}$ and a policy π the difference between the two teleport transition kernels $P^{\pi}_{\tau'}$ and P^{π}_{τ} can be expressed as:*

$$P^{\pi}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s) = (\tau - \tau')\big(P^{\pi}(\cdot|s) - \xi(\cdot)\big). \tag{5.9}$$

Proof. Applying the expectation over policy's actions on Lemma 5.1, we obtain:

$$\begin{aligned}
\mathop{\mathbb{E}}_{a\sim\pi}\Big[P_{\tau'}(\cdot|s,a) - P_{\tau}(\cdot|s,a)\Big] &= (\tau - \tau')\sum_{a\in\mathcal{A}}\pi(a|s)\Big(P(\cdot|s,a) - \xi(\cdot)\Big) \\
&= (\tau - \tau')\sum_{a\in\mathcal{A}}\Big(\pi(a|s)P(\cdot|s,a) - \xi(\cdot)\sum_{a\in\mathcal{A}}\pi(a|s)\Big) \\
&= (\tau - \tau')\big(P^{\pi}(\cdot|s) - \xi(\cdot)\big).
\end{aligned}$$

□

Moving on with the analysis of the TMDP components, it's now the turn of the reward function. A general expression of the expected reward R_τ under the teleport transition model can be the following:

$$\begin{aligned}
R_\tau(s,a) &= \sum_{s'\in\mathcal{S}} P_\tau(s'|s,a)R(s,a,s') \\
&= (1-\tau)R(s,a) + \tau R_\xi(s,a),
\end{aligned} \tag{5.10}$$

where:

$$\begin{aligned}
R(s,a) &= \mathbb{E}_P\big[R(s,a,s') = \sum_{s'\in\mathcal{S}} P(s'|s,a)R(s,a,s')\big], \\
R_\xi(s,a) &= \mathbb{E}_\xi\big[R(s,a,s') = \sum_{s'\in\mathcal{S}} \xi(s')R(s,a,s')\big],
\end{aligned}$$

and $R(s,a,s')$ is the immediate reward upon reaching state s' after choosing action a in state s.

This formalization, accounting for the stochasticity of the environment, can be decomposed into two components. The former is the expected reward under the original transition model, weighted by $(1-\tau)$. The latter is the expected reward under the teleport probability distribution ξ, scaled by τ.

Remark 5.1. *The general formulation can be simplified under the assumption that a trajectory is truncated after a teleport. In this scenario, that reflects our setting, the term R_ξ becomes irrelevant. This is because, in practice, a teleport effectively ends the current episode and initiates a new one from a state $s' \sim \xi$. As a result, the tuple associated*

with the teleport itself is not employed in the learning process. While we maintain the general formulation for theoretical analysis, we adopt this simplified version for practical implementation.

Following the same rationale, let us formally define the difference of the expected rewards when moving from τ to τ'.

Lemma 5.2. *Given a TMDP $\mathcal{T}$ the reward difference that two expected reward under teleportation $R_{\tau'}$ and R_τ assigns to each stat-action pair is formally defined as:*

$$R_{\tau'}(s,a) - R_\tau(s,a) = (\tau - \tau')\big(R(s,a) - R_\xi(s,a)\big), \tag{5.11}$$

where $R(s,a,s')$ is the immediate reward upon reaching state s' after choosing action a in state s and depends entirely on the original problem dynamics. This formulation accounts for the effects of environmental stochasticity.

Proof. By expanding the two teleport reward functions $R_{\tau'}$ and R_τ, according to (Equation 5.10):

$$\begin{aligned} R_{\tau'}(s,a) - R_\tau(s,a) &= (1-\tau')R(s,a) + \tau' R_\xi(s,a) - (1-\tau)R(s,a) + \tau R_\xi(s,a) \\ &= (\tau - \tau')R(s,a) - (\tau - \tau')R_\xi(s,a) \\ &= (\tau - \tau')\big(R(s,a) - R_\xi(s,a)\big) \end{aligned}$$

□

The definition of the *value function* under a teleport transition matrix, V_τ^π requires careful consideration. Applying directly (Definition 2.6) in the context of TMDP, we obtain:

$$V_\tau^\pi(s) := \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim P_\tau(s_t,a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R_\tau(s_t,a_t) \Big| s_0 = s \right] = \mathop{\mathbb{E}}_{\nu_\tau \sim \rho_\tau^\pi} \big[G_\gamma(\nu_\tau) | s_0 = s \big], \tag{5.12}$$

where the trajectory ν_τ is sampled from the distribution of trajectories under teleport transition function $\rho_\tau^\pi(\nu_\tau)$, derived by simply replacing P with P_τ from (Equation 2.8).

Remark 5.2. *It is worth emphasising that, as noted in Remark 5.1, our practical implementation uses a reward function that disregards teleport rewards due to trajectory truncation. This approach directly yields the result $V_\tau^\pi = (1-\tau)V^\pi$, which is evident from Equation 5.10 when we neglect the influence of R_ξ. This reasoning extends to all value functions introduced in this framework.*

Accordingly, the *state action value function* is defined as:

$$Q_\tau^\pi(s,a) \coloneqq \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim P_\tau(s_t,a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R_\tau(s_t,a_t) \middle| s_0 = s, a_0 = a \right]. \tag{5.13}$$

To learn the transition model, we introduce the state-action-next state value function, denoted by U_τ^π. This quantity expresses the expected return when taking action a in state s leads into the next state s':

$$U_\tau^\pi(s,a,s') = R(s,a,s') + \gamma V_\tau^\pi(s'). \tag{5.14}$$

It is worth noting that the three value functions, V_τ^π, Q_τ^π and U_τ^π are closely connected by the following relations:

$$V_\tau^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q_\tau^\pi(s,a) \tag{5.15}$$

$$Q_\tau^\pi(s,a) = \sum_{s' \in \mathcal{S}} P_\tau^\pi(s'|s,a) U_\tau^\pi(s,a,s') \tag{5.16}$$

5.2.1. Performance Metric

As in (CMDP, [34]), a new perspective on performance evaluation rises. We cannot rely solely on the effectiveness of the policy, we must also consider the impact of updating the model. Thus, the performance of a model-policy pair (P_τ, π) can be evaluated as:

$$J_{\mu,\tau}^\pi = \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d_{\mu,\tau}^\pi(s) \sum_{a \in \mathcal{A}} \pi(a|s) R_\tau(s,a), \tag{5.17}$$

where $d_{\mu,\tau}^\pi$ identifies the γ-discounted state visit distribution:

$$d_{\mu,\tau}^\pi(s) = (1-\gamma)\mu(s) + \gamma \sum_{s' \in \mathcal{S}} d_{\mu,\tau}^\pi(s') P_\tau^\pi(s'|s). \tag{5.18}$$

Informally, it represents the discounted probability of landing in state s during an episode. This could happen either by starting directly in state s, reaching it through regular model transitions or even through teleportation. Similarly, we can also introduce the γ-discounted state-action distribution:

$$\delta_{\mu,\tau}^\pi(s,a) = \pi(a|s) d_{\mu,\tau}^\pi(s) \tag{5.19}$$

Remark 5.3. *To avoid any misunderstanding a clarification is needed. Although TMDPs share similarities with CMDPs, their focus diverges substantially. While the objective of a CMDP is to find the model-policy pair $(P^\star, \pi^\star)$ that maximizes performance, in a TMDP the target model for optimization is known and fixed: the original one. Nonetheless, evaluating the impact of both the model and the policy on performance remains crucial for determining how to progressively converge to $\tau = 0$, effectively implementing the curriculum strategy.*

5.2.2. Advantage Functions

In Section 2.2.4 we introduced the advantage function (Equation 2.19) to measure the benefits of action a in state s. In the TMDP context, we will refer to this quantity as the *policy advantage function*, expressed as:

$$A^\pi_\tau(s,a) = Q^\pi_\tau(s,a) - V^\pi_\tau(s). \tag{5.20}$$

It is also interesting to quantify the benefits of explicitly reaching s', when picking action a in state s. To this end, we formalize the *model advantage function* as:

$$A^\pi_\tau(s,a,s') = U^\pi_\tau(s,a,s') - Q^\pi_\tau(s,a). \tag{5.21}$$

Extending these definitions, we can quantify the *one-step* performance improvement when moving from a policy π to a new one π'. Formally, we define the *relative policy advantage function*:

$$A^{\tau,\pi'}_{\tau,\pi}(s) = \sum_{a\in\mathcal{A}} \pi'(a|s) A^\pi_\tau(s,a), \tag{5.22}$$

This quantity is significant to guarantee safe updates regarding monotonic performance improvements. Evaluating the relative advantage allows us to determine whether or not it is beneficial to move to the next policy. This concept extends to RL the idea of the policy improvement step applied by Policy Iteration algorithms in the context of dynamic programming (Section 2.3.2). Similarly, when changing τ to τ' (resulting in a shift from P_τ to $P_{\tau'}$), we obtain the *relative model advantage function*:

$$A^{\tau',\pi}_{\tau,\pi}(s,a) = \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a) A^\pi_\tau(s,a,s'). \tag{5.23}$$

Then, evaluating the expectation under the γ-discounted state visit distributions (see

Appendix A for the proofs) we get the *expected relative policy advantage function*:

$$\begin{aligned}\mathbb{A}^{\pi',\tau}_{\pi,\mu} &= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau,\pi'}_{\tau,\pi}(s) \\ &= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \Big(\pi'(a|s) - \pi(a|s)\Big) Q^{\pi}_{\tau}(s,a),\end{aligned} \tag{5.24}$$

and the *expected relative model advantage function*:

$$\mathbb{A}^{\tau',\pi}_{\tau,\mu} = \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} \delta^{\pi}_{\mu,\tau}(s,a) A^{\tau',\pi}_{\tau,\pi}(s,a) = (\tau - \tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}\,, \tag{5.25}$$

where $\mathbb{A}^{\xi,\pi}_{P,\tau,\mu} = \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} \delta^{\pi}_{\mu,\tau}(s,a) \sum_{s'\in\mathcal{S}} \Big(P(s'|s,a) - \xi(s')\Big) U^{\pi}_{\tau}(s,a,s')$.

Besides defining the decoupled advantage terms for the policy and the model, we seek to quantify the combined benefit of selecting action a and transitioning to state s' from state s, instead of following the current policy-model pair's action and state distribution, in line with (CMDP, [34]). To achieve this, we introduce the *kernel advantage function* as follows:

$$\widetilde{A}^{\pi}_{\tau}(s,a,s') = U^{\pi}_{\tau}(s,a,s') - V^{\pi}_{\tau}(s) \tag{5.26}$$

It is important to highlight the relationship between the kernel advantage function and the policy and model advantage terms:

$$\widetilde{A}^{\pi}_{\tau}(s,a,s') = A^{\pi}_{\tau}(s,a,s') - A^{\pi}_{\tau}(s,a)$$

Building on what we did for decoupled advantage terms, we can devise the *coupled relative advantage function* when performing a one-step improvement from the model-policy pair (P_τ, π) to the new one $(P_{\tau'}, \pi')$ as:

$$A^{\tau',\pi'}_{\tau,\pi}(s) = \sum_{s'\in\mathcal{S}} \sum_{a\in\mathcal{A}} \pi'(a|s) P_{\tau'}(s'|s,a) \widetilde{A}^{\pi}_{\tau}(s,a,s') \tag{5.27}$$

and the corresponding expected value under the γ-discounted distribution:

$$\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} = \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau',\pi'}_{\tau,\pi}(s). \tag{5.28}$$

The connection between these *expected advantages* can be formally expressed by the following lemma (the proof is available in Appendix A):

Lemma 5.3. *The following equality relates the coupled relative advantage function and the relative advantage functions of the policy and the model:*

$$A_{\tau,\pi}^{\tau',\pi'}(s) = A_{\tau,\pi}^{\tau,\pi'}(s) + \sum_{a \in \mathcal{A}} \pi'(a|s) A_{\tau,\pi}^{\tau',\pi}(s,a)$$

In the subsequent chapter, we leverage Equation 5.28 to establish a lower bound on performance improvement within the TMDP framework. We initially present results considering the combined effect of updating both the model and the policy. Subsequently, following Lemma 5.3, we decouple these effects to derive a bound that can be optimized with greater precision.

6 | A Curriculum Approach for TMDPs

In this chapter, we present a curriculum-learning approach that leverages the structure of TMDPs to automate and integrate task generation and sequencing.

It is important to note that this work is not intended as a definitive solution for devising a curriculum learning strategy in the context of RL, rather we aim to establish an initial framework upon which future research can build and elaborate.

Extending the Safe Policy Iteration framework [30, 34, 44, 46] we present results for theoretical performance guarantees within the TMDP domain. Under specific limiting assumptions, in Section 6.2 we introduce a theoretical algorithm that serves as a benchmark, despite not being directly applicable to real-world scenarios. Finally, in Section 6.3 relax these constraints, developing a practical and adaptable solution without relying on heuristics or prior knowledge.

6.1. Theoretical foundations

In this section, our objective is to establish a lower bound for the performance gain when transitioning from the model-policy pair (P_τ, π) to $(P_{\tau'}, \pi')$. Drawing inspiration from the CMDP framework [34]), we first adapt to the context of TMDP a series of upper bounds on the difference of the γ-discounted state visit distribution when updating the model-policy pairs. By leveraging these results we will determine the corresponding lower bounds on the resulting performance improvement (all the proofs can be found in Appendix A).

6.1.1. Bounds on state visit distribution

We establish an upper bound on the shift of the γ-discounted state visitation distributions resulting from changing the model-policy pairs. This bound accounts for the combined influence of both policy and model variations.

Proposition 6.1 (Proposition 5.1.1 from [34]). *Let (P_τ, π) and $(P_{\tau'}, \pi')$ two model-policy pairs, where P_τ and $P_{\tau'}$ are the probability transition matrix associated with parameters τ and τ'. The L_1-norm of the difference between the γ-discounted state distributions can be upper-bounded as:*

$$\left\| d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau} \right\|_1 \leq \frac{\gamma}{1-\gamma} D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}}, \tag{6.1}$$

where $D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\| P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s) \right\|_1$.

This proposition bounds the difference between γ-discounted state distributions using the state kernel dissimilarity evaluated through a pre-metric [15]. While the state kernel captures the combined effects of both policy and transition model variation, from a practical perspective it is useful to separate their contributions, obtaining the following looser bound.

Corollary 6.1. *Let (P_τ, π) and $(P_{\tau'}, \pi')$ two model-policy pairs. The L_1-norm of the difference between the γ-discounted state distributions can be upper-bounded as:*

$$\left\| d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau} \right\|_1 \leq \frac{\gamma}{1-\gamma} (D_{\mathbb{E}}^{\pi',\pi} + |\tau - \tau'| D_{\mathbb{E}}^{P,\xi}), \tag{6.2}$$

where $D_{\mathbb{E}}^{\pi',\pi} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\| \pi'(\cdot|s) - \pi(\cdot|s) \right\|_1$ *and* $D_{\mathbb{E}}^{P,\xi} := \mathbb{E}_{(s,a)\sim \delta^{\pi}_{\mu,\tau}} \left\| P(\cdot|s,a) - \xi(\cdot) \right\|_1$.

Note. *It is important to note that the absolute value on the difference of the teleport probabilities, $|\tau' - \tau|$, is necessary for mathematical rigour and generalization. However, in our specific scenario where the goal is to monotonically converge towards $\tau = 0$, the absolute value can be omitted without loss of generality.*

Teleport Bias

The TMDP framework extends the standard MDP by introducing two key elements: the teleport probability, τ, which governs the frequency of teleportations, and the teleport probability distribution ξ, specifying where to sample the next state when a teleport occurs. While this augmentation enhances exploration and potentially simplifies the learning of complex environments, it also introduces a bias that must be carefully evaluated. Quantifying this bias is crucial for assessing performance and designing effective algorithms within the TMDP framework. Therefore, we will begin by isolating the contributions of the policy and model distribution shifts and then providing a formal definition of the teleportation bias.

Corollary 6.2 (Corollary 5.1.3 from [34]). *Let π and π' be two policies, the L_1-norm of the difference between the γ-discounted state distribution, when $\tau = \tau'$ can be upper-bounded as:*

$$\|d^{\pi'}_{\mu,\tau} - d^{\pi}_{\mu,\tau}\|_1 \leq \frac{\gamma}{1-\gamma} D^{\pi',\pi}_{\mathbb{E}}, \tag{6.3}$$

where $D^{\pi',\pi}_{\mathbb{E}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\|\pi'(\cdot|s) - \pi(\cdot|s)\right\|_1$.

Proof. From Equation 6.2: $D^{P_\tau,P_\tau}_{\mathbb{E}} := \mathbb{E}_{(s,a)\sim\delta^{\pi}_{\mu,\tau}} \left\|P_\tau(\cdot|s,a) - P_\tau(\cdot|s,a)\right\|_1 = 0.$ □

Symmetrically, when $\pi = \pi'$, we can focus on the difference obtained with two different teleport probabilities τ and τ':

Corollary 6.3. *Let P_τ and $P_{\tau'}$ two probability transition matrix. The L_1-norm of the difference between the γ-discounted state distributions, when fixing the policy $\pi = \pi'$ can be upper-bounded as:*

$$\|d^{\pi}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\|_1 \leq \frac{\gamma|\tau - \tau'|}{1-\gamma} D^{P,\xi}_{\mathbb{E}} \tag{6.4}$$

where $D^{P,\xi}_{\mathbb{E}} := \mathbb{E}_{(s,a)\sim\delta^{\pi}_{\mu,\tau}} \left\|P(\cdot|s,a) - \xi(\cdot)\right\|_1$.

Proof. From Equation 6.2, trivially $D^{\pi,\pi}_{\mathbb{E}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\|\pi(\cdot|s) - \pi(\cdot|s)\right\|_1 = 0.$ □

We can now quantify the teleport bias by evaluating (Corollary 6.3) when $\tau' = 0$.

Corollary 6.4. *Let $\mathcal{T}$ be a TMDP, the bias introduced by the teleport probability τ and the teleport distribution ξ can be upper bounded as:*

$$Bias(\tau) := \|d^{\pi}_{\mu} - d^{\pi}_{\mu,\tau}\|_1 \leq \frac{\gamma\,\tau}{1-\gamma} D^{P,\xi}_{\mathbb{E}}, \tag{6.5}$$

where d^{π}_{μ} is the γ-discounted state visit distribution of the original problem, i.e. $\tau' = 0$.

6.1.2. Bound on performance improvement

Following this direction, we will now derive a series of lower bounds on performance improvement. Leveraging the definition of the coupled relative advantage function (Equation 5.27) and considering the combined effect of both the model and policy we can establish the following lower bound on performance improvement.

Theorem 6.1 (Theorem 5.1.1 from [34]). *The performance improvement of the model-policy pair (P_τ, π) over $(P_{\tau'}, \pi')$ is given by:*

$$J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau} = \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) A^{\tau',\pi'}_{\tau,\pi}(s). \tag{6.6}$$

The nonlinear and challenging relation between $d^{\pi'}_{\mu,\tau'}$ and the subsequent model-policy pair $(P_{\tau'}, \pi')$ prevents a direct application of this theorem. Therefore, we intend to establish a computationally efficient lower bound for performance difference, only relying on information from the current model-policy pair (P_τ, π).

Theorem 6.2 (Coupled Bound, Theorem 5.1.2 from [34]). *The performance improvement of the model-policy pair $(P_{\tau'}, \pi')$ over (P_τ, π), can be lower bounded as:*

$$\underbrace{J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau}}_{\substack{\textit{performance}\\ \textit{improvement}}} \geq \underbrace{\frac{\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}}{1-\gamma}}_{\textit{advantage}} - \underbrace{\frac{\gamma \Delta A^{\tau',\pi'}_{\tau,\pi} D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}}}{2(1-\gamma)^2}}_{\textit{dissimilarity penalization}}, \tag{6.7}$$

where $\Delta A^{\tau',\pi'}_{\tau,\pi} = \sup_{s,s' \in \mathcal{S}} |A^{\tau',\pi'}_{\tau,\pi}(s') - A^{\tau',\pi'}_{\tau,\pi}(s)|$.

Proof. From the bound on the γ-discounted state distribution (Proposition 6.1) we can obtain:

$$\begin{aligned}
J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau'} &= \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) A^{\tau',\pi'}_{\tau,\pi}(s) \pm \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau',\pi'}_{\tau,\pi}(s) \\
&= \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau',\pi'}_{\tau,\pi}(s) + \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} \big(d^{\pi'}_{\mu,\tau'}(s) - d^{\pi}_{\mu,\tau}(s)\big) A^{\tau',\pi'}_{\tau,\pi}(s) && (6.8) \\
&\geq \frac{\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}}{1-\gamma} - \frac{1}{1-\gamma} \Big| \sum_{s \in \mathcal{S}} \big(d^{\pi'}_{\mu,\tau'}(s) - d^{\pi}_{\mu,\tau}(s)\big) A^{\tau',\pi'}_{\tau,\pi}(s) \Big| && (6.9) \\
&\geq \frac{\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}}{1-\gamma} - \frac{1}{1-\gamma} \|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\|_1 \frac{\Delta A^{\tau',\pi'}_{\tau,\pi}}{2} && (6.10) \\
&\geq \frac{\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}}{1-\gamma} - \frac{\gamma A^{\tau',\pi'}_{\tau,\pi} D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}}}{2(1-\gamma)^2} && (6.11)
\end{aligned}$$

where Equation 6.9 is obtained from Equation 6.8 by noting that $b \geq -|b|$, Equation 6.10 is derived from Equation 6.9 through the application of (Corollary 2.4 from [26]), and Equation 6.11 is a result of Proposition 6.1. □

The bound consists of two components, similar to those in [46] and [30]. The first, called

advantage, quantifies the potential performance improvement attainable by updating the current model-policy pair (P_τ, π) with $(P_{\tau'}, \pi')$. The second component, *dissimilarity penalization*, constrains updates resulting in excessive dissimilarity between the original and updated models and policies.

In practical applications, the *coupled bound* could not always be appropriate, e.g. in certain scenarios, we may have limited control over the model while having full access to the policy, or vice-versa. Therefore, decoupling the contributions of policy and model updates is beneficial. By establishing a connection between the kernel advantage and the terms solely associated with the policy and model, and applying Corollary 6.1, it is possible to derive the following lemma (see Appendix A for the proof).

Lemma 6.1. *The following bound relates the joint relative advantage function and the relative advantage functions:*

$$\left|\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} - \left(\mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi}\right)\right| \leq \gamma|\tau - \tau'| D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \frac{\Delta U^{\pi}_{\tau}}{2}, \tag{6.12}$$

where: $D^{\pi',\pi}_{\mathbb{E}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\|\pi'(\cdot|s) - \pi(\cdot|s)\right\|_1$, $D^{P,\xi}_{\infty} := \sup_{s\in\mathcal{S},a\in\mathcal{A}} \|P(\cdot|s,a) - \xi(\cdot)\|_1$ *and* $\Delta U^{\pi}_{\tau} := \sup_{\substack{s,s',\overline{s},\overline{s}'\in\mathcal{S},\\ a,a'\in\mathcal{A}}} \left|U^{\pi}_{\tau}(s',a',\overline{s}') - U^{\pi}_{\tau}(s,a,\overline{s})\right|$.

Moreover, to better isolate the model and policy contributions, the next lemma provides an upper bound also for the term $\Delta A^{\tau',\pi'}_{\tau,\pi}$.

Lemma 6.2. *Let (P_τ, π) and $(P_{\tau'}, \pi')$ be two model-policy pairs, it holds that:*

$$\frac{\Delta A^{\tau',\pi'}_{\tau,\pi}}{2} \leq (D^{\pi',\pi}_{\infty} + \gamma|\tau - \tau'| D^{P,\xi}_{\infty}) \frac{\Delta U^{\pi}_{\tau}}{2}. \tag{6.13}$$

As a direct consequence of Lemmas 6.1 and 6.2 we develop the following theorem.

Theorem 6.3 (Teleport Bound)**.** *The performance improvement of the model-policy pair $(P_{\tau'}, \pi')$ over (P_τ, π), can be lower bounded as:*

$$\underbrace{J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau}}_{\substack{\textit{performance}\\ \textit{improvement}}} \geq B(\pi, P_\tau) = \underbrace{\frac{\mathbb{A}^{\pi',\tau}_{\pi,\mu} + (\tau - \tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma}}_{\textit{advantage}} - \underbrace{\frac{\gamma \Delta U^{\pi}_{\tau} D}{2(1-\gamma)^2}}_{\substack{\textit{dissimilarity}\\ \textit{penalization}}} \tag{6.14}$$

where D is a dissimilarity term defined as:

$$D = D^{\pi',\pi}_{\mathbb{E}} (D^{\pi',\pi}_{\infty} + |\tau - \tau'| D^{P,\xi}_{\infty}) + (\tau - \tau') D^{P,\xi}_{\mathbb{E}} (D^{\pi',\pi}_{\infty} + \gamma|\tau - \tau'| D^{P,\xi}_{\infty}).$$

Proof. By applying the bounds established in Lemma 6.1 for the coupled expected relative advantage $\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}$ in Lemma 6.2 for $\Delta A^{\tau',\pi'}_{\tau,\pi}$, and in Corollary 6.1 for the γ-discounted state distribution difference, and by recalling Equation 5.25, we get:

$$\begin{aligned}
J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau} &\geq \frac{\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}}{1-\gamma} - \frac{\|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\|_1}{1-\gamma}\frac{\Delta A^{\tau',\pi'}_{\tau,\pi}}{2} \\
&\geq \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu} + (\tau-\tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} - \frac{\gamma|\tau-\tau'|}{1-\gamma} D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \frac{\Delta U^{\pi}_{\tau}}{2} + \\
&\quad - \frac{\gamma}{(1-\gamma)^2}(D^{\pi',\pi}_{\mathbb{E}} + |\tau-\tau'| D^{P,\xi}_{\mathbb{E}})(D^{\pi',\pi}_{\infty} + \gamma|\tau-\tau'| D^{P,\xi}_{\infty})\frac{\Delta U^{\pi}_{\tau}}{2} \geq \\
&\geq \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu} + (\tau-\tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} - \frac{\gamma \Delta U^{\pi}_{\tau}}{2(1-\gamma)^2}\Big(D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty} + |\tau-\tau'| D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} + \\
&\quad + |\tau-\tau'| D^{P,\xi'}_{\mathbb{E}} D^{\pi',\pi}_{\infty} + \gamma(\tau-\tau')^2 D^{P,\xi'}_{\mathbb{E}} D^{P,\xi}_{\infty}\Big).
\end{aligned}$$

□

Separating the effects of the policy and model updates yields the following corollaries.

Corollary 6.5. *The performance improvement of the policy pair π' over π, when $\tau' = \tau$, can be lower bounded as:*

$$\underbrace{J^{\pi'}_{\mu,\tau} - J^{\pi}_{\mu,\tau}}_{\substack{\textit{performance}\\ \textit{improvement}}} \geq \underbrace{\frac{\mathbb{A}^{\pi',\tau}_{\pi,\mu}}{1-\gamma}}_{\textit{advantage}} - \underbrace{\frac{\gamma \Delta U^{\pi}_{\tau} D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty}}{2(1-\gamma)^2}}_{\textit{dissimilarity penalization}} \tag{6.15}$$

Corollary 6.6. *The performance improvement of the model $P'_{\tau'}$ over P_{τ}, when $\pi' = \pi$, can be lower bounded as:*

$$\underbrace{J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau}}_{\substack{\textit{performance}\\ \textit{improvement}}} \geq \underbrace{\frac{(\tau-\tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma}}_{\textit{advantage}} - \underbrace{\frac{\gamma^2(\tau-\tau')^2 \Delta U^{\pi}_{\tau} D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}}{2(1-\gamma)^2}}_{\textit{dissimilarity penalization}} \tag{6.16}$$

Having established the necessary foundation, we are ready to introduce the *Teleport Model Policy Iteration* (TMPI) algorithm.

Table 6.1: Candidate optimal pairs for the teleport bound. The optimal pair is the one leading to the highest value. All values are clipped within the range $[0, 1]$.

$\overline{\alpha}_\tau = \frac{(1-\gamma)\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{\gamma \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty}}$	$\overline{\tau}' = \tau$
$\overline{\alpha} = 0$	$\overline{\tau}'_0 = \tau - \frac{(1-\gamma)\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{\gamma^2 \Delta U^\pi_\tau D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}}$
$\overline{\alpha}_0 = \overline{\alpha}_\tau - \frac{\tau}{2}\left(\frac{D^{P,\xi}_{\mathbb{E}}}{D^{\pi',\pi}_{\mathbb{E}}} + \frac{D^{P,\xi}_{\infty}}{D^{\pi',\pi}_{\infty}}\right)$	$\tau' = 0$
$\overline{\alpha} = 1$	$\overline{\tau}'_1 = \overline{\tau}'_0 + \frac{1}{2\gamma}\left(\frac{D^{\pi',\pi}_{\mathbb{E}}}{D^{P,\xi}_{\mathbb{E}}} + \frac{D^{\pi',\pi}_{\infty}}{D^{P,\xi}_{\infty}}\right)$

6.2. Teleport Model Policy Iteration

In this section, we present the *Teleport Model Policy Iteration* algorithm, which addresses curriculum learning within the Teleport MDP (TMDP) framework. As discussed in Section 4.2, curriculum design encompasses task generation, sequencing, and transfer learning. We adopt a low-level transfer approach, initializing the policy and value functions of the new task $\tau^{(t+1)}$ using those learned in the previous one $\tau^{(t)}$. The remaining challenges of task generation and sequencing are addressed jointly by determining when and how to update the teleport rate τ, addressing the questions presented in Section 5.1. Implementing a static curriculum involves the definition of a pre-defined schedule for updating τ. In contrast, devising a dynamic curriculum that adapts to the agent's performance poses a greater challenge. The solution we adopt extends the Safe Policy Iteration framework [30, 34, 44, 46], leveraging the lower bounds established in the previous section to guide the process.

6.2.1. The algorithm

The general idea is to start with higher values of the teleport rate τ, whose initial value becomes a hyperparameter, and then iteratively update both the policy π and the teleport rate τ according to the following rules:

$$\pi' = \alpha^\star \overline{\pi} + (1 - \alpha^\star)\pi, \qquad \alpha^\star \in [0, 1],$$
$$P_{\tau'} = (1 - \tau^\star)P + \tau^\star \xi, \qquad \tau^\star \in [0, \tau],$$

where $\overline{\pi}$ is the target policy, $\alpha^\star$ and $\tau^\star$ are the values that jointly optimize the lower bound on performance improvement, established by the Teleport Bound (Theorem 6.3).

Note. *For clarity, we have explicitly presented the update rule for the teleport transition matrix $P_{\tau'}$. However, it is important to note that this model update is a direct consequence of updating the teleport rate with the rule $\tau' \leftarrow \tau^\star$. Furthermore, we will henceforth refer to the Teleport Bound as $B(\alpha, \tau')$.*

The following theorem presents a significant result concerning the lower bound optimization (Appendix A provides the complete proof).

Theorem 6.4. *For any policy $\overline{\pi} \in \Pi$ and teleport transition model $P_\tau \in \mathcal{P}$, the lower bound is optimized for:*

$$\alpha^\star, \tau^\star = \arg\max_{\alpha,\tau'} \{B(\alpha, \tau') : (\alpha, \tau') \in \mathcal{V}\},$$

where $\mathcal{V} = \{(\overline{\alpha}_\tau, \tau, (0, \overline{\tau}_0), (\overline{\alpha}_0, 0), (1, \overline{\tau}_1)\}$. The candidate coefficients are presented in Table 6.1, with the corresponding Teleport Bound values shown in Table 6.2.

The theorem specifies that the optimal pair $(\alpha^\star, \tau^\star)$ lies on the boundaries, defined over the space $[0, 1] \times [0, \tau]$. As a consequence, either the policy or the teleport rate is updated while the other remains fixed, or one is updated and the other is set to target. This result draws inspiration from Theorem 5.3.1 of (CMDP, [34]), but with a crucial distinction. Within the CMDP framework, the target $\overline{P}$ is chosen as the greedy model with respect to the U function (Equation 5.14) and optimized in conjunction with the policy, aligning the two updates. In contrast, TMDP focuses on progressively reducing τ to 0, thereby converging to the original transition dynamics, P. Knowing in advance the target model, our objective is to guide the agent's exploration of the state and action spaces in a way

$B(\overline{\alpha}_\tau, \tau) = \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}{}^2}{2\gamma \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty}},$
$B(0, \overline{\tau}'_0) = \frac{\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}{}^2}{2\gamma^2 \Delta U^\pi_\tau D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}},$
$B(\overline{\alpha}_0, 0) = \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}{}^2}{2\gamma \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi,}_{\infty}} + \frac{\tau \mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} - \frac{\tau \mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{2(1-\gamma)} \left(\frac{D^{P,\xi}_{\infty}}{D^{\pi',\pi}_{\infty}} + \frac{D^{P,\xi}_{\mathbb{E}}}{D^{\pi',\pi}_{\mathbb{E}}} \right) + -\frac{\gamma \tau^2 \Delta U^\pi_\tau}{2(1-\gamma)^2} \left(\gamma D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty} - \frac{\left(D^{\pi',\pi}_{\infty} D^{P,\xi}_{\mathbb{E}} + D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \right)^2}{4 D^{\pi',\pi}_{\infty} D^{\pi',\pi}_{\mathbb{E}}} \right),$
$B(1, \overline{\tau}'_1) = \frac{\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}{}^2}{2\gamma^2 \Delta U^\pi_\tau D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}} + \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{1-\gamma} + \frac{\Delta U^\pi_\tau}{8(1-\gamma)^2} \frac{\left(D^{\pi',\pi}_{\infty} D^{P,\xi}_{\mathbb{E}} + D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \right)^2}{D^{P,\xi}_{\infty} D^{P,\xi}_{\mathbb{E}}} + -\frac{1}{2\gamma(1-\gamma)} \left(\mathbb{A}^{\xi,\pi}_{P,\tau,\mu} \left(\frac{D^{\pi',\pi}_{\infty}}{D^{P,\xi}_{\infty}} + \frac{D^{\pi',\pi}_{\mathbb{E}}}{D^{P,\xi}_{\mathbb{E}}} \right) + \frac{\gamma^2 \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty}}{1-\gamma} \right)$

Table 6.2: Teleport bound values for each candidate optimal pair (α, τ').

that provides an advantage over learning the target task from scratch. To this end, the update rules for the teleport rate τ' ensure monotonic non-increase by clipping the new value to the interval $[0, \tau]$, where τ represents the previous value of the teleport rate.

It is important to highlight that the expected relative advantage $\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu}$ (Equation 5.28) being based on the current model-policy pair (P_τ, π) provides a *local* measure of performance improvement. Conversely, the actual performance improvement $J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau}$ offers a global perspective, considering the expectation over $d^{\pi'}_{\mu,\tau'}$ (Theorem 6.1). This distinction aligns with the *measure mismatch* phenomenon identified by Kakade [29], since the expected relative advantage might be zero despite a positive actual performance improvement. Consequently, TMPI, similar to other safe policy iteration algorithms could get stuck in local optima. Informally, guarantees of performance improvement could hinder the exploration of lower-performance regions, potentially necessary to achieve optimality.

Algorithm 6.1 Teleport Model Policy Iteration (TMPI)

Limiting Assumptions:

(i) Reset the agent to a random state during each trajectory.
(ii) Full knowledge of the model dynamics.
(iii) Non-negative rewards.

Require: Initial policy $\pi^{(0)}$, teleport rate $\tau^{(0)} \approx 0.5$, batch size N and epochs E.

```
for t=0,1,2... until convergence do
  for N trajectories do
    while ¬terminated ∨ ¬teleport do                ▷ Teleport sampling loop
      Sample p ∼ U(0,1)
      if p ≤ τ^(t) then                             ▷ Teleport branch
        s' ∼ ξ                                      ▷ Assumption (i) required.
        teleport = True
      else                                          ▷ Regular branch
        Sample a ∼ π^(t)(·|s)
        Observe reward r, next state s'
        ...
      end if
    end while
  end for
  Learn π̄^(t) processing the batch of N trajectories for E epochs
  Compute the terms A^{π',τ}_{π,μ}, A^{ξ,π}_{P,τ,μ}, ΔU^π_τ and D      ▷ Assumption (ii) required.
  Compute candidate pairs: V^(t) = {(ᾱ^(t)_τ, τ^(t)), (0, τ̄^(t)_0), (ᾱ^(t)_0, 0), (1, τ̄^(t)_1)}
  Get optimal pair: α*, τ* = arg max_{α,τ}{B(α,τ) : (α,τ) ∈ V^(t)}
  Update policy π^(t+1) = α* π̄^(t) + (1 − α*) π^(t)
  Update teleport rate τ^(t+1) ← τ*
end for
```

Algorithm 6.1 outlines the general structure of Teleport Model Policy Iteration (TMPI). The main difference from traditional reinforcement learning algorithms lies in the sampling loop. In the context of TMDP, we refer to it as the *teleport sampling loop*, where the agent's next state is determined not only by the environment's transition dynamics but also by the teleport rate τ and the associated probability distribution ξ. The easiest way to implement this behaviour involves truncating the episode upon teleportation and restarting the agent from a state sampled from ξ, rather than the initial probability distribution. In this setup, the teleport assumption (Section 5.1) resembles having access to a restart distribution at the beginning of an episode. However, more sophisticated implementations provide greater flexibility, such as applying a curriculum to the teleport distribution ξ, allowing several teleportations within a single trajectory and accommodating vector-based environments.

6.3. Static and Dynamic Teleport Curriculum

The Teleport Bound (Theorem 6.3) offers valuable theoretical insights but faces practical limitations when the model dynamics are unknown. The dissimilarity penalization terms can be effectively upper-bounded, still, the overall quality of the solution is highly sensitive to the accuracy of the advantage term estimates. To address this challenge, a potential solution is to develop a sampling technique for estimating these quantities, similar to the one employed by Kakade and Langford [30]. Despite their sample inefficiency, several techniques exist for estimating the policy advantage. However, estimating the model advantage necessitates crossing into the realm of *model-based* approaches, which falls outside the scope of this manuscript. Consequently, we will explore alternative strategies.

To better understand the rationale behind Safe Policy Optimization and its objectives, it is crucial to examine the drawbacks of traditional Policy Gradient methods. As discussed in Section 3.4.5, distribution shift is one of the primary issues. Safe Learning aims to mitigate this phenomenon by introducing a dissimilarity penalization term and evaluating the worth of updates through an advantage measure.

Trust Region Methods (Section 3.4.5) pursue a similar objective but employ a slightly different approach. The idea is to constrain policy updates within a *trust region* to ensure stable learning, typically by limiting the KL divergence (Equation 3.36) between old and new policies. Proximal Policy Optimization (PPO, [60]) has gained prominence in the field due to its remarkable performance and practical applicability, despite—or perhaps because of—its relatively simple implementation compared to more complex algorithms such as Trust Region Policy Optimization (TRPO, [58]).

Building upon this reasoning, we now present a technique to achieve comparable goals in a context where both the policy and the model are subject to updates. Leveraging the results provided in Section 6.1.1, we can devise a looser lower bound on the state visit distribution shift associated with the model update that does not require any knowledge of the transition dynamics.

Corollary 6.7. *Let P_τ and $P_{\tau'}$ two probability transition matrix. The L_1-norm of the difference between the γ-discounted state distributions, when fixing the policy $\pi = \pi'$ can be upper-bounded as:*

$$\|d^\pi_{\mu,\tau'} - d^\pi_{\mu,\tau}\|_1 \leq \frac{2\gamma|\tau - \tau'|}{1-\gamma} = \epsilon_\tau, \tag{6.17}$$

where $\epsilon_\tau \in \mathbb{R}$.

Proof. Trivially from Corollary 6.3 we can observe that $D^{P,\xi}_{\mathbb{E}} \leq 2$ due to the *stochastic* property of P and ξ. Informally, the maximum difference between the two probability distributions occurs when both assign a probability of 1 to a different event. □

Then, we can state the following theorem.

Theorem 6.5. *Let $\mathcal{T}$ be a TMDP, π a fixed policy and $d^\pi_{\mu,\tau}$ the γ-discounted state visit distribution associated with τ and π. Given a threshold $\epsilon_\tau \in \mathbb{R}^+$ the following update rule:*

$$\tau^{(t+1)} \leftarrow \tau^{(t)} - \epsilon_\tau \frac{(1-\gamma)}{2\gamma}, \tag{6.18}$$

guarantees that:

(i) the L_1 norm of the difference between the γ-discounted state visit distribution of each single update of the teleport rate τ under the same policy π is upper bounded as $\|d^\pi_{\mu,\tau^{(t+1)}} - d^\pi_{\mu,\tau^{(t)}}\|_1 \leq \epsilon_\tau$.

(ii) The teleport rate τ converges to 0 in a fixed number of updates

$$n = \left\lceil \frac{2\gamma\tau}{\epsilon_\tau(1-\gamma)} \right\rceil. \tag{6.19}$$

Proof. Let us first prove (i). Considering Corollary 6.7 in a TMDP curriculum setting, the goal is to converge to $\tau = 0$ with monotonically decreasing updates, thus the absolute can be dropped since the condition is held by design. Then, by simply isolating τ' we get the proof.

Focusing now on (ii), considering the initial teleport rate $\tau^{(0)}$, at each time step the value is updated by a constant amount

$$d = \epsilon_\tau \frac{(1-\gamma)}{2\gamma},$$

thus, the value of $\tau^{(n)}$ after n updates becomes:

$$\tau^{(n)} = \tau^{(0)} - nd.$$

Determining the smallest n such that $\tau^{(n)} \leq 0$, and solving for n yields to:

$$n \geq \frac{\tau^{(0)}}{d} \geq \frac{2\gamma\tau^{(0)}}{\epsilon_\tau(1-\gamma)}.$$

Finally, since we are interested in an integer value, we take the ceiling of the right-hand side to get $n = \left\lceil \frac{2\gamma\tau}{\epsilon_\tau(1-\gamma)} \right\rceil$. □

This theorem provides the theoretical foundation for implementing a practical static curriculum, as outlined in Algorithm 6.2. Notably, *Static-Teleport Curriculum* (S-T) allows us to control the learning process without prior knowledge of the transition dynamics. By ensuring that the γ-discounted state visit distribution does not change more than a given threshold ϵ_τ, we can arbitrarily limit the distribution shift of each update. This constraint promotes a more stable learning process, following the clipping approach adopted in PPO.

As a direct consequence, we can provide also the following Corollary, useful to compute the threshold ϵ_τ that ensures a convergence in a fixed number of updates n.

Corollary 6.8. *Under the assumption that Theorem 6.5 is satisfied, we can compute ϵ_τ that allow to converge to $\tau = 0$ in n updates as:*

$$\epsilon_\tau \approx \frac{2\gamma\tau}{n(1-\gamma)}. \tag{6.20}$$

Proof. From Theorem 6.5, we know that the number of updates required to reach $\tau = 0$. Removing the ceiling and solving for ϵ_τ gives us the proof. □

These results focus on the model update only, allowing us to devise a linear scheduler for updating the teleport rate. To develop a dynamic technique that adapts to the agent's learning progress, we must consider the policy contribution too. The bound on the state

Algorithm 6.2 Static Teleport Curriculum (S-T)

Limiting Assumptions:
(i) Reset the agent to a custom state during each trajectory.

Require: Initial policy $\pi^{(0)}$, teleport rate $\tau^{(0)} \approx 0.5$, learning rate α, teleport update rate β, batch size N, epochs E and total time steps T.

1: Compute the number of updates: $n = T\beta$
2: Compute ϵ_τ to converge in n updates: $\epsilon_\tau = \frac{2\gamma\tau}{n(1-\gamma)}$ ▷ (Equation 6.20)
3: **for** t=0,1,..., T **do**
4: Collect N trajectories ▷ Assumption (i) required.
5: Learn $\pi^{(t+1)}$ processing the batch of N trajectories for E epochs
6: Update teleport rate: $\tau^{(t+1)} = \tau^{(t)} - \epsilon_\tau \frac{(1-\gamma)}{2\gamma}$ ▷ (Equation 6.18)
7: **end for**

visit distribution shift due to policy changes is provided by Corollary 6.2. Incorporating both model and policy updates, we obtain the next corollary.

Corollary 6.9. *Let (P_τ, π) and $(P_{\tau'}, \pi')$ two model-policy pairs. The L_1-norm of the difference between the γ-discounted state distributions can be upper-bounded as:*

$$\left\| d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau} \right\|_1 \leq \epsilon_\pi + \epsilon_\tau \leq \epsilon, \tag{6.21}$$

where $\left\| d^{\pi'}_{\mu,\tau} - d^{\pi}_{\mu,\tau} \right\|_1 \leq \epsilon_\pi$ and $\left\| d^{\pi}_{\mu,\tau'} - d^{\pi}_{\mu,\tau} \right\|_1 \leq \epsilon_\tau$.

Proof. The policy threshold term, ϵ_π, is simply obtained by renaming the right-hand side of Corollary 6.2, whereas the term ϵ_τ comes from Corollary 6.7. Then by simply applying these threshold values to Corollary 6.1 we get the proof. □

This corollary enables the development of a dynamic curriculum, as illustrated in Algorithm 6.3. The Dynamic Teleport Curriculum (D-T) adjusts the teleport rate in response to the agent's learning progress, balancing distribution shifts caused by both policy and model changes. Given a total allowed shift ϵ, the algorithm first computes ϵ_π to quantify the distribution shift associated with the policy update. Trust region methods are particularly well-suited for policy learning in this context, as they provide a way to control this step size. Subsequently, ϵ_τ is computed as the difference between the total allowed shift ϵ and the policy-induced shift ϵ_π, effectively representing the remaining *budget* for model updates. If a model update is allowed ($\epsilon_\tau > 0$), the actual step size is further constrained to prevent excessive changes.

Informally, when the policy undergoes significant changes, reflecting that the agent is acquiring new knowledge, the teleport rate update is reduced or potentially skipped to

maintain stability. Conversely, when the policy stabilizes, it suggests that the agent might be capable of handling increased task complexity, thus warranting a reduction in the teleport rate. It is worth mentioning that both Algorithm 6.2 and Algorithm 6.3 present a general structure that can be used to *augment* existing RL algorithms to handle *teleport-based* curricula.

6.3.1. Implementation Details

In Section 6.2, we introduced the concept of the *teleport sampling loop*. The simplest implementation of this idea involves truncating an episode when a teleport occurs and restarting the agent from a new state drawn from the teleport probability distribution ξ. While straightforward, this approach has several limitations. Notably, it precludes the possibility of multiple teleports within a single episode. In sparse-reward scenarios, this can lead to entirely unproductive trajectories, particularly during the early stages of learning when the value of states is yet unknown. Moreover, it renders the framework unsuitable for handling vector environments, which are increasingly appealing in modern reinforcement learning applications. A more sophisticated solution involves introducing a *teleport flag* to be used during the learning process. This flag serves to detect teleport events and, when triggered, masks the associated trajectory. This masking prevents the algorithm from learning from teleport tuples that are inconsistent in terms of actions, as teleports are action-independent. The experiments presented in the following sections are based on our custom implementation of this approach. However, it is worth noting that this teleport framework can be readily integrated into commonly used libraries. The key to such integration lies in carefully masking teleport tuples with the teleport flag.

Algorithm 6.3 Dynamic Teleport Curriculum (D-T)

Limiting Assumptions:

(i) Reset the agent to a custom state during each trajectory.

Require: Initial policy $\pi^{(0)}$, teleport rate $\tau^{(0)} \approx 0.5$, learning rate α, state distribution shift threshold ϵ, maximum model shift $\epsilon_{\tau_{\mathbf{max}}}$, batch size N and epochs E.

1: **for** t=0,1,... until convergence **do**
2: Collect N trajectories ▷ Assumption (i) required.
3: Learn $\pi^{(t+1)}$ processing the batch of N trajectories for E epochs
4: Compute $\epsilon_\pi = \frac{\gamma}{1-\gamma} D_\infty^{\pi',\pi}$ ▷ (Corollary 6.2)
5: Compute $\epsilon_\tau = \epsilon - \epsilon_\pi$
6: **if** $\epsilon_\tau > 0$ and $\tau^{(t)} > 0$ **then**
7: Update teleport rate: $\tau^{(t+1)} = \tau^{(t)} - \min(\epsilon_\tau, \epsilon_{\tau_{\mathbf{max}}}) \frac{(1-\gamma)}{2\gamma}$ ▷ (Equation 6.18)
8: **end if**
9: **end for**

7 | Experimental Results

In this chapter, we present empirical evidence that shows the effectiveness of curricula designed within the Teleporting Markov Decision Process (TMDP) framework. Our primary focus is on sparse reward domains, where efficient exploration poses significant challenges to reinforcement learning agents. We argue that TMDPs offer a promising alternative to traditional exploration strategies such as reward shaping and restart distributions, providing several advantages, in terms of simplicity and adaptability.

Our evaluation encompasses two widely used reinforcement learning algorithms: Proximal Policy Optimization (PPO) and Q-learning. We examine these algorithms in a tabular setting with discrete state and action spaces, comparing their vanilla implementations against versions enhanced with TMDP-based curricula[1].

In Section 7.1, we introduce the Frozen Lake environment and compare the learning curves of our TMDP-enhanced algorithms against a reward-shaping approach, highlighting the benefits of our method. Subsequently, in Section 7.2, we delve into the River Swim domain. This environment allows us to evaluate the performance of our TMDP-based algorithms in contrast to methods employing restart distributions.

7.1. Frozen Lake domain

In this section, our objective is to compare the teleport curriculum strategies presented in (Chapter 6) with reward shaping techniques. While reward shaping is known for its effectiveness in reinforcement learning, it presents design challenges. Modifying the reward signal can lead to unintended consequences and potentially trap the agent in local optima. In Section 7.1.1, we introduce the *Frozen Lake* environment. Then, Section 7.1.2 presents the reward-shaping technique we adopted and finally Section 7.1.3 shows our empirical results.

[1]To be as general as possible we adopted a fixed probability distribution ξ defined as a uniform distribution over the state space.

7.1.1. Environment description

In the context of RL, a *Frozen Lake* (Figure 7.1) is a classic grid world environment with sparse rewards where the agent must navigate from a start state toward a goal state. The grid is made up of different types of cells:

Figure 7.1: Example of a $10x10$ Frozen Lake environment. The agent lies in the initial position, whereas its goal is reaching the bottom-right state while avoiding holes, [71].

- **Start (S)**: the initial position of the agent.
- **Goal (G)**: the target state the agent aims to reach.
- **Frozen (F)**: safe cells the agent can walk on without any risk.
- **Hole (H)**: dangerous cells causing a premature end of the episode.

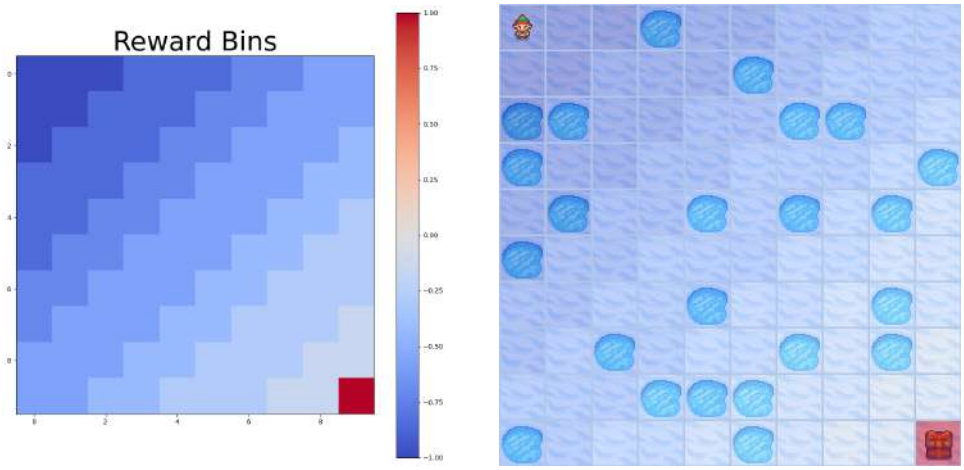
The goal of the agent is to find the optimal path, from the start (S) to the goal (G), while avoiding holes (H). The state space, $\mathcal{S}$, has a dimension of nrows $\times$ nrows, whereas the action space, $\mathcal{A}$, admits four actions: left (0), down (1), right (2) and up (3). Additionally, the environment's stochasticity can be controlled through the *is_slippery*[2] parameter and the percentage of holes in the state space can also be adjusted. A positive reward is received upon reaching the goal, with no feedback for other actions, making this an ideal environment to evaluate exploration-exploitation strategies.

7.1.2. Reward Shape

Our initial experiments revealed that teleport-based curriculum algorithms consistently reached the goal state. In contrast, their vanilla (no curricula) counterparts failed. To ensure a fair comparison, we introduced a reward-shaping technique, illustrated in Figure

[2]If this parameter is set, the agent moves in the intended direction with a probability of 1/3; otherwise, it moves in either perpendicular direction with an equal probability of 1/3 for each.

7.2. Specifically, we used the Manhattan distance to assign each state to a discrete bin, with rewards linearly interpolated between $[-1, 0]$ while using a distinct bin for the goal with a reward of 1. However, this introduced a common challenge to reward shaping: the negative rewards produced a suboptimal behaviour of jumping into the nearest hole to maximize the reward function.



(a) Visual representation of reward bins. (b) Reward bins on Frozen Lake.

Figure 7.2: Figure (a) provides a visual representation of 7 reward bins with a reward in $[-1, 0)$, plus a single element bin for the goal. The application of reward bins to a Frozen Lake environment is then shown in (b).

To address this, we introduced strong negative feedback for falling into a hole ($-|\mathcal{S}|$). However, the vanilla PPO implementation still struggled to reach the goal. This led us to refine the negative feedback by adding a shaping component that encourages moving away from the starting position. Formally, given the reward function $R_{\text{bins}}(\cdot)$ obtained by binning the state space according to Manhattan distance, the overall shaped reward function R_{shaped} can be expressed as:

$$R_{\text{shaped}}(s_{t+1}) = \begin{cases} r, & \text{if } s_{t+1} \text{ is safe (F)}, \\ +1, & \text{if } s_{t+1} \text{ is goal (G)}, \\ \min(r|\mathcal{S}| + d(s_0, s_{t+1}), cl) & \text{if } s_{t+1} \text{ is hole (H)}, \end{cases}$$

where $r = R_{\text{bins}}(s_{t+1})$, $d(x, y)$ is the Manhattan distance, and cl is the reward assigned to the closest bin to the goal. This ensures that the negative feedback for holes remains negative while providing an incentive to move away from the starting state and towards the goal.

Table 7.1: Additional information and hyperparameters used for Frozen Lake domain.

Algorithm	$\lvert\mathcal{S}\rvert$	Reward Bins	τ^0	γ	ϵ_{shift}	ϵ_{PPO}	λ_{GAE}	$\lambda_{\text{e.t.}}$	*is_slippery*
PPO	30×30	ALL[1]	0	0.99	-	0.28	0.95	-	False
D-TPPO	30×30	ALL	0.2	0.99	0.004	0.2	1	-	False
S-TPPO	30×30	ALL	0.2	0.99	-	0.2	1	-	False
Q	30×30	ALL	0	0.99	-	-	-	1	False
S-TQ	30×30	ALL	0.2	0.99	-	-	-	1	False
TMPI	30×30	0[2]	0.2	0.99	-	-	1	-	False

[1] With *ALL* we refer to 15, 10, 7, 3, 1 and 0 reward bins.
[2] Requires non-negative rewards.

Remark 7.1. *The definition of these bins shifts the reward range to* $[-1, 1]$ *making Algorithm 6.1, unsuitable for this setting. Despite its theoretical interest, its practical applicability is limited by the even stronger assumption of full knowledge of the transition model. Therefore, we will rely on reward shaping to evaluate the performance of no curricula algorithms against teleport-based curriculum approaches.*

7.1.3. Experiments

We conducted experiments on a Frozen Lake 30×30 environment, considering different reward bin settings. Each algorithm ran for 10 trials with random seeds and the presented results are averaged across all runs with 95% C.I. Hyperparameter specifications are detailed in Table 7.1.

In the 10-bin reward setting, all algorithms achieved optimal performance (Figure 7.3). However, curriculum-based algorithms converged faster than their vanilla counterparts, despite a slightly slower start. While non-curriculum PPO showed a similar initial

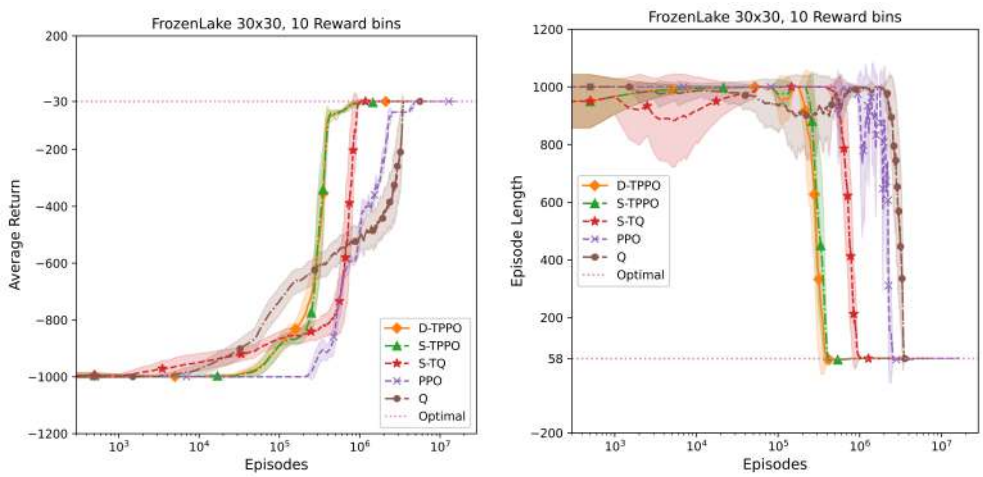


Figure 7.3: Average return and episode length, Frozen Lake 30×30 environment 10 reward bins.

behaviour, it struggled in the later stages, requiring additional time steps to converge. This trend was consistently observed across all tested environments, leading to sub-optimal convergence for vanilla PPO. Concerning Q-learning algorithms, the standard implementation shows almost linear learning curves while the curriculum version out-performed it, though both displayed slightly higher variance compared to PPO-based approaches. Figure 7.4 offers a different perspective on the training, showing how each algorithm updates the teleport rate τ (left) and the resulting effective discount factor γ_{eff} (right). This allows us to see how the curriculum impacts learning progress.

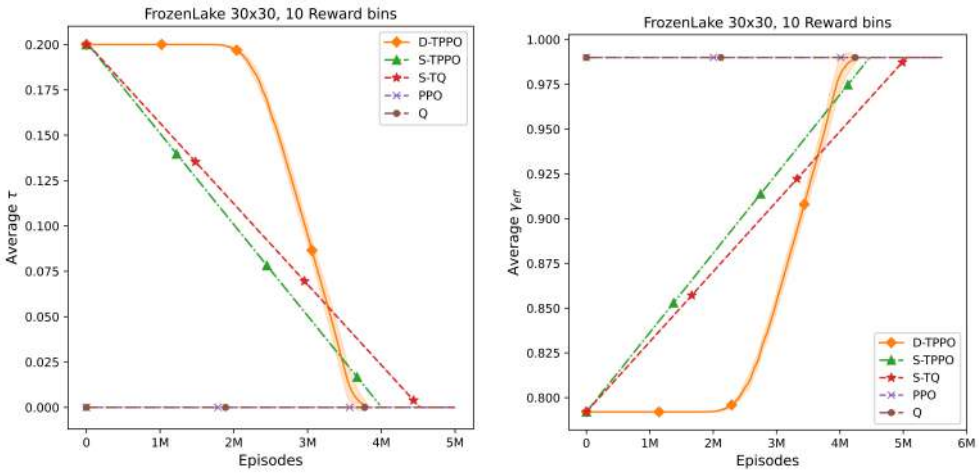


Figure 7.4: Average teleport rate τ and effective discount factor γ_{eff}, Frozen Lake 30×30 environment 10 reward bins.

The two static curricula (S-TPPO, S-TQ) produce very similar curves, with a slight offset due to differences in training duration. In contrast, the dynamic curriculum differs significantly. Comparing these results with Figure 7.3 we can notice that the D-TPPO

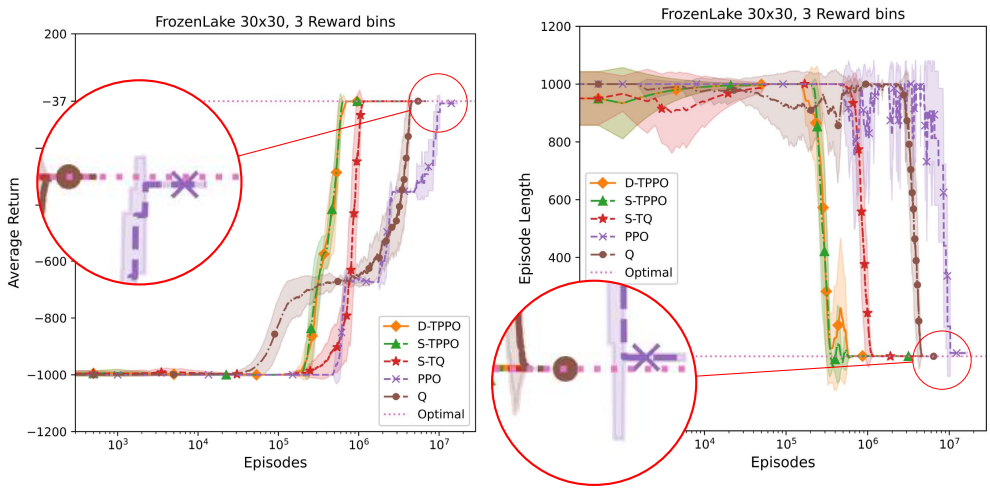


Figure 7.5: Average return and episode length, Frozen Lake 30×30 environment 3 reward bins.

algorithm first converges to the optimal solution, then adjusts the teleport rate, unlike the linear updates of the static curricula. Examining the impact of reward bin configurations, we observed that all algorithms, except PPO, learned the optimal policies. PPO exhibited two distinct behaviours, requiring additional actions. In contrast, its curriculum-based counterparts consistently achieved optimal performance and faster convergence. In the 3-bin setting (Figure 7.5), the vanilla PPO agent successfully reached the goal, requiring more actions than the optimal policy.

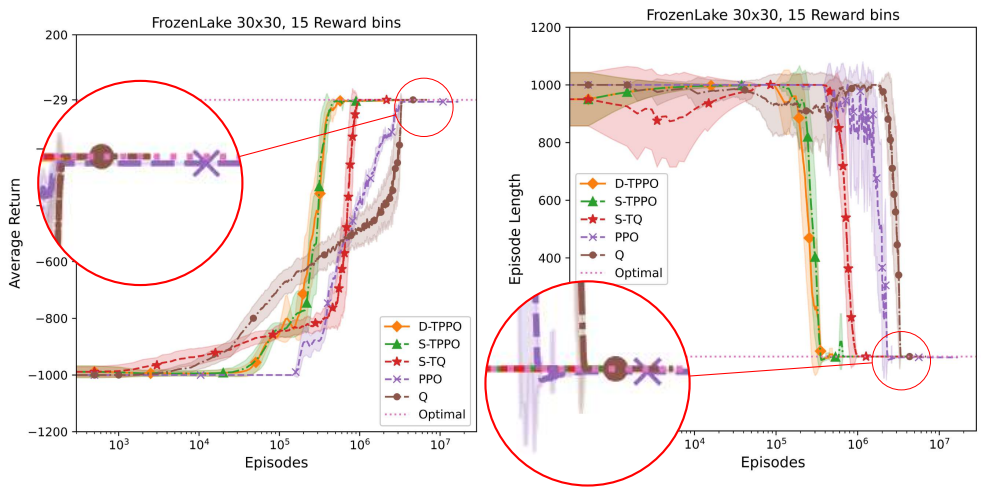


Figure 7.6: Average return and episode length, Frozen Lake 30×30 environment 15 reward bins.

Whereas, in the 15-bin environment (Figure 7.6), the agent took more actions and failed to reach the goal, falling into an adjacent hole. This outcome suggests an inclination of the algorithm toward exploitation, rather than exploitation. Figure 7.7 provides interesting details about this behaviour.

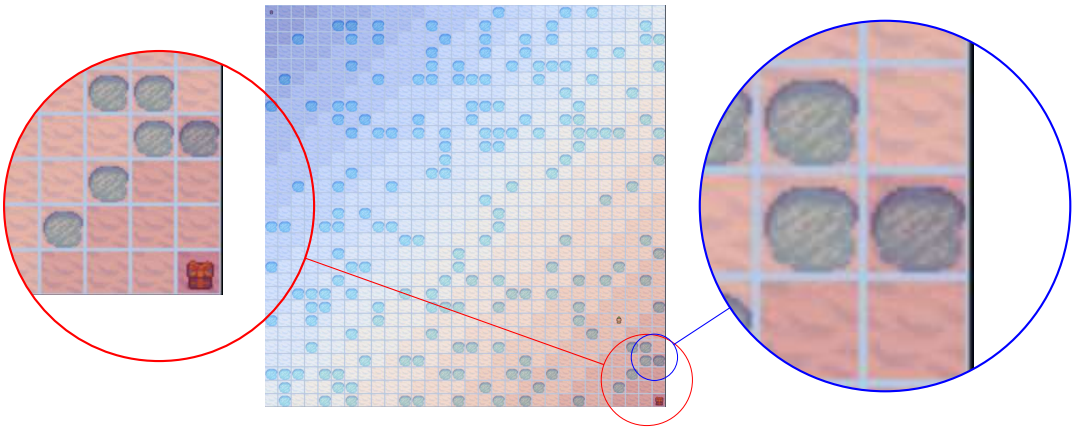


Figure 7.7: 15 Bins $30x30$ Frozen Lake environment. The vanilla PPO agent consistently ends up in the rightmost hole due to an unlucky combination of bins and holes.

Our final experiment yielded the most promising results. In the original domain, non-curriculum algorithms failed to reach the goal (Figure 7.8) hindered by the extensive state space and the limitations of standard exploration strategies. In contrast, all teleport-based curriculum approaches achieved optimal performance. Under the standard setting with sparse reward, we also evaluated the theoretical algorithm Teleport MPI (Algorithm 6.1), assuming full knowledge of transition dynamics. Consistent with previous experiments, both curriculum PPO implementations surpassed all other algorithms, with Dynamic Teleport PPO (D-TPPO) demonstrating superior performance to its static counterpart.

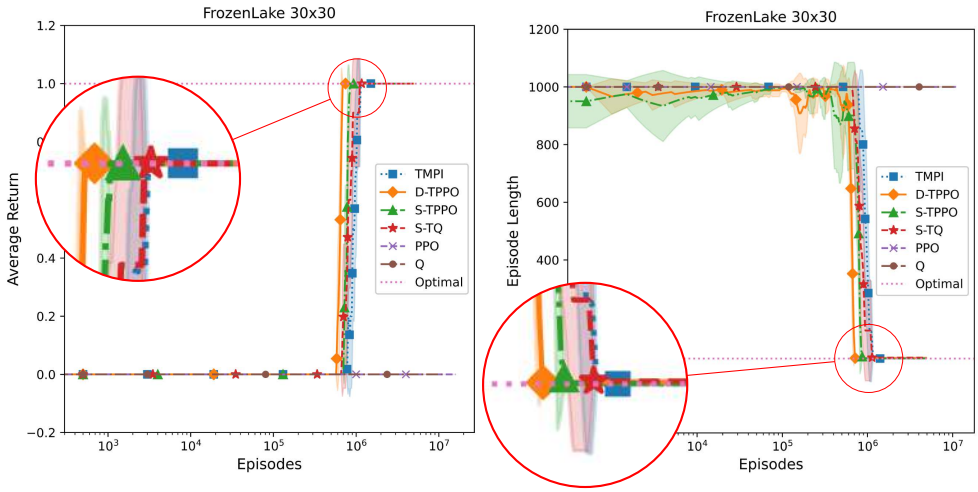


Figure 7.8: Average return and episode length, Frozen Lake 30×30 environment 0 bins.

Examining the updates of the teleport rate τ (Figure 7.9), we observed a slightly different trend. In this scenario, static curriculum algorithms were trained with the same number of time steps, resulting in identical schedules. Then, as observed in earlier experiments with D-TPPO, TMPI first acquired substantial knowledge and learned the optimal policy before adjusting τ. Conversely, D-TPPO began updates earlier, temporarily pausing when facing higher uncertainty, and resuming adjustments later. This adaptive behaviour allowed D-TPPO to be the last to converge to the original model but the fastest to achieve optimal performance.

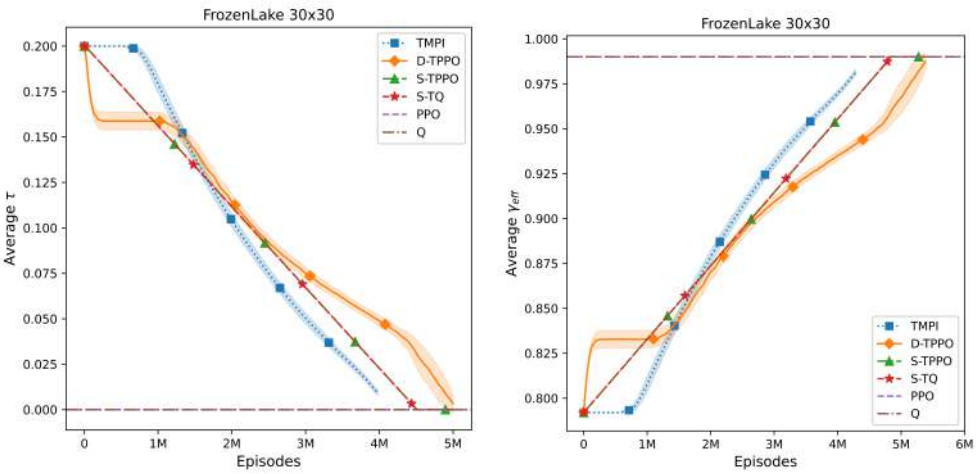


Figure 7.9: Average teleport rate τ and effective discount factor γ_{eff}, Frozen Lake 30×30 environment 0 bins.

7.2. River Swim domain

The focus of this Section is the evaluation of teleport-based curricula (Chapter 6) against restart distribution. While the teleport assumption (Section 5.1) is stronger than having access to restart distributions only at the beginning of a trajectory, TMDP offers a more versatile approach allowing control not only where the agent is teleported but also the frequency of these relocations. In Section 7.2.1, we describe the *River Swim* environment. Finally Section 7.2.2 shows our empirical results.

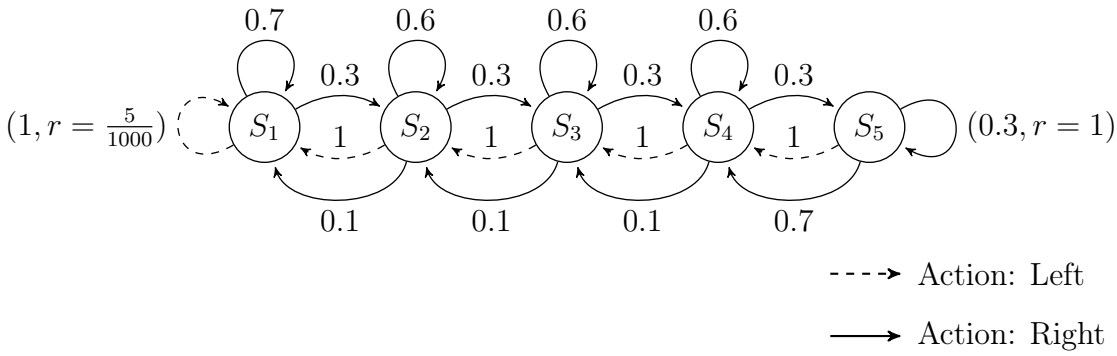


Figure 7.10: River Swim MDP with 5 states. Actions are non-deterministic with explicitly reported probabilities.

7.2.1. Environment description

In RL, the *River Swim* is a simple environment with sparse rewards where the agent must navigate against a strong current. The environment consists of a linear chain of

Table 7.2: Additional information and hyperparameters used for River Swim domain.

Algorithm	$\lvert\mathcal{S}\rvert$	τ^0	γ	ϵ_{shift}	ϵ_{PPO}	λ_{GAE}	$\lambda_{\text{e.t.}}$
PPO	70	0	0.99	-	0.2	0.3	-
D-TPPO	70	0.2	0.99	0.001	0.2	0.3	-
S-TPPO	70	0.2	0.99	-	0.2	0.3	-
Q	70	0	0.99	-	-	-	1
S-TQ	70	0.2	0.99	-	-	-	1
TMPI	70	0.2	0.99	-	-	1	-

states, representing different positions in the river (Figure 7.10). The objective is to swim upstream, starting from the leftmost state and reaching the rightmost state, despite the current pushing it backwards.

The state space, $\mathcal{S}$, comprises n discrete states, while the action space, $\mathcal{A}$, consists of two actions: swim left (0) and swim right (1). The environment's difficulty can be adjusted by modifying the transition probabilities, which simulate the river's current strength. When the agent attempts to swim right, the outcome is non-deterministic: it succeeds with a small probability p, remains in the same state with probability $1 - p - q$, and is pushed back left with probability q. Swimming left, instead, is deterministic.

A large reward is received upon reaching the rightmost state, and a small one is given for remaining in the leftmost state. No reward is provided for other actions, making this environment particularly suitable for evaluating exploration strategies in the face of misleading local optima.

7.2.2. Experiments

We conducted experiments on a River Swim environment with 70 states, comparing a uniform restart distribution against the original initial distribution, where the agent starts deterministically in the leftmost state. Each algorithm ran for 10 trials with random seeds and the presented results are averaged across all runs with 95% C.I. Hyperparameter specifications are detailed in Table 7.2.

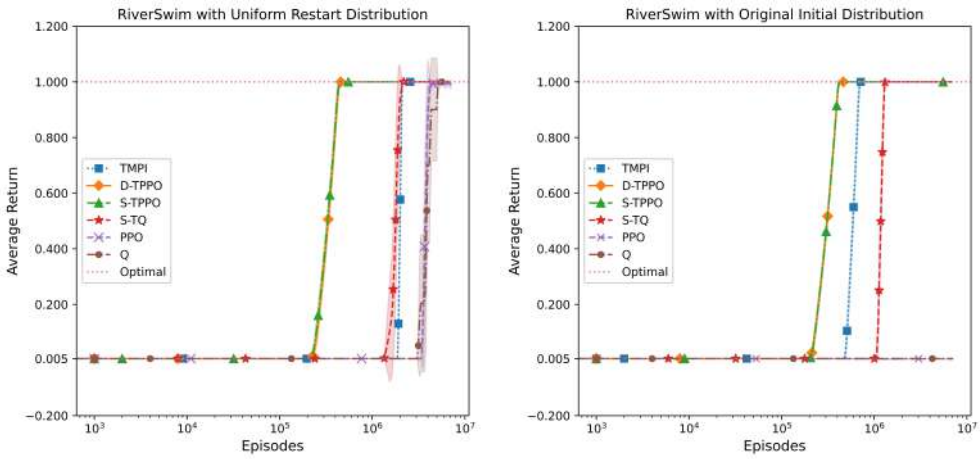


Figure 7.11: Average return in River swim environment environment. Uniform restart distribution (left) and original initial distribution (right).

Figure 7.11 presents the average return of *River Swim* experiments, revealing trends similar to those observed in the *Frozen Lake* environment. Once again, the TMDP-based PPO implementation demonstrates superior performance, outperforming all other algorithms in both distribution settings.

At the opposite end of the performance spectrum, we find the two vanilla implementations, PPO and Q-learning. Their behaviour contrasts between the two settings. In the uniform restart distribution scenario, these algorithms converge to the optimal policy, albeit slowly if compared to their curricula counterparts. However, when trained with the original initial probability, they fail to learn the optimal policy in favour of the closer local optimum, reflecting a lack of exploration.

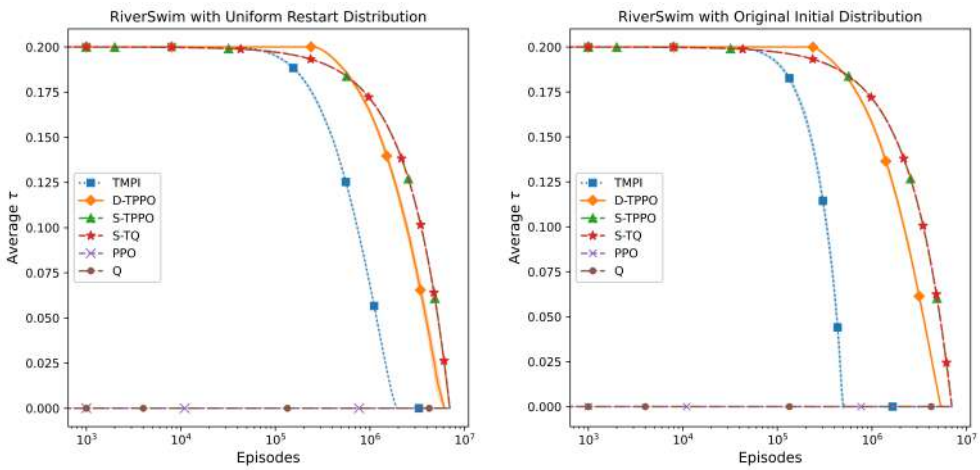


Figure 7.12: Average teleport rate in River swim environment. Uniform restart distribution (left) and original initial distribution (right).

S-TQ exhibits consistent performance across both settings, with a slight reduction in variance observed in the original setting. On the other hand, TMPI displays a notable improvement in the original setting compared to the uniform restart scenario. Both behaviours could be linked to excessive exploration when combining the two approaches, as their similar effects might lead to overlapping strategies. This phenomenon is more evident by analyzing the teleport rate curves presented in Figure 7.12. The faster convergence of TMPI to $\tau = 0$ in the original initial distribution setting suggests that it was able to better exploit gathered information without the need for extra exploration. D-TPPO, despite showing very similar behaviour, has a slightly higher variance in the uniform restart distribution, providing additional evidence to support the extra exploration that the two combined techniques lead to.

In conclusion, we have provided empirical evidence on the effectiveness of TMDP-based curricula in both the RiverSwim and Frozen Lake environments. Our results show that in tested scenarios the curriculum-based approaches consistently outperform their vanilla counterparts, underscoring the potential of TMDP frameworks to enhance curriculum learning, paving the way for future research.

8 | Conclusions

In this thesis, we have introduced and explored a novel approach for curriculum learning in reinforcement learning: the Teleport Markov Decision Process (TMDP). Extending the traditional Markov Decision Process formulation with the addition of a teleportation mechanism, the TMDP allows for flexible and scalable curriculum design. Addressing the open questions identified in Section 4.5, our research makes the following contributions to the field of curriculum learning in reinforcement learning:

1. **Theoretical results**: building upon the Safe Policy Iteration framework, in Chapters 5 and 6, we have established a solid mathematical foundation for TMDPs, deriving lower bounds for performance improvement guarantees. This led to the definition of the *Teleport Model Policy Iteration* (TMPI) algorithm. While TMPI has limited applicability to real-world scenarios, it offers valuable theoretical insights into the properties of the TMDP framework.

2. **Automated task creation and sequencing**: within the context of TMDP, the complex phases of task creation and sequencing are reduced to answering two key questions: **when** should we update the teleport rate, and **how** can we determine its new value? A new task is implicitly instantiated by selecting a teleport probability distribution ξ and the teleport rate τ. The *descending-order* relation over the teleport rates allows for efficient comparison of task complexity, with higher values of τ corresponding to simpler tasks and vice versa. Chapter 6 introduces two practically relevant algorithms, *Static Teleport* (S-T) and *Dynamic Teleport* (D-T), enabling the integration of teleport-based curricula into standard RL algorithms.

3. **Reusing curricula**: with a curriculum defined as an ordered list of teleport rates, reusing curricula across different tasks becomes a viable option, reducing the costs of curriculum design.

8.1. Limitation and Future Works

Our results provide empirical evidence of the effectiveness of teleport algorithms in the tested environments. However, the journey towards a fully automated curriculum learning solution, seamlessly integrated into RL data pipelines, is far from complete. It is worth noting the limitations of the TMDP framework. Its applicability is contingent upon the satisfaction of the Teleport Assumption (Section 5.1), which may not be feasible in all non-emulated environments.

This work has focused on custom, tabular implementations with discrete state and action spaces. Nonetheless, we are working to extend the teleport-based mechanism to widely used RL libraries. This integration, primarily involving modifications to the experience buffer and training loop to accommodate teleport tuples, would enable the evaluation of TMDP-based curricula in more complex scenarios, including continuous state-action spaces and function approximation.

Another promising direction to explore is the optimization of the teleport probability distribution ξ. In this work, we have assumed no prior knowledge of the environment, thus relying on a *uniform* and *fixed* probability distribution over the state space to be as general as possible. However, the TMDP framework holds significant potential for improvement by incorporating prior knowledge into this distribution. Furthermore, extending the theoretical results to allow changes in the teleport probability distribution ξ during learning could enhance performance. For instance, beginning with a uniform distribution and gradually biasing the teleportation towards unexplored regions of the state space could improve exploration, following approaches similar to those proposed by (CHER, [18]) or entropy-based techniques [25, 27].

Exploring model-based approaches to assess the impact of teleports on state space reconstruction and employing techniques such as Dynamic Programming (DP, [9]) or Approximate Dynamic Programming (ADP, [8]) is another interesting avenue for future research.

In conclusion, Curriculum Learning holds the potential to revolutionize traditional machine learning training and we believe that the Teleport MDP offers a promising direction for future research, offering a robust framework for developing more adaptive and efficient solutions.

Bibliography

[1] R. Amit, R. Meir, and K. Ciosek. Discount factor as a regularizer in reinforcement learning, 2020.

[2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

[3] M. Asada, S. Noda, S. Tawaratsumida, and K. Hosoda. Purposive behavior acquisition for a real robot by vision-based reinforcement learning. *Machine learning*, 23: 279–303, 1996.

[4] S. Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.

[5] J. Baxter and P. L. Bartlett. Infinite-horizon policy-gradient estimation. *Journal of Artificial Intelligence Research*, 15:319–350, 2001.

[6] A. Beck and M. Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Oper. Res. Lett.*, 31(3):167–175, 2003.

[7] R. Bellman. The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6):503–515, 1954.

[8] D. Bertsekas. *Reinforcement learning and optimal control*, volume 1. Athena Scientific, 2019.

[9] D. P. Bertsekas and J. N. Tsitsiklis. *Introduction to Probability.* Optimization and computation series. Athena Scientific, 2nd edition, 2008. ISBN 978-1886529236.

[10] D. Blackwell. Discrete Dynamic Programming. *The Annals of Mathematical Statistics*, 33(2):719 – 726, 1962. doi: 10.1214/aoms/1177704593. URL `https://doi.org/10.1214/aoms/1177704593`.

[11] L. Bottou. Online learning and stochastic approximations. *Online learning in neural networks*, 17(9):142, 1998.

[12] B. L. Bowerman. *Nonstationary Markov Decision Process and related topics in Nonstationary Markov Chains.* Iowa State University, 1974.

[13] K. Ciosek and S. Whiteson. Expected policy gradients for reinforcement learning. *Journal of Machine Learning Research*, 21, 2020.

[14] M. P. Deisenroth, G. Neumann, J. Peters, et al. A survey on policy search for robotics. *Foundations and Trends in Robotics*, 2(1-2):1–142, 2013.

[15] M. M. Deza and E. Deza. *Encyclopedia of distances*, pages 1–583. Springer, 2009.

[16] M. Dorigo, V. Maniezzo, , and A. Colorni. The ant system: An autocatalytic optimizing process. *Technical Report*, 1991.

[17] J. L. Elman. Learning and development in neural networks: The case of conjugate structure. In *Proceedings of the 1993 Connectionist Models Summer School*, 1993.

[18] M. Fang, T. Zhou, Y. Du, L. Han, and Z. Zhang. Curriculum-guided hindsight experience replay. *Advances in neural information processing systems*, 32, 2019.

[19] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on robot learning*, pages 482–495. PMLR, 2017.

[20] F. Foglino, C. C. Christakou, and M. Leonetti. An optimization framework for task sequencing in curriculum learning. In *2019 Joint IEEE 9th International Conference on Development and Learning and Epigenetic Robotics (ICDL-EpiRob)*, pages 207–214. IEEE, 2019.

[21] V. François-Lavet, R. Fonteneau, and D. Ernst. How to discount deep reinforcement learning: Towards new dynamic strategies, 2016.

[22] T. Fujii, Y. Arai, H. Asama, and I. Endo. Multilayered reinforcement learning for complicated collision avoidance problems. In *Proceedings. 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146)*, volume 3, pages 2186–2191. IEEE, 1998.

[23] F. Glover and M. Laguna. Tabu search. *Handbook of combinatorial optimization*, pages 2093–2229, 1998.

[24] D. E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning.* Addison-Wesley Longman Publishing Co, Boston, MA, USA, 1 edition, 1989. ISBN 0201157675.

[25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.

[26] M. Haviv and L. Van der Heyden. Perturbation bounds for the stationary probabilities of a finite markov chain. *Advances in Applied Probability*, 16(4):804–818, 1984.

[27] E. Hazan, S. Kakade, K. Singh, and A. Van Soest. Provably efficient maximum entropy exploration. *arXiv preprint arXiv:1812.02690*, 2019.

[28] N. Jiang, A. Kulesza, S. Singh, and R. L. Lewis. The dependence of effective planning horizon on model accuracy. In *Adaptive Agents and Multi-Agent Systems*, 2015. URL `https://api.semanticscholar.org/CorpusID:1477377`.

[29] S. M. Kakade. *On the sample complexity of reinforcement learning*. University of London, University College London (United Kingdom), 2003.

[30] S. M. Kakade and J. Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning*, volume 2, pages 267–274, 2002.

[31] M. J. Kearns and S. Singh. Bias-variance error bounds for temporal difference updates. In *Conference on Learning Theory*, pages 142–147, 2000.

[32] V. Konda and J. Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, volume 12, 1999.

[33] Y. Mansour and S. Singh. On the complexity of policy iteration. In *Proceedings of the Fifteenth conference on Uncertainty in artificial intelligence*, pages 401–408, 1999.

[34] A. M. Metelli, M. Mutti, and M. Restelli. Configurable markov decision processes. In *International Conference on Machine Learning*, pages 3491–3500. PMLR, 2018.

[35] T. Mitchell. *Machine Learning*. McGraw-Hill International Editions. McGraw-Hill, 1997. ISBN 9780071154673. URL `https://books.google.it/books?id=EoYBngEACAAJ`.

[36] Mnih, V., Kavukcuoglu, K., Silver, and D. et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, Feb 2015. ISSN 1476-4687. doi: 10.1038/nature14236. URL `https://doi.org/10.1038/nature14236`.

[37] S. Narvekar, J. Sinapov, M. Leonetti, and P. Stone. Source task creation for curricu-

lum learning. In *Proceedings of the 2016 International Conference on Autonomous Agents 'I&' Multiagent Systems*, AAMAS '16, page 566–574, Richland, SC, 2016. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450342391.

[38] S. Narvekar, J. Sinapov, and P. Stone. Autonomous task sequencing for customized curriculum design in reinforcement learning. In *IJCAI*, pages 2536–2542, 2017.

[39] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020. URL `http://jmlr.org/papers/v21/20-212.html`.

[40] G. Neu, A. Jonsson, and V. Gómez. A unified view of entropy regularized markov decision processes. *CoRR*, abs/1705.07798, 2017. `https://arxiv.org/abs/1705.07798`.

[41] A. Ng, D. Harada, and S. J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, 1999. URL `https://api.semanticscholar.org/CorpusID:5730166`.

[42] J. R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1997.

[43] P. S. Ow and T. E. Morton. Filtered beam search in scheduling. *The International Journal Of Production Research*, 26:35–62, 1988.

[44] M. Papini, M. Pirotta, and M. Restelli. Adaptive batch size for safe policy gradients. In *NeurIPS*, page 3591–3600, 2017.

[45] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta. Robust adversarial reinforcement learning. In *International Conference on Machine Learning*, pages 2817–2826. PMLR, 2017.

[46] M. Pirotta, M. Restelli, A. Pecorino, and D. Calandriello. Safe policy iteration. In *International conference on machine learning*, pages 307–315. PMLR, 2013.

[47] M. Pirotta, M. Restelli, and L. Bascetta. Adaptive step-size for policy gradient methods. In *NeurIPS 26*, pages 1394–1402. Curran Associates, Inc., 2013a.

[48] M. Pirotta, M. Restelli, and L. Bascetta. Policy gradient in lipschitz markov decision processes. *Machine Learning*, 100((2-3)):255–283, 2015.

[49] D. Prokhorov and D. Wunsch. Adaptive critic designs. *IEEE Transactions on Neural Networks*, 8(5):997–1007, 1997. doi: 10.1109/72.623201.

[50] M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2005.

[51] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.

[52] C. P. Robert and G. Casella. *Monte Carlo Statistical Methods*, volume 2. Springer, 1999.

[53] G. A. Rummery and M. Niranjan. On-line q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, University of Cambridge, Department of Engineering, 1994.

[54] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[55] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *International conference on machine learning*, pages 1312–1320. PMLR, 2015.

[56] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In Y. Bengio and Y. LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.

[57] B. Scherrer. Improved and generalized upper bounds on the complexity of policy iteration. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[58] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In F. Bach and D. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, Lille, France, 07–09 Jul 2015. PMLR. URL `https://proceedings.mlr.press/v37/schulman15.html`.

[59] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[60] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.

[61] J. Schulman, X. Chen, and P. Abbeel. Equivalence between policy gradients and soft q-learning, 2018.

[62] L. Shani, Y. Efroni, and S. Mannor. Adaptive trust region policy optimization: Global convergence and faster rates for regularized mdps. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 5668–5675. AAAI Press, 2020.

[63] F. L. D. Silva and A. H. R. Costa. Object-oriented curriculum generation for reinforcement learning. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '18, page 1026–1034, Richland, SC, 2018. International Foundation for Autonomous Agents and Multiagent Systems.

[64] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning*, 38(3): 287–308, 2000.

[65] S. P. Singh, T. S. Jaakkola, and M. I. Jordan. Learning without state-estimation in partially observable markovian decision processes. In *ICML*, pages 284–292. Morgan Kaufmann, 1994.

[66] P. Soviany, R. T. Ionescu, P. Rota, and N. Sebe. Curriculum learning: A survey, 2022.

[67] S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, and R. Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL `https://openreview.net/forum?id=SkT5Yg-RZ`.

[68] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, 2 edition, 2018. ISBN 9780262039246.

[69] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999a.

[70] M. Svetlik, M. Leonetti, J. Sinapov, R. Shah, N. Walker, and P. Stone. Automatic curriculum graph generation for reinforcement learning agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, pages 1093–1099, 2017.

[71] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. T. J. Shen, and O. G. Younis. Gymnasium, Mar. 2023. URL `https://zenodo.org/record/8127025`.

[72] C. Watkins and K. C. U. of Cambridge). *Learning from Delayed Rewards*. Cambridge University, 1989. URL `https://books.google.it/books?id=6MBgNwAACAAJ`.

[73] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.

[74] P. V. Yoshua Bengio, Jean-Pierre Laurière. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 115–122, 2009.

[75] M. Zimmer, Y. Boniface, and A. Dutech. Developmental reinforcement learning through sensorimotor space enlargement. In *2018 Joint IEEE 8th International Conference on Development and Learning and Epigenetic Robotics (ICDL-EpiRob)*, pages 33–38. IEEE, 2018.

A | Proofs and Derivations

Equation 5.24. *The expected relative policy advantage function can be expressed as:*

$$\begin{aligned}\mathbb{A}^{\pi',\tau}_{\pi,\mu} &= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau,\pi'}_{\tau,\pi}(s) \qquad (A.1)\\ &= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \Big(\pi'(a|s) - \pi(a|s)\Big) Q^{\pi}_{\tau}(s,a),\end{aligned}$$

Proof. Expanding therelative policy advantage function (Equation 5.22):

$$\begin{aligned}A^{\tau,\pi'}_{\tau,\pi}(s) &= \sum_{a\in\mathcal{A}} \pi'(a|s) A^{\pi}_{\tau}(s,a)\\ &= \sum_{a\in\mathcal{A}} \pi'(a|s) Q^{\pi}_{\tau}(s,a) - V^{\pi}_{\tau}(s) \qquad (A.2)\\ &= \sum_{a\in\mathcal{A}} \pi'(a|s) Q^{\pi}_{\tau}(s,a) - \sum_{a\in\mathcal{A}} \pi(a|s) Q^{\pi}_{\tau}(s,a) \qquad (A.3)\\ &= \sum_{a\in\mathcal{A}} \Big(\pi'(a|s) - \pi(a|s)\Big) Q^{\pi}_{\tau}(s,a), \qquad (A.4)\end{aligned}$$

where A.2 derives by the application of 5.22 and noting that $\sum_{a\in\mathcal{A}} \pi'(a|s) = 1$, A.3 follows from A.2 and 5.15. Then we obtain the proof by replacing A.4 into A.1. □

Equation 5.25. *The expected relative model advantage function is given by:*

$$\begin{aligned}\mathbb{A}^{\tau',\pi}_{\tau,\mu} &= \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} \delta^{\pi}_{\mu,\tau}(s,a) A^{\tau',\pi}_{\tau,\pi}(s,a) \qquad (A.5)\\ &= (\tau - \tau') \mathbb{A}^{\xi,\pi}_{P,\tau,\mu},\end{aligned}$$

where $\mathbb{A}^{\xi,\pi}_{P,\tau,\mu} = (\tau - \tau') \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} \delta^{\pi}_{\mu,\tau}(s,a) \sum_{s'\in\mathcal{S}} \Big(P(s'|s,a) - \xi(s')\Big) U^{\pi}_{\tau}(s,a,s')$.

Proof. Expanding the relative model advantage function (Equation 5.23):

$$
\begin{aligned}
A^{\tau',\pi}_{\tau,\pi}(s,a) &= \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a)A^{\pi,\tau}(s,a,s') && \text{(A.6)}\\
&= \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a)U^{\pi}_{\tau}(s,a,s') - Q^{\pi}_{\tau}(s,a) && \text{(A.7)}\\
&= \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a)U^{\pi}_{\tau}(s,a,s') - \sum_{s'\in\mathcal{S}} P_{\tau}(s'|s,a)U^{\pi}_{\tau}(s,a,s') && \text{(A.8)}\\
&= \sum_{s'\in\mathcal{S}} \Big(P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\Big)U^{\pi}_{\tau}(s,a,s') && \text{(A.9)}\\
&= (\tau-\tau')\sum_{s'\in\mathcal{S}} \Big(P(s'|s,a) - \xi(s')\Big)U^{\pi}_{\tau}(s,a,s'), && \text{(A.10)}
\end{aligned}
$$

where A.7 comes from A.6 by applying 5.21 and noting $\sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a) = 1$, A.8 is derived by A.7 and 5.16, A.9 follows lemma 5.1. Finally, we obtain the proof by replacing A.10 in A.5. □

Lemma 5.3. *The following equality relates the coupled relative advantage function and the relative advantage functions of the policy and the model:*

$$
A^{\tau',\pi'}_{\tau,\pi}(s) = A^{\tau,\pi'}_{\tau,\pi}(s) + \sum_{a\in\mathcal{A}} \pi'(a|s)A^{\tau',\pi}_{\tau,\pi}(s,a)
$$

Proof.

$$
\begin{aligned}
A^{\tau',\pi'}_{\tau,\pi}(s) &= \sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a)U^{\pi}_{\tau}(s,a,s') - V^{\pi}_{\tau}(s)\\
&\quad \pm \sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} P_{\tau}(s'|s,a)U^{\pi}_{\tau}(s,a,s')\\
&= \sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} P_{\tau}(s'|s,a)U^{\pi}_{\tau}(s,a,s') - V^{\pi}_{\tau}(s) +\\
&\quad + \sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} \Big(P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\Big)U^{\pi}_{\tau}(s,a,s') && \text{(A.11)}\\
&= \sum_{a\in\mathcal{A}} \pi'(a|s)Q^{\pi}_{\tau}(s,a) - V^{\pi}_{\tau}(s) +\\
&\quad + \sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} \Big(P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\Big)U^{\pi}_{\tau}(s,a,s') && \text{(A.12)}\\
&= A^{\tau,\pi'}_{\tau,\pi}(s) + \sum_{a\in\mathcal{A}} \pi'(a|s)A^{\tau',\pi}_{\tau,\pi}(s,a), && \text{(A.13)}
\end{aligned}
$$

where A.12 comes from A.11 and 5.16, and for A.13 we applied 5.20 and 5.21. □

Proposition 6.1. *Let (P_τ, π) and $(P_{\tau'}, \pi')$ two model-policy pairs, where P_τ and $P_{\tau'}$ are the probability transition matrix associated with parameters τ and τ'. The L_1-norm of the difference between the γ-discounted state distributions can be upper-bounded as:*

$$\left\|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\right\|_1 \leq \frac{\gamma}{1-\gamma} D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}}, \tag{A.14}$$

where $D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \left\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\right\|_1$.

Proof. We start applying the recursive formulation of the state occupancy measure Equation 5.18:

$$\begin{aligned}
d^{\pi'}_{\mu,\tau'}(s) - d^{\pi}_{\mu,\tau}(s) &= \gamma \sum_{s'\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s') P^{\pi'}_{\tau'}(s|s') - \gamma \sum_{s'\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s') P^{\pi}_{\tau}(s|s') \\
&= \gamma \sum_{s'\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s') P^{\pi'}_{\tau'}(s|s') - \gamma \sum_{s'\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s') P^{\pi}_{\tau}(s|s') \\
&\qquad \pm \gamma \sum_{s'\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s') P^{\pi'}_{\tau'}(s|s') \\
&= \gamma \sum_{s'\in\mathcal{S}} \Big(d^{\pi}_{\mu,\tau'}(s') - d^{\pi}_{\mu,\tau}(s')\Big) P^{\pi}_{\tau'}(s|s') + \\
&\qquad + \gamma \sum_{s'\in\mathcal{S}} \Big(P^{\pi'}_{\tau'}(s|s') - P^{\pi}_{\tau}(s|s')\Big) d^{\pi}_{\mu,\tau}(s').
\end{aligned} \tag{A.15}$$

Applying the L_1 norm we can state the following:

$$\begin{aligned}
\left\|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\right\|_1 &\leq \gamma \sum_{s\in\mathcal{S}} \Big| \sum_{s'\in\mathcal{S}} \Big(d^{\pi'}_{\mu,\tau'}(s) - d^{\pi}_{\mu,\tau}(s)\Big) P^{\pi'}_{\tau'}(s|s') \Big| + \\
&\qquad + \gamma \sum_{s\in\mathcal{S}} \Big| \sum_{s'\in\mathcal{S}} \Big(P^{\pi}_{\tau'}(s|s') - P^{\pi}_{\tau}(s|s')\Big) d^{\pi}_{\mu,\tau}(s') \Big| && \text{(A.16)} \\
&\leq \gamma \sum_{s'\in\mathcal{S}} \Big| d^{\pi'}_{\mu,\tau'}(s') - d^{\pi}_{\mu,\tau}(s') \Big| \sum_{s\in\mathcal{S}} P^{\pi'}_{\tau'}(s|s') + \\
&\qquad + \gamma \sum_{s'\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s') \sum_{s\in\mathcal{S}} \Big| P^{\pi'}_{\tau'}(s|s') - P^{\pi}_{\tau}(s|s') \Big| && \text{(A.17)} \\
&\leq \gamma \left\|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\right\|_1 + \gamma \mathop{\mathbb{E}}_{s\sim d^{\pi}_{\mu,\tau}} \left\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\right\|_1 && \text{(A.18)} \\
&\leq \frac{\gamma}{1-\gamma} \mathop{\mathbb{E}}_{s\sim d^{\pi}_{\mu,\tau}} \left\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\right\|_1 = \frac{\gamma}{1-\gamma} D_{\mathbb{E}}^{P^{\pi'}_{\tau'}, P^{\pi}_{\tau}}, && \text{(A.19)}
\end{aligned}$$

where A.16 we used the subadditivity of norm $\|x+y\| \leq \|x\| + \|y\|$ and A.18 derives from A.17 by observing that $\sum_{s'\in\mathcal{S}} P^{\pi'}_{\tau'}(s'|s) = 1$. □

Corollary 6.1. *Let (P_τ, π) and $(P_{\tau'}, \pi')$ two model-policy pairs. The L_1-norm of the difference between the γ-discounted state distributions can be upper-bounded as:*

$$\left\|d^{\pi'}_{\mu,\tau'} - d^{\pi}_{\mu,\tau}\right\|_1 \leq \frac{\gamma}{1-\gamma}(D^{\pi',\pi}_{\mathbb{E}} + |\tau - \tau'| D^{P,\xi}_{\mathbb{E}}),$$

where $D^{\pi',\pi}_{\mathbb{E}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}}\left\|\pi'(\cdot|s) - \pi(\cdot|s)\right\|_1$ *and* $D^{P,\xi}_{\mathbb{E}} := \mathbb{E}_{(s,a)\sim\delta^{\pi}_{\mu,\tau}}\left\|P(\cdot|s,a) - \xi(\cdot)\right\|_1$.

Proof. We prove this corollary by decomposing the expression $\mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}}\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\|_1$. Let us focus first on the L_1-norm.

$$\begin{aligned}
\left\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\right\|_1 &= \sum_{s'\in\mathcal{S}}\left|P^{\pi'}_{\tau'}(s'|s) - P^{\pi}_{\tau}(s'|s)\right| \\
&= \sum_{s'\in\mathcal{S}}\Big|\sum_{a\in\mathcal{A}} P_{\tau'}(s'|s,a)\pi'(a|s) - P_{\tau}(s'|s,a)\pi(a|s)\Big| \\
&= \sum_{s'\in\mathcal{S}}\Big|\sum_{a\in\mathcal{A}} P_{\tau'}(s'|s,a)\pi'(a|s) - P_{\tau}(s'|s,a)\pi(a|s) \pm P_{\tau'}(s'|s,a)\pi(a|s)\Big| \\
&= \sum_{s'\in\mathcal{S}}\Big|\sum_{a\in\mathcal{A}} \big(\pi'(a|s) - \pi(a|s)\big)P_{\tau'}(s'|s,a) + \\
&\qquad + \big(P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\big)\pi(a|s)\Big| \\
&\leq \sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\big|\pi'(a|s) - \pi(a|s)\big|P_{\tau'}(s'|s,a) + \\
&\qquad + \sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\big|P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\big|\pi(a|s)
\end{aligned}$$

We take now the expectation w.r.t. $d^{\pi}_{\mu,\tau}$ exploiting the monotonicity property:

$$\begin{aligned}
\mathop{\mathbb{E}}_{s\sim d^{\pi}_{\mu,\tau}}\|P^{\pi'}_{\tau'}(\cdot|s) - P^{\pi}_{\tau}(\cdot|s)\|_1 &\leq \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s)\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\big|\pi'(a|s) - \pi(a|s)\big|P_{\tau'}(s'|s,a) + \\
&\qquad + \sum_{s\in\mathcal{S}}\sum_{a\in\mathcal{A}} d^{\pi}_{\mu,\tau}(s)\pi(a|s)\sum_{s'\in\mathcal{S}}\big|P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\big| \qquad \text{(A.20)} \\
&\leq \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s)\sum_{a\in\mathcal{A}}\big|\pi'(a|s) - \pi(a|s)\big| + \\
&\qquad + |\tau - \tau'|\sum_{s\in\mathcal{S}}\delta^{\pi}_{\mu,\tau}(s,a)\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\big|P(s'|s,a) - \xi(s')\big| \qquad \text{(A.21)} \\
&= \mathop{\mathbb{E}}_{s\sim d^{\pi}_{\mu,\tau}}\|\pi'(\cdot|s) - \pi(\cdot|s)\|_1 + |\tau - \tau'|\mathop{\mathbb{E}}_{(s,a)\sim\delta^{\pi}_{\mu,\tau}}\|P(\cdot|s,a) - \xi(\cdot)\|_1 \\
&= D^{\pi',\pi}_{\mathbb{E}} + |\tau - \tau'| D^{P_{\tau'},P_{\tau}}_{\mathbb{E}}
\end{aligned}$$

where A.21 follows from A.20 observing that $\delta^{\pi}_{\mu,\tau}(s,a) = d^{\pi}_{\mu,\tau}(s)\pi(a|s)$ and applying Lemma 5.1. □

Theorem 6.1 *The performance improvement of the model-policy pair (P_τ, π) over $(P_{\tau'}, \pi')$ is given by:*

$$J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau} = \frac{1}{1-\gamma} \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) A^{\tau',\pi'}_{\tau,\pi}(s).$$

Proof. Let us start from $J^{\pi'}_{\mu,\tau'}$:

$$\begin{aligned}
(1-\gamma)J^{\pi'}_{\mu,\tau'} &= \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) \sum_{a\in\mathcal{A}} \pi'(a|s) R_{\tau'}(s,a) \\
&= \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) \sum_{a\in\mathcal{A}} \pi'(a|s) R_{\tau'}(s,a) \pm \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) V^{\pi}_{\tau}(s) && \text{(A.22)} \\
&= \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) \sum_{a\in\mathcal{A}} \pi'(a|s) R_{\tau'}(s,a) + \\
&\quad + \sum_{s'\in\mathcal{S}} \Big((1-\gamma)\mu(s') + \gamma \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} d^{\pi'}_{\mu,\tau'}(s) \pi'(a|s) P_{\tau'}(s'|s,a) \Big) V^{\pi}_{\tau}(s') + \\
&\quad - \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) V^{\pi}_{\tau}(s) && \text{(A.23)} \\
&= \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) \Big(\sum_{a\in\mathcal{A}} \pi'(a|s) \sum_{s'\in\mathcal{S}} P_{\tau'}(s'|s,a) \big(R(s,a,s') + \gamma V^{\pi}_{\tau}(s') \big) + \\
&\quad - V^{\pi}_{\tau}(s) \Big) + (1-\gamma) \sum_{s'\in\mathcal{S}} \mu(s') V^{\pi}_{\tau}(s') && \text{(A.24)} \\
&= (1-\gamma) J^{\pi}_{\mu,\tau} + \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) \sum_{s'\in\mathcal{S}} \sum_{a\in\mathcal{A}} \pi'(a|s) P_{\tau'}(s'|s,a) \tilde{A}^{\pi}_{\tau}(s,a,s') && \text{(A.25)} \\
&= (1-\gamma) J^{\pi}_{\mu,\tau} + \sum_{s\in\mathcal{S}} d^{\pi'}_{\mu,\tau'}(s) A^{\tau',\pi'}_{\tau,\pi}(s), && \text{(A.26)}
\end{aligned}$$

where we have applied the recursive formulation of $d^{\pi'}_{\mu,\tau'}$ 5.18 to get A.23 from A.22. Then A.25 comes from A.24 by observing that $\sum_{s'\in\mathcal{S}} \mu(s') V^{\pi}_{\tau}(s') = J_{\mu,\tau}$ and applying the definitions $U^{\pi}_{\tau}(s,a,s') = R(s,a,s') + \gamma V^{\pi}_{\tau}(s')$ and $\tilde{A}^{\pi}_{\tau}(s,a,s') = U^{\pi}_{\tau}(s,a,s') - V^{\pi}_{\tau}(s)$. □

Lemma 6.1 *The following bound relates the joint relative advantage function and the relative advantage functions:*

$$\Big| \mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} - \big(\mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi} \big) \Big| \leq \gamma |\tau - \tau'| D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \frac{\Delta U^{\pi}_{\tau}}{2},$$

where $D^{\pi',\pi}_{\mathbb{E}} := \mathbb{E}_{s\sim d^{\pi}_{\mu,\tau}} \big\| \pi'(\cdot|s) - \pi(\cdot|s) \big\|_1$, $D^{P,\xi}_{\infty} = \sup_{s\in\mathcal{S},a\in\mathcal{A}} \| P(\cdot|s,a) - \xi(\cdot) \|_1$ *and* $\Delta U^{\pi}_{\tau} = \sup_{s,s',\overline{s},\overline{s}'\in\mathcal{S},a,a'\in\mathcal{A}} \big| U^{\pi}_{\tau}(s',a',\overline{s}') - U^{\pi}_{\tau}(s,a,\overline{s}) \big|$.

Proof. Expanding the expected coupled relative advantage (Equation 5.28) we get:

$$\begin{aligned}
\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} &= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) A^{\tau',\pi'}_{\tau,\pi}(s) \\
&= \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s)\big(A^{\tau,\pi'}_{\tau,\pi}(s) + \sum_{a\in\mathcal{A}} \pi'(a|s) A^{\tau',\pi}_{\tau,\pi}(s,a) \pm \sum_{a\in\mathcal{A}} \pi(a|s) A^{\tau',\pi}_{\tau,\pi}(s,a)\big) \\
&= \mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi} + \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \big(\pi'(a|s) - \pi(a|s)\big) A^{\tau',\pi}_{\tau,\pi}(s,a) \qquad \text{(A.27)}
\end{aligned}$$

From equation A.27 follows the inequalities:

$$\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} \geq \mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi} - \Big|\sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \big(\pi'(a|s) - \pi(a|s)\big) A^{\tau',\pi}_{\tau,\pi}(s,a)\Big|,$$

$$\mathbb{A}^{\tau',\pi'}_{\tau,\pi,\mu} \leq \mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi} + \Big|\sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \big(\pi'(a|s) - \pi(a|s)\big) A^{\tau',\pi}_{\tau,\pi}(s,a)\Big|,$$

then, bounding the right-hand side:

$$\begin{aligned}
\Big|\mathbb{A}^{\tau',\pi'}_{\tau,\pi} - \big(\mathbb{A}^{\tau,\pi'}_{\tau,\pi} + \mathbb{A}^{\tau',\pi}_{\tau,\pi}\big)\Big| &\leq \sum_{s\in\mathcal{S}} \sum_{a\in\mathcal{A}} d^{\pi}_{\mu,\tau}(s) \Big|\big(\pi'(a|s) - \pi(a|s)\big) A^{\tau',\pi}_{\tau,\pi}(s,a)\Big| \\
&\leq \sum_{s\in\mathcal{S}} d^{\pi}_{\mu,\tau}(s) \sum_{a\in\mathcal{A}} \Big|\pi'(a|s) - \pi(a|s)\Big| \frac{\Delta A^{\tau',\pi}_{\tau,\pi}}{2} \\
&\leq \mathop{\mathbb{E}}_{s\sim d^{\pi}_{\mu,\tau}} \|\pi'(a|s) - \pi(a|s)\|_1 \frac{\Delta A^{\tau',\pi}_{\tau,\pi}}{2} \\
&\leq D^{\pi',\pi}_{\mathbb{E}} \frac{\Delta A^{\tau',\pi}_{\tau,\pi}}{2}.
\end{aligned}$$

We conclude bounding $\Delta A^{\tau',\pi}_{\tau,\pi}$ as follows:

$$\begin{aligned}
\frac{\Delta A^{\tau',\pi}_{\tau,\pi}}{2} &\leq \|\Delta A^{\tau',\pi}_{\tau,\pi}\|_\infty \\
&\leq \sup_{s\in\mathcal{S},a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} \Big|\big(P_{\tau'}(s'|s,a) - P_{\tau}(s'|s,a)\big) U^{\pi}_{\tau}(s,a,s')\Big| \\
&\leq \gamma|\tau - \tau'| \sup_{s\in\mathcal{S},a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} \Big|\big(P(s'|s,a) - \xi(s)\big) U^{\pi}_{\tau}(s,a,s')\Big| \\
&\leq \gamma|\tau - \tau'| \sup_{s\in\mathcal{S},a\in\mathcal{A}} \Big\|P(s'|s,a) - \xi(s)\Big\|_1 \frac{\Delta U^{\pi}_{\tau}}{2} \\
&\leq \gamma|\tau - \tau'| D^{P,\xi}_{\infty} \frac{\Delta U^{\pi}_{\tau}}{2}.
\end{aligned}$$

Combining all these results we get the lemma. □

Lemma 6.2 *Let (P_τ, π) and $(P_{\tau'}, \pi')$ be two model-policy pairs, it holds that:*

$$\frac{\Delta A^{\tau',\pi'}_{\tau,\pi}}{2} \leq (D^{\pi',\pi}_\infty + \gamma|\tau - \tau'|D^{P,\xi}_\infty)\frac{\Delta U^\pi_\tau}{2},$$

where $D^{P,\xi}_\infty = \sup_{s\in\mathcal{S},a\in\mathcal{A}}\|P(\cdot|s,a) - \xi(\cdot)\|_1$, $D^{\pi',\pi}_\infty = \sup_{s\in\mathcal{S}}\|\pi'(\cdot|s) - \pi(\cdot|s)\|_1$ *and* $\Delta U^\pi_\tau = \sup_{s,s',\overline{s},\overline{s}'\in\mathcal{S},a,a'\in\mathcal{A}}\big|U^\pi_\tau(s',a',\overline{s}') - U^\pi_\tau(s,a,\overline{s})\big|$

Proof. Let us rewrite the expression of the relative advantage $A^{\tau',\pi'}_{\tau,\pi}$ (Equation 5.27):

$$\begin{aligned}
A^{\tau',\pi'}_{\tau,\pi}(s) &= \sum_{a\in\mathcal{A}}\pi'(a|s)\big(R_{\tau'}(s,a) + \gamma\sum_{s'\in\mathcal{S}}P_{\tau'}(s'|s,a)V^\pi_\tau(s')\big) - V^\pi_\tau(s)\\
&= \sum_{a\in\mathcal{A}}\pi'(a|s)\big(R_{\tau'}(s,a) + \gamma\sum_{s'\in\mathcal{S}}P_{\tau'}(s'|s,a)V^\pi_\tau(s')\big)+\\
&\qquad - \sum_{a\in\mathcal{A}}\pi(a|s)\big(R_\tau(s,a) + \gamma\sum_{s'\in\mathcal{S}}P_\tau(s'|s,a)V^\pi_\tau(s')\big)\\
&= \gamma\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\Big(\pi'(a|s)P_{\tau'}(s'|s,a) - \pi(a|s)P_\tau(s'|s,a) \pm \pi'(a|s)P_\tau(s'|s,a)\Big)V^\pi_\tau(s')+\\
&\qquad + \sum_{a\in\mathcal{A}}\pi'(a|s)R_{\tau'}(s,a) - \pi(a|s)R_\tau(s,a) \pm \pi'(a|s)R_\tau(s,a)\\
&= \sum_{a\in\mathcal{A}}\big(\pi'(a|s) - \pi(a|s)\big)R_\tau(s,a) + \sum_{a\in\mathcal{A}}\big(R_{\tau'}(s,a) - R_\tau(s,a)\big)\pi'(a|s)+\\
&\qquad \gamma\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\big(\pi'(a|s) - \pi(a|s)\big)P_\tau(s'|s,a)V^\pi_\tau(s')+\\
&\qquad + \gamma\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\pi'(a|s)\big(P_{\tau'}(s'|s,a) - P_\tau(s'|s,a)\big)V^\pi_\tau(s')\\
&= \sum_{a\in\mathcal{A}}\big(\pi'(a|s) - \pi(a|s)\big)\big(R_\tau(s,a) + \gamma\sum_{s'\in\mathcal{S}}P_\tau(s'|s,a)V^\pi_\tau(s')\big)+\\
&\qquad + (\tau - \tau')\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\pi'(a|s)\big(P(s'|s,a) - \xi(s')\big)\gamma V^\pi_\tau(s')+\\
&\qquad + (\tau - \tau')\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\pi'(a|s)\big(P(s'|s,a) - \xi(s')\big)R(s,a,s') =\\
&= \sum_{a\in\mathcal{A}}\big(\pi'(a|s) - \pi(a|s)\big)Q^\pi_\tau(s,a)+\\
&\qquad (\tau - \tau')\sum_{s'\in\mathcal{S}}\sum_{a\in\mathcal{A}}\pi'(a|s)\big(P(s'|s,a) - \xi(s')\big)U^\pi_\tau(s.a.s'),
\end{aligned}$$

then we can obtain the proof noticing that $sup_{s\in\mathcal{S}}\Delta Q^{\pi}_{\tau}(s,\cdot) \leq \Delta Q^{\pi}_{\tau}$ and recalling that $\Delta Q^{\pi}_{\tau} \leq \Delta U^{\pi}_{\tau}$:

$$\begin{aligned}\frac{\Delta A^{\tau',\pi'}_{\tau,\pi}}{2} &\leq \|A^{\tau',\pi'}_{\tau,\pi}\|_{\infty} \leq D^{\pi',\pi}_{\infty}\frac{\sup_{s\in\mathcal{S}}\Delta Q^{\pi}_{\tau}(s,\cdot)}{2} + \gamma|\tau-\tau'|D^{P,\xi}_{\infty}\frac{\Delta U^{\pi}_{\tau}}{2}\\ &\leq (D^{\pi',\pi}_{\infty} + \gamma|\tau-\tau'|D^{P,\xi}_{\infty})\frac{\Delta U^{\pi}_{\tau}}{2}.\end{aligned}$$

□

Theorem 6.4 *For any policy $\overline{\pi} \in \Pi$ and teleport probability model $P_{\tau} \in \mathcal{P}$, the lower bound is optimized for:*

$$\alpha^{\star},\tau^{\star} = \arg\max_{\alpha,\tau'}\{B(\alpha,\tau') : (\alpha,\tau') \in \mathcal{V}\},$$

where $\mathcal{V} = \{(\overline{\alpha}_{\tau},\tau,(0,\overline{\tau}_0),(\overline{\alpha}_0,0),(1,\overline{\tau}_1)\}$. The candidate coefficient values are presented in Table 6.1, with their corresponding Teleport Bound values shown in Table 6.2.

Proof. We can start by explicitly writing update coefficients in the Teleport Bound:

$$\begin{aligned}J^{\pi'}_{\mu,\tau'} - J^{\pi}_{\mu,\tau} \geq \frac{\alpha\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu} + (\tau-\tau')\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} - \frac{\gamma\Delta U^{\pi}_{\tau}}{2(1-\gamma)^2}\Big(\alpha^2 D^{\pi',\pi}_{\mathbb{E}}D^{\pi',\pi}_{\infty} + \alpha|\tau-\tau'|D^{\pi',\pi}_{\mathbb{E}}D^{P,\xi}_{\infty} +\\ + \alpha|\tau-\tau'|D^{P,\xi}_{\mathbb{E}}D^{\pi',\pi}_{\infty} + \gamma(\tau-\tau')^2 D^{P,\xi}_{\mathbb{E}}D^{P,\xi}_{\infty}\Big),\end{aligned}$$

taking now partial derivatives w.r.t. α and τ' we can find the following stationary points:

$$\begin{aligned}\frac{\partial B}{\partial\alpha} &= \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{1-\gamma} - \frac{\gamma\Delta U^{\pi}_{\tau}}{2(1-\gamma)^2}\Big(2\alpha D^{\pi',\pi}_{\mathbb{E}}D^{\pi',\pi}_{\infty} + (\tau-\tau')\big(D^{\pi',\pi}_{\mathbb{E}}D^{P,\xi}_{\infty} + D^{P,\xi}_{\mathbb{E}}D^{\pi',\pi}_{\infty}\big)\Big),\\ \frac{\partial B}{\partial\tau'} &= -\frac{\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} + \frac{\gamma\Delta U^{\pi}_{\tau}}{2(1-\gamma)^2}\Big(2\gamma(\tau-\tau')D^{P,\xi}_{\mathbb{E}}D^{P,\xi}_{\infty} + \alpha\big(D^{\pi',\pi}_{\mathbb{E}}D^{P,\xi}_{\infty} + D^{P,\xi}_{\mathbb{E}}D^{\pi',\pi}_{\infty}\big)\Big).\end{aligned}$$

When the current policy is not set to target or symmetrically the current value of *tau* is not set to 0 the linear system of derivatives has a single solution. To determine the properties of such a point, we examine the second-order derivatives:

$$\begin{aligned}\frac{\partial B^2}{\partial^2\alpha} &= -\frac{\gamma}{(1-\gamma)^2}D^{\pi',\pi}_{\mathbb{E}}D^{\pi',\pi}_{\infty}\Delta U^{\pi}_{\tau},\\ \frac{\partial B^2}{\partial\alpha\partial\tau'} = \frac{\partial B^2}{\partial\tau'\partial\alpha} &= \frac{\gamma\Delta U^{\pi}_{\tau}}{2(1-\gamma)^2}\big(D^{\pi',\pi}_{\mathbb{E}}D^{P,\xi}_{\infty} + D^{P,\xi}_{\mathbb{E}}D^{\pi',\pi}_{\infty}\big),\\ \frac{\partial B^2}{\partial^2\tau'} &= -\frac{\gamma^2}{(1-\gamma)^2}D^{P,\xi}_{\mathbb{E}}D^{P,\xi}_{\infty}\Delta U^{\pi}_{\tau}.\end{aligned}$$

Writing then the Hessian matrix $\mathcal{H}(\alpha,\tau)$:

$$\mathcal{H}(\alpha,\tau) = -\frac{\gamma\Delta U_\tau^\pi}{2(1-\gamma)^2}\begin{pmatrix} 2D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi} & -D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{P,\xi} - D_{\mathbb{E}}^{P,\xi}D_{\infty}^{\pi',\pi} \\ -D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{P,\xi} - D_{\mathbb{E}}^{P,\xi}D_{\infty}^{\pi',\pi} & 2\gamma D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi} \end{pmatrix},$$

with trace and determinant:

$$tr(\mathcal{H}(\alpha,\tau)) = -\frac{\gamma\Delta U_\tau^\pi}{2(1-\gamma)^2}\Big(2D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi} + 2\gamma D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi}\Big) \leq 0,$$

$$\begin{aligned} det(\mathcal{H}(\alpha,\tau)) &= \frac{\gamma^2\Delta U_\tau^{\pi^2}}{4(1-\gamma)^4}\Big((4\gamma-2)D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi}D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi} + \\ &\quad - (D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{P,\xi})^2 - (D_{\infty}^{\pi',\pi}D_{\mathbb{E}}^{P,\xi})^2\Big) \leq \\ &\leq \frac{\gamma^2\Delta U_\tau^{\pi^2}}{4(1-\gamma)^4}\Big(2D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi}D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi} + \\ &\quad - (D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{P,\xi})^2 - (D_{\infty}^{\pi',\pi}D_{\mathbb{E}}^{P,\xi})^2\Big) \leq \\ &\leq -\frac{\gamma^2\Delta U_\tau^{\pi^2}}{4(1-\gamma)^4}\Big(D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{P,\xi} - D_{\infty}^{\pi',\pi}D_{\mathbb{E}}^{P,\xi}\Big)^2 \leq 0. \end{aligned}$$

When $\pi \neq \overline{\pi}$ and $\tau \neq 0$ we find that both trace and determinant are non-positive. This indicates that the Hessian is negative semi-definite, implying that the previously identified unique stationary point is a saddle point, not relevant for optimization purposes. However, being our function $B(\alpha,\tau')$ continuous and quadratic on a compact set, the existence of global maximum and minimum is ensured by the Weierstrass theorem. As the saddle point cannot be the maximum, this optimal point must lie on the boundaries $[0,1]\times[0,\tau]$. Then setting to zero the equations $\frac{\partial B}{\partial \tau'}\big|_{\alpha=0}$, $\frac{\partial B}{\partial \tau'}\big|_{\alpha=1}$, $\frac{\partial B}{\partial \alpha}\big|_{\tau'=0}$, $\frac{\partial B}{\partial \alpha}\big|_{\tau'=\tau}$ we get the following candidate optimal pairs:

$$\begin{aligned} \overline{\alpha}_\tau &= \frac{(1-\gamma)\mathbb{A}_{\tau,\pi,\mu}^{\tau,\pi'}}{\gamma\Delta U_\tau^\pi D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi,}} \\ \overline{\tau}'_0 &= \tau - \frac{(1-\gamma)\mathbb{A}_{P,\tau,\mu}^{\xi,\pi}}{\gamma^2\Delta U_\tau^\pi D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi}}, \\ \overline{\alpha}_0 &= \frac{(1-\gamma)\mathbb{A}_{\tau,\pi,\mu}^{\tau,\pi'}}{\gamma\Delta U_\tau^\pi D_{\mathbb{E}}^{\pi',\pi}D_{\infty}^{\pi',\pi,}} - \frac{\tau}{2}\Big(\frac{D_{\mathbb{E}}^{P,\xi}}{D_{\mathbb{E}}^{\pi',\pi}} + \frac{D_{\infty}^{P,\xi}}{D_{\infty}^{\pi',\pi}}\Big), \\ \overline{\tau}'_1 &= \tau - \frac{(1-\gamma)\mathbb{A}_{P,\tau,\mu}^{\xi,\pi}}{\gamma^2\Delta U_\tau^\pi D_{\mathbb{E}}^{P,\xi}D_{\infty}^{P,\xi}} + \frac{1}{2\gamma}\Big(\frac{D_{\mathbb{E}}^{\pi',\pi}}{D_{\mathbb{E}}^{P,\xi}} + \frac{D_{\infty}^{\pi',\pi}}{D_{\infty}^{P,\xi}}\Big). \end{aligned}$$

If either the policy or the model matches the target, with $\gamma \in (0,1)$, the Hessian matrix becomes singular. In both cases, this condition leads to a maximum point. Specifically,

when $\tau = 0$, we have $\alpha^\star = \overline{\alpha}_0$. Conversely, when $\pi = \overline{\pi}$, we find $\tau^\star = \overline{\tau}'_1$. Finally, by evaluating the Teleport Bound (Theorem 6.3) at the optimal candidate pairs, we obtain:

$$
\begin{aligned}
B(\overline{\alpha}_\tau, \tau) &= \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}{}^2}{2\gamma \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi,}_{\infty}} \\
B(0, \overline{\tau}'_0) &= \frac{\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}{}^2}{2\gamma^2 \Delta U^\pi_\tau D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}} \\
B(\overline{\alpha}_0, 0) &= \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}{}^2}{2\gamma \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi,}_{\infty}} + \frac{\tau \mathbb{A}^{\xi,\pi}_{P,\tau,\mu}}{1-\gamma} - \frac{\tau \mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{2(1-\gamma)} \Big(\frac{D^{P,\xi}_{\infty}}{D^{\pi',\pi}_{\infty}} + \frac{D^{P,\xi}_{\mathbb{E}}}{D^{\pi',\pi}_{\mathbb{E}}} \Big) + \\
&\quad - \frac{\gamma \tau^2 \Delta U^\pi_\tau}{2(1-\gamma)^2} \left(\gamma D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty} - \frac{\big(D^{\pi',\pi}_{\infty} D^{P,\xi}_{\mathbb{E}} + D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \big)^2}{4 D^{\pi',\pi}_{\infty} D^{\pi',\pi}_{\mathbb{E}}} \right), \\
B(1, \overline{\tau}'_1) &= \frac{\mathbb{A}^{\xi,\pi}_{P,\tau,\mu}{}^2}{2\gamma^2 \Delta U^\pi_\tau D^{P,\xi}_{\mathbb{E}} D^{P,\xi}_{\infty}} + \frac{\mathbb{A}^{\tau,\pi'}_{\tau,\pi,\mu}}{1-\gamma} + \frac{\Delta U^\pi_\tau}{8(1-\gamma)^2} \frac{\big(D^{\pi',\pi}_{\infty} D^{P,\xi}_{\mathbb{E}} + D^{\pi',\pi}_{\mathbb{E}} D^{P,\xi}_{\infty} \big)^2}{D^{P,\xi}_{\infty} D^{P,\xi}_{\mathbb{E}}} + \\
&\quad - \frac{1}{2\gamma(1-\gamma)} \left(\mathbb{A}^{\xi,\pi}_{P,\tau,\mu} \Big(\frac{D^{\pi',\pi}_{\infty}}{D^{P,\xi}_{\infty}} + \frac{D^{\pi',\pi}_{\mathbb{E}}}{D^{P,\xi}_{\mathbb{E}}} \Big) + \frac{\gamma^2 \Delta U^\pi_\tau D^{\pi',\pi}_{\mathbb{E}} D^{\pi',\pi}_{\infty}}{1-\gamma} \right)
\end{aligned}
$$

□

B | Additional Experiments

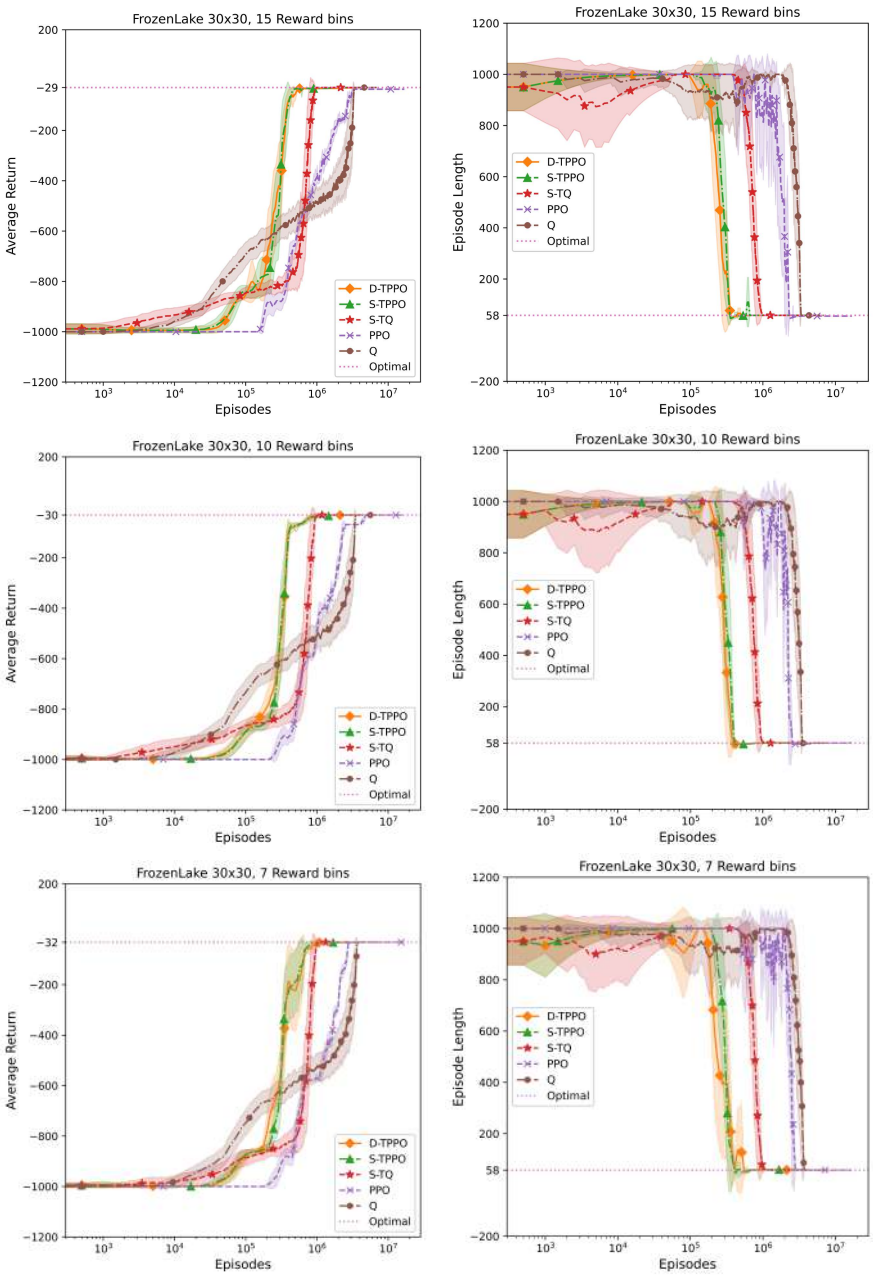


Figure B.1: Frozen Lake average return and episode len 15, 10 and 7 bins

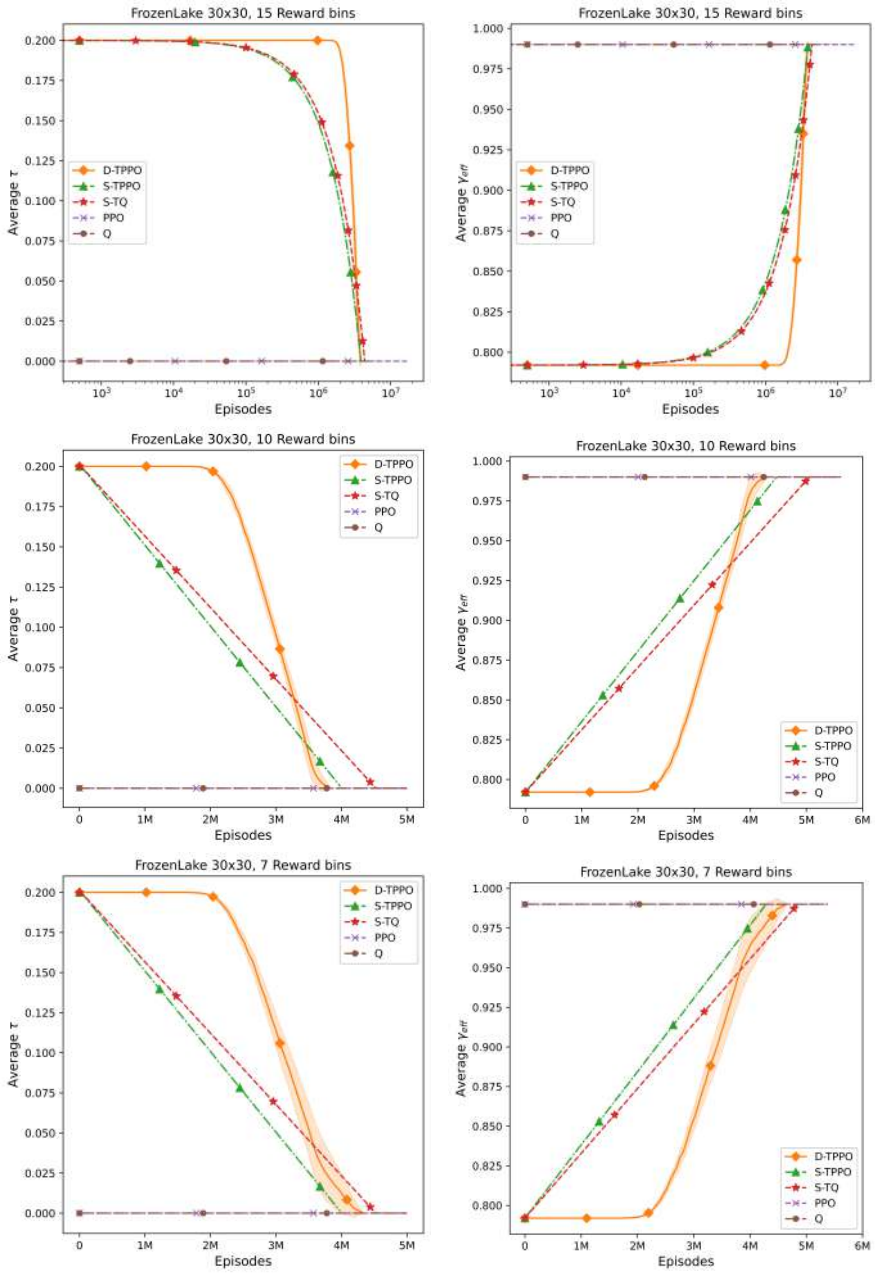


Figure B.2: Frozen Lake average tau and effective gamma 15, 10 and 7 bins

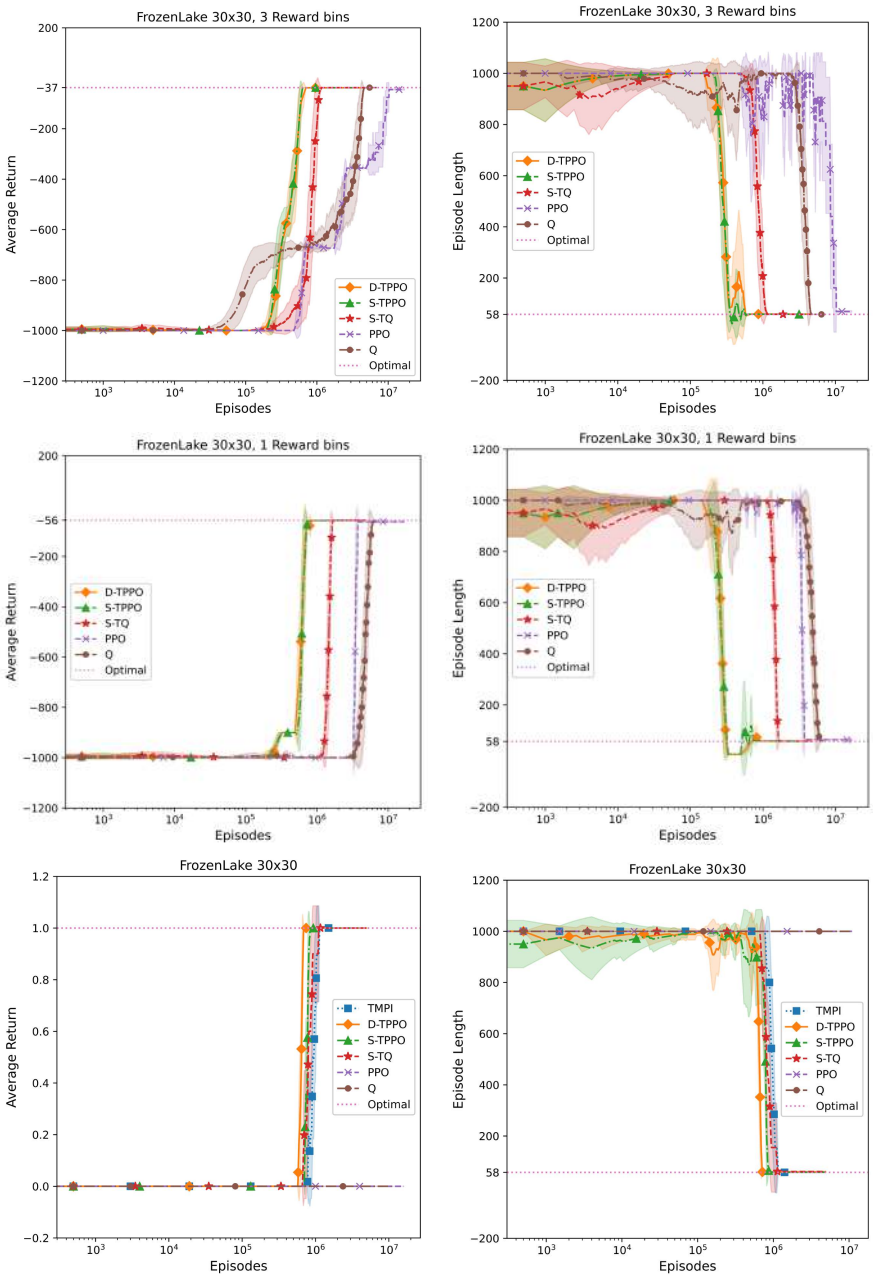


Figure B.3: Frozen Lake average return and episode len 3, 1 and 0 bins

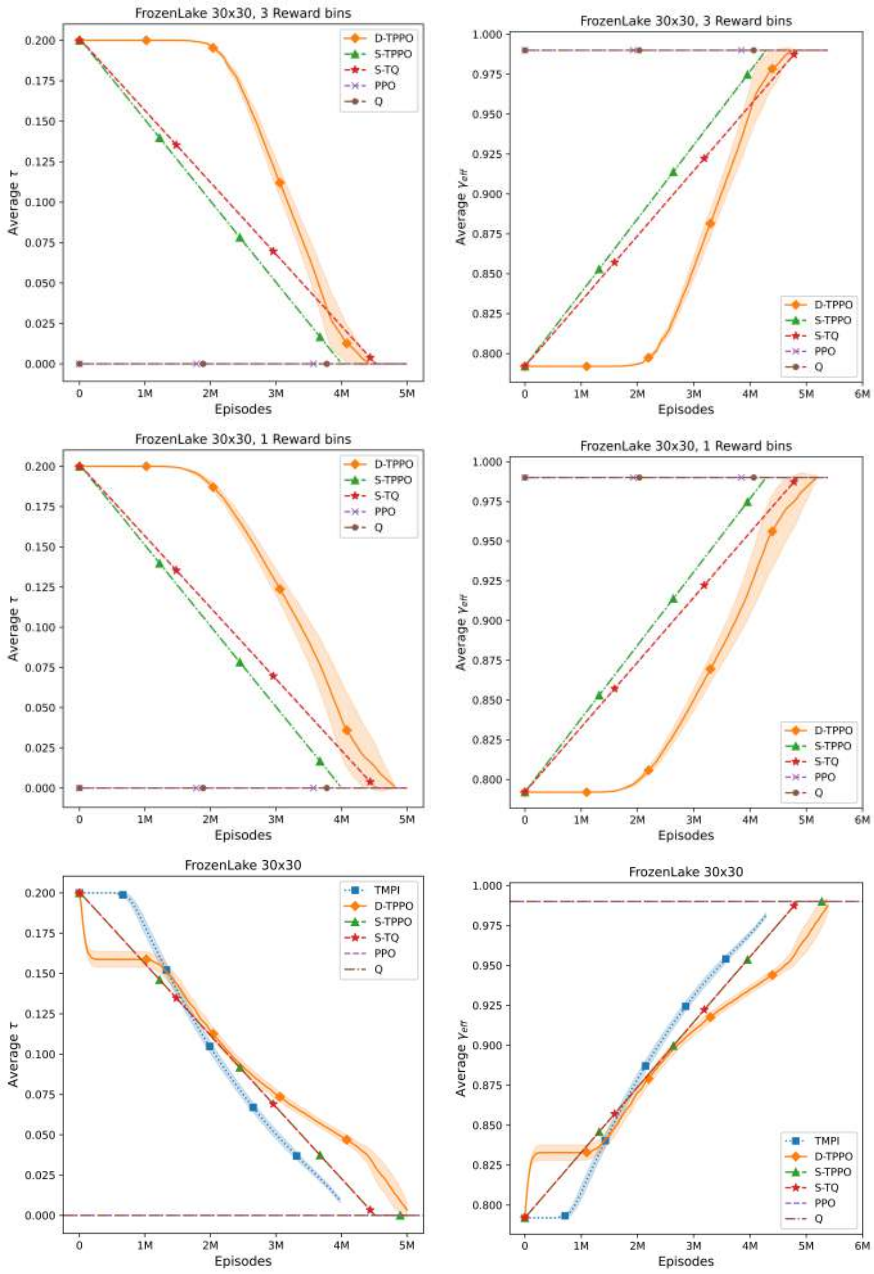


Figure B.4: Frozen Lake average tau and effective gamma 3, 1 and 0 bins

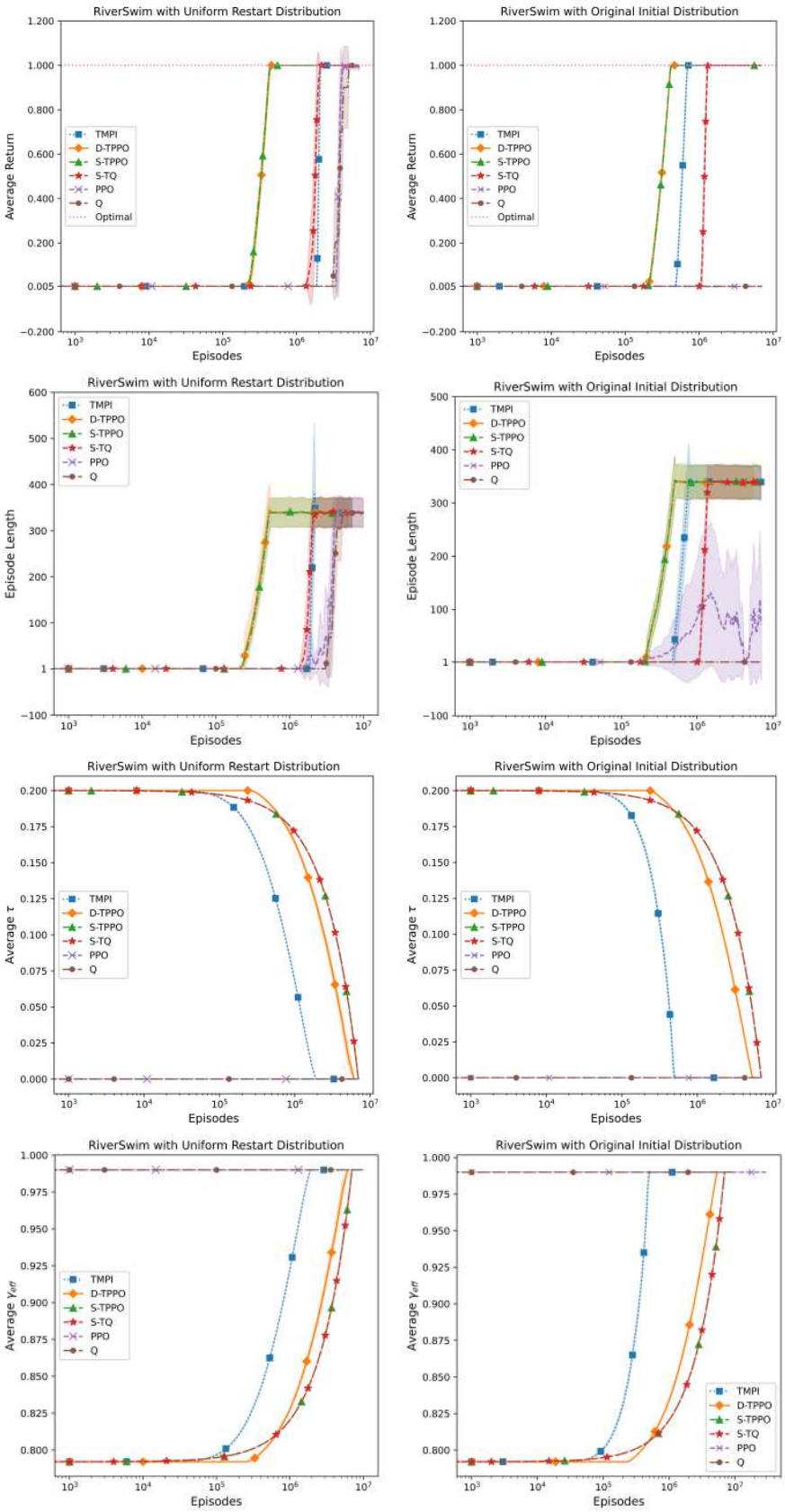


Figure B.5: Uniform restart (left), original initial distribution (right).

List of Figures

2.1 Supervised Learning examples 6
2.2 Unsupervised Learning examples 7
2.3 Agent-Environment Interaction 8

5.1 Chain MDP with 5 states 62

7.1 Frozen Lake Environment 88
7.2 Reward shaping bins 89
7.3 10 Bins Frozen Lake average return and episode length 90
7.4 10 Bins Frozen Lake teleport rate and effective gamma 91
7.5 3 Bins Frozen Lake average return and episode length 91
7.6 15 Bins Frozen Lake average return and episode length 92
7.7 Frozen Lake local optima vanilla PPO 92
7.8 0 Bins Frozen Lake average return and episode length 93
7.9 0 Bins Frozen Lake teleport rate and effective gamma 94
7.10 River Swim MDP with 5 states 94
7.11 Average Return River Swim 96
7.12 Average Teleport Rate River Swim 96

B.1 Frozen Lake average return and episode len 15, 10 and 7 bins 119
B.2 Frozen Lake average tau and effective gamma 15, 10 and 7 bins 120
B.3 Frozen Lake average return and episode len 3, 1 and 0 bins 121
B.4 Frozen Lake average tau and effective gamma 3, 1 and 0 bins 122
B.5 River Swim Results 123

List of Tables

6.1 Candidate optimal pairs for (α, τ') 79
6.2 Teleport bound candidate values. 80

7.1 Frozen Lake hyperparameters 90
7.2 River Swim hyperparameters 95

List of Algorithms

3.1 Q-learning 34
3.2 Actor-Only Policy Gradient 41
3.3 Monte Carlo PGT 41
3.4 Actor-Critic Policy Gradient 42
3.5 Trust Region Policy Optimization (TRPO) 46
3.6 Proximal Policy Optimization (PPO) 47
6.1 Teleport Model Policy Iteration (TMPI) 81
6.2 Static Teleport Curriculum (S-T) 85
6.3 Dynamic Teleport Curriculum (D-T) 86

List of Symbols

Variable	Description
$\mathcal{M}$	Markov Decision Process
$\mathcal{T}$	Teleport Markov Decision Process
$\mathcal{S}$	state space
$\mathcal{A}$	action space
P	transition probability matrix
R	reward function
γ	discount factor
μ	initial state distribution
π	policy
θ	policy parameters
π_θ	parametric policy
V^π	value function
Q^π	action-value function
A^π	advantage function
$J(\pi), J^\pi$	policy-dependent performance index
T^π	Bellman expectation operator
$T^\star$	Bellman optimality operator
$L(\hat{V})$	value function root mean squared error
α	learning rate
N	batch size
δ_t	temporal difference error
$\lambda, \lambda_{e.t.}$	Bias variance trade off for eligibility traces
τ	teleport probability
P_τ	teleport transition probability function
ξ	state teleport probability distribution

Ξ	state teleport probability distribution matrix
V_τ^π	state value function under teleport transition model
Q_τ^π	state-action value function under teleport transition model
U_τ^π	state-action-next state value function
$J_{\mu,\tau}^\pi$	performance of a model-policy pair
ρ_τ^π	distribution of trajectories under teleport transition function
R_τ	expected reward under teleport transition model
$d_{\mu,\tau}^\pi$	γ-discounted state distribution
$\delta_{\mu,\tau}^\pi$	γ-discounted state-action distribution
$A_\tau^\pi(s,a)$	policy advantage function
$A_\tau^\pi(s,a,s')$	model advantage function
$A_{\tau,\pi}^{\tau,\pi'}$	relative policy advantage function
$A_{\tau,\pi}^{\tau',\pi}$	relative model advantage function
$\mathbb{A}_{\pi,\mu}^{\pi',\tau}$	expected relative policy advantage function
$\mathbb{A}_{\tau,\mu}^{\tau',\pi}$	expected relative model advantage function
$\tilde{A}_\tau^\pi$	kernel advantage function
$A_{\tau,\pi}^{\tau',\pi'}$	coupled relative advantage function
$\mathbb{A}_{\tau,\pi,\mu}^{\tau',\pi'}$	expected coupled relative advantage function
$D_{\mathbb{E}}^{P_{\tau'}^{\pi'},P_\tau^\pi}$	state kernel dissimilarity
$D_{\mathbb{E}}^{\pi',\pi}$	policy expected dissimilarity term
$D_{\mathbb{E}}^{P,\xi}$	model expected dissimilarity term
$D_{\infty}^{\pi',\pi}$	policy dissimilarity term
$D_{\infty}^{P,\xi}$	model dissimilarity term
$\Delta A_{\tau,\pi}^{\tau',\pi'}$	maximum advantage function difference
ΔU_τ^π	maximum state-action-next state value function difference
$\mathcal{V}^{(t)}$	set of candidate pairs
$\alpha^\star$	optimal policy update factor
$\tau^\star$	optimal teleport probability update
$B(\alpha,\tau)$	Teleport bound value for candidate pairs
ϵ_{PPO}	PPO clipping threshold
TMPI	Teleport Model Policy Iteration
S-T	Static Teleport curriculum
D-T	Dynamic Teleport curriculum

Acknowledgements

Quando si vive la vita come una partita di scacchi, si è sempre concentrati sulla prossima mossa, talvolta perdendo di vista ciò che ci ha portati fin qui. È in momenti come questi, però, che volgere uno sguardo al passato diventa importante. La continua ricerca di crescita personale passa anche dal saper riconoscere i propri meriti, ma soprattutto, quelli altrui.

Vorrei pertanto ringraziare il Prof. Marcello Restelli, il Dott. Alberto Maria Metelli e il Dott. Luca Sabbioni, la vostra sapiente guida ha delineato l'intero percorso, rendendolo di fatto realizzabile.

In secondo luogo, non per importanza, la mia famiglia e mio fratello. Mi siete sempre stati vicini da quando ho aperto gli occhi, se sono quello che sono oggi lo devo a voi. Un ringraziamento speciale spetta a mio nonno Livio: senza di te tutto questo non sarebbe stato possibile. Te ne sei andato prima che questo giorno arrivasse, ma il tuo ricordo e il tuo sostegno mi hanno accompagnato fin qui. Marco, tu invece mi hai trasmesso la passione per un mondo che mi sembrava lontano anni luce, rivelatosi poi un vero e proprio stile di vita.

Meri, tu sei stata la mia bussola, mi hai fatto ritrovare la rotta nel mare in tempesta. Non potrò mai ringraziarti abbastanza.

Infine, vorrei ringraziare gli amici di sempre, i compagni universitari e il mio coinquilino con i quali ho condiviso ricordi che rimarranno indelebili a vita.

Grazie.

Cristian

Milano, 16 Luglio 2024